



SAPIENZA
UNIVERSITÀ DI ROMA

Automi, calcolabilità e complessità

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF. DANIELE VENTURI

2 gennaio 2025



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Questo testo è stato integrato con il libro [1].

Indice

Prefazione	1
Introduzione al corso	5
 I Automi e linguaggi	 7
1 Linguaggi regolari	9
1.1 Automi finiti	9
1.1.1 Definizione formale di computazione	11
1.1.2 Progettare automi finiti	12
1.1.3 Le operazioni regolari	12
1.2 Non determinismo	14
1.2.1 Definizione formale di automa finito non deterministico	15
1.2.2 Equivalenza tra gli NFA e i DFA	16
1.2.3 Chiusura rispetto alle operazioni regolari	18
1.3 Espressioni regolari	20
1.3.1 Equivalenza con gli automi finiti	21
1.4 Linguaggi non regolari	27
1.4.1 Il pumping lemma per i linguaggi regolari	27
1.5 Esercizi svolti	29
 2 Linguaggi context-free	 31
2.1 Grammatiche context-free	31
2.1.1 Progettare grammatiche context-free	32
2.1.2 Ambiguità	33
2.1.3 Forma normale di Chomsky	34
2.2 Automi a pila	36
2.2.1 Definizione formale di automa a pila	36
2.2.2 Equivalenza con le grammatiche context-free	38
2.3 Linguaggi non context-free	42
2.3.1 Pumping lemma per i linguaggi context-free	43
 II Teoria della computabilità	 49
3 La tesi di Church-Turing	51
3.1 Macchine di Turing	51
3.1.1 Definizione formale di macchina di Turing	51
3.2 Varianti di macchine di turing	54
3.2.1 Macchina di Turing multinastro	54
3.2.2 Macchina di Turing non deterministica	55
3.2.3 Enumeratori	56

4	Dedicibilità	57
4.1	Linguaggi decidibili	57
4.1.1	Problemi decidibili relativi a linguaggi regolari	57
4.1.2	Problemi decidibili relativi a linguaggi context-free	59
4.2	Indecidibilità	60
4.2.1	Metodo della diagonalizzazione	61
4.2.2	Linguaggio indecidibile	62
4.2.3	Linguaggio non Turing-riconoscibile	63
5	Riducibilità	65
5.1	Problemi indecidibili dalla teoria dei linguaggi	65
5.2	Riducibilità mediante funzione	67
5.2.1	Funzioni calcolabili	67
5.2.2	Definizione formale di riducibilità mediante funzione	67
5.3	Esercizi svolti	70
III	Teoria della complessità	73
6	Complessità di tempo	75
6.1	Misure di complessità	75
6.1.1	Notazione O-grande ed o-piccola	75
6.2	Classe P	77
6.3	Classe NP	78
6.4	NP-completezza	80
6.4.1	Riducibilità in tempo polinomiale	82
6.4.2	Classe NP-Complete	83
6.4.3	Teorema di Cook-Levin	84
6.4.4	Classi coP, coNP e coEXP	87
7	Complessità spaziale	91
7.1	Rapporto tra spazio e tempo	93
7.2	Teorema di Savitch	94
7.3	Classe PSPACE	95
7.4	Classe L	97
7.5	Riduzione in spazio logaritmico	98
7.6	Classe NL-Complete	100
7.6.1	Teorema di Immerman-Szelepcsényi	101
	Bibliografia	105

Introduzione al corso

Il corso verte su tre temi principali: Automi, Calcolabilità e Complessità. Si parte dal concetto di automi come modelli semplici di calcolo e si passa a trattare problemi di calcolabilità e risorse computazionali (tempo e spazio). Infine, viene introdotta la complessità computazionale, che cerca di distinguere problemi risolvibili in modo efficiente da quelli che non lo sono.

Parte I

Automati e linguaggi

Capitolo 1

Linguaggi regolari

1.1 Automi finiti

Il primo modello di calcolo studiato è quello degli automi a stati finiti (DFA - Deterministic Finite Automata). Si tratta di modelli semplici con memoria limitata. Sono spesso utilizzati per risolvere problemi concreti, come il riconoscimento di pattern.

Esempio 1.1.1 – Sensore di una porta automatica

Il sistema di controllo per una porta automatica è un esempio di automa a stati finiti.

Una porta automatica ha un sensore davanti per rilevare la presenza di una persona che sta per attraversare la soglia. Un altro sensore è collocato dall'altro lato della soglia in modo che il sistema di controllo possa mantenere aperta la porta abbastanza a lungo da permettere alla persona di attraversarla e anche per evitare che la porta colpisca qualcuno che è dietro di essa quando si apre.

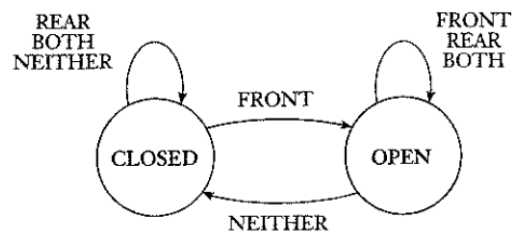


Figura 1.1: Diagramma di stato per un sistema di controllo di una porta automatica. Il sistema di controllo è in uno dei due stati: "OPEN" o "CLOSED," che rappresentano la condizione corrispondente della porta. Ci sono quattro possibili condizioni di input: "FRONT" (il cui significato è che una persona è sul sensore davanti alla soglia), "REAR" (significa che una persona è sul sensore dietro alla soglia), "BOTH" (significa che vi sono persone su entrambi i sensori), e "NEITHER" (significa che nessuno è su alcun sensore).

stato/input	NEITHER	FRONT	REAR	BOTH
CLOSED	CLOSED	OPEN	CLOSED	CLOSED
OPEN	CLOSED	OPEN	OPEN	OPEN

Tabella 1.1: Tabella delle transizioni di stato per il sistema di controllo di una porta automatica

Definizione 1.1.1 – Automa finito

In una prospettiva matematica, un DFA è definito come una tupla $(Q, \Sigma, \delta, q_0, F)$ dove:

1. Q è un insieme finito chiamato insieme degli stati;
2. Σ è un insieme finito chiamato alfabeto di input;
3. $\delta : Q \times \Sigma \rightarrow Q$ è la funzione di transizione;

4. $q_0 \in Q$ è lo stato iniziale;
5. $F \subseteq Q$ è l'insieme degli stati finali o di accettazione.

Il DFA accetta una stringa se, partendo dallo stato iniziale e seguendo le transizioni dettate dalla stringa, termina in uno stato finale.

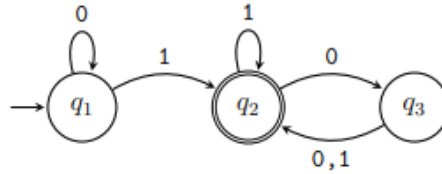


Figura 1.2: Automa finito M_1

La Figura 1.2 mostra il diagramma di stato di un automa M_1 . Esso ha tre stati, etichettati q_1 , q_2 e q_3 . Lo stato iniziale, q_1 è indicato dall'arco entrante in esso e non uscente da uno stato. Lo stato accettante, q_2 , è quello con un doppio cerchio. Gli archi che vanno da uno stato a un altro sono chiamati transizioni.

Quando l'automa riceve una stringa in input esso elabora questa stringa e produce un output. L'elaborazione inizia nello stato iniziale di M_1 . L'automa riceve i simboli della stringa di input uno a uno da sinistra a destra. Dopo aver letto ciascun simbolo, M_1 si muove da uno stato a un altro lungo la transizione che ha quel simbolo come etichetta. Quando legge l'ultimo simbolo, M_1 produce il suo output. L'output è "accetta" se M_1 è, in quel momento, in uno stato accettante e "rifiuta" se non lo è.

Possiamo descrivere M_1 formalmente ponendo $M_1 = (Q, \Sigma, \delta, q_1, F)$ dove:

1. $Q = \{q_1, q_2, q_3\}$;
2. $\Sigma = \{0, 1\}$;
3. $d =$

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2
4. q_1 è lo stato iniziale;
5. $F = \{q_2\}$.

Se A è l'insieme di tutte le stringhe che la macchina M accetta, diciamo che A è il linguaggio della macchina M e scriviamo $L(M) = A$. Diciamo che M riconosce A o che M accetta A .

Una macchina può accettare diverse stringhe, ma riconosce sempre soltanto un linguaggio. Se la macchina non accetta alcuna stringa, essa riconosce ancora un linguaggio ossia, il linguaggio vuoto $\emptyset$. Nel nostro automa 1.2, sia $A = \{x \mid x \text{ contiene almeno un } 1 \text{ e un numero pari di } 0 \text{ segue l'ultimo } 1\}$, allora $L(M_1) = A$, o equivalentemente, M_1 riconosce A .

Esempio 1.1.2 – Automa finito M_2

Ecco il diagramma di stato di un automa finito M_2 :

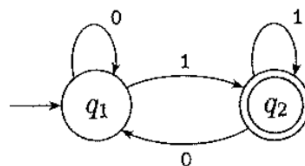


Figura 1.3: Diagramma di stato per un automa finito M_2

Nella descrizione formale, M_2 è $(\{q_1, q_2\}, \{0, 1\}, \delta, q_1, \{q_2\})$. La funzione di transizione δ

	0	1
è: q_1	q_1	q_2
q_2	q_1	q_2

M_2 accetta tutte le stringhe che terminano con un 1. Quindi $L(M_2) = \{w | w \text{ termina con un } 1\}$.

1.1.1 Definizione formale di computazione

Per definire il linguaggio associato di un automa si usa la funzione di transizione estesa: $\delta^* : Q \times \Sigma^* \rightarrow Q^1$, definita ricorsivamente come:

$$\begin{cases} \delta^*(q, \epsilon) = \delta(q, \epsilon) = q & \text{dove } \epsilon \text{ rappresenta la stringa vuota} \\ \delta^*(q, ax) = \delta^*(\delta(q, a), x) & \text{dove } a \in \Sigma, x \in \Sigma^* \end{cases}$$

Definiamo la coppia $(q, x) \in Q \times \Sigma^*$ come configurazione di un DFA che indica lo stato attuale e la parte ancora da leggere.

Definiamo come passo di computazione la relazione binaria $\vdash_D$ definita come $(p, ax) \vdash_D (q, x) \iff \delta(p, a) = q$, dove $p, q \in Q, a \in \Sigma$ e $x \in \Sigma^*$. Posso estendere $\vdash_D$ a $\vdash_D^*$, ottenuta aggiungendo tutte le coppie in $Q \times \Sigma^*$ che rendono $\vdash_D$ chiusa per riflessività e transitività. In particolare la riflessività mi dice che $\forall q \in Q, x \in \Sigma^*, (q, x) \vdash_D^* (q, x)$, e la transitività mi dice che se $(q, abx) \vdash_D (p, bx) \wedge (p, bx) \vdash_D (r, x) \implies (q, abx) \vdash_D^* (r, x)$.

Sia $M = (Q, \Sigma, \delta, q_0, F)$ un automa finito e sia $w = w_1 w_2 \dots w_n$ una stringa dove ogni w_i è un elemento dell'alfabeto Σ . Allora M accetta w se esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, per $i=0, \dots, n-1$, e
3. $r_n \in F$.

La condizione 1 afferma che la macchina inizia nello stato iniziale. La condizione 2 afferma che la macchina passa da stato a stato in base alla funzione di transizione. La condizione 3 afferma che la macchina accetta il suo input se termina la lettura in uno stato accettante.

Definizione 1.1.2 – Stringa accettata

Una stringa $x \in \Sigma^*$ è accettata da DFA se:

$$\delta^*(q_0, x) \in F$$

Il linguaggio riconosciuto da D è:

$$L(D) = \{x \in \Sigma^* | \delta^*(q_0, x) \in F\}$$

Definizione 1.1.3 – Linguaggi regolari

Un linguaggio è definito regolare se:

$$REG = \{L \subseteq \Sigma^* | \exists \text{ DFA } D \text{ t.c. } L(D) = L\}$$

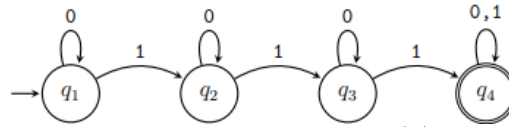
ovvero se un automa finito lo riconosce.

¹ Σ^* descrive il linguaggio che consiste di tutte le stringhe sull'alfabeto Σ .

1.1.2 Progettare automi finiti

Esempio 1.1.3 – Linguaggi regolari

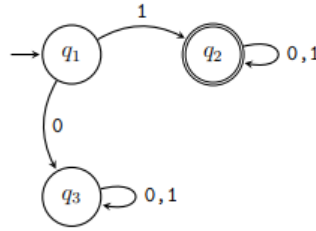
Costruire un DFA per il linguaggio $L(D) = \{x \in \{0,1\}^* | w_H(x) \geq 3\}$.



Se $x \in L$ allora D accetta x , se D accetta x allora si ha $w_H(x) \geq 3$.

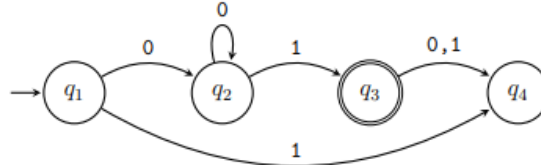
Esempio 1.1.4 – Linguaggi regolari

Costruire un DFA per il linguaggio $L(D) = \{x \in \{0,1\}^* | x = 1y \text{ con } y \in \{0,1\}^*\}$.



Esempio 1.1.5 – Linguaggi regolari

Costruire un DFA per il linguaggio $L(D) = \{x \in \{0,1\}^* | x = 0^n 1 \text{ con } n \in \mathbb{N} - \{0\}\}$.



1.1.3 Le operazioni regolari

Definiamo tre operazioni sui linguaggi, chiamate operazioni regolari, e le usiamo per studiare le proprietà dei linguaggi regolari.

Definizione 1.1.4 – Operazione sui linguaggi

Dati i linguaggi $L, L_1, L_2 \subseteq \Sigma^*$, definiamo le seguenti operazioni:

- *Operatore unione:* $L_1 \cup L_2 = \{x \in \Sigma^* | x \in L_1 \vee x \in L_2\}$;
- *Operatore intersezione:* $L_1 \cap L_2 = \{x \in \Sigma^* | x \in L_1 \wedge x \in L_2\}$;
- *Operatore complemento:* $\neg L = \{x \in \Sigma^* | x \notin L\}$.
- *Operatore concatenazione:* $L_1 \circ L_2 = \{xy \in \Sigma^* | x \in L_1, y \in L_2\}$

L'operazione di concatenazione antepone una stringa di L_1 ad una stringa di L_2 in tutti i modi possibili per ottenere le stringhe nel nuovo linguaggio.

- *Operatore potenza:* $L^n = \begin{cases} \{\epsilon\} & \text{se } n = 0 \\ L \circ L^{n-1} & \text{se } n > 0 \end{cases}$
- *Operatore star:* $L^* = \{x_1 \dots x_k \in \Sigma^* | k \geq 0, \forall i \in [1, k] x_i \in L\}$

L'operazione star opera concatenando un numero qualsiasi di stringhe in L insieme per ottenere una stringa nel nuovo linguaggio. Poiché "un numero qualsiasi" include 0 come possibilità, la stringa vuota è sempre un elemento di L^* , indipendentemente da chi sia L .

Esempio 1.1.6 – Operazione di concatenazione sui linguaggi

Siano $L_1 = \{a, ab, ba\}$ e $L_2 = \{ab, b\}$ allora:

$$L_1 \circ L_2 = \{aab, ab, abab, abb, baab, bab\}$$

Esempio 1.1.7 – Operazione di potenza sui linguaggi

Siano $L_1 = \{a, ab, ba\}$ allora:

$$L^2 = \{aa, aab, aba, abab, abba, baa, baab, baba\}$$

Esempio 1.1.8 – Operazione star sui linguaggi

Sia $L = \{a, b\}$ allora:

$$L^* = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$$

In generale, una classe di oggetti è chiusa rispetto a un'operazione se l'applicazione di questa operazione a elementi della classe restituisce un oggetto ancora nella classe.

Teorema 1.1.1 – Chiusura rispetto all'unione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di unione. In altre parole, se A_1 e A_2 sono linguaggi regolari, lo è anche $A_1 \cup A_2$.

Osservazione. Poiché A_1 e A_2 sono regolari, sappiamo che un automa M_1 riconosce A_1 e un automa finito M_2 riconosce A_2 . Per provare che $A_1 \cup A_2$ è regolare, esibiamo un automa finito, M , che riconosce $A_1 \cup A_2$.

Questa è una prova costruttiva. Costruiamo M da M_1 ed M_2 . Per riconoscere il linguaggio unione, la macchina M deve accettare il suo input esattamente quando M_1 o M_2 lo accetterebbero.

Come possiamo far sì che la macchina M simuli M_1 ed M_2 ? Immagina che tu sia M . Come ricevi i simboli input, uno a uno, tu simuli sia M_1 che M_2 contemporaneamente. In questo modo, è necessario scorrere l'input solo una volta. Ma come puoi tenere traccia di entrambe le simulazioni con una memoria finita? Tutto quello che hai bisogno di ricordare è lo stato in cui ciascuna macchina sarebbe se avesse letto l'input fino a quel punto. Quindi, devi ricordare una coppia di stati. Quante possibili coppie ci sono? Se M_1 ha k_1 stati ed M_2 ha k_2 stati, il numero delle coppie di stati è il prodotto $k_1 \times k_2$. Questo prodotto sarà il numero degli stati in M , uno per ogni coppia. Le transizioni di M vanno da coppia a coppia, aggiornando lo stato corrente sia per la componente in M_1 che per quella in M_2 . Gli stati accettanti di M sono quelle coppie tali che M_1 o M_2 è in uno stato accettore.

Dimostrazione. Supponiamo che M_1 riconosca A_1 dove $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ ed M_2 riconosca A_2 dove $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

Costruiamo M che riconosce $A_1 \cup A_2$ dove $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) | r_1 \in Q_1 \wedge r_2 \in Q_2\}$. Questo insieme è il prodotto cartesiano degli insiemi Q_1 e Q_2 ed è denotato con $Q_1 \times Q_2$. E' l'insieme di tutte le coppie di stati, il primo in Q_1 e il secondo in Q_2 .
2. Σ , l'alfabeto, è lo stesso di M_1 ed M_2 (il teorema resta vero se hanno alfabeti diversi, Σ_1 e Σ_2).
3. δ , la funzione di transizione, è definita come segue. Per ogni $(r_1, r_2) \in Q$ e ogni $a \in \Sigma$ sia:

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

Quindi δ riceve uno stato di M (che in realtà è una coppia di stati, uno in M_1 e uno in M_2), con un simbolo di input, e restituisce lo stato successivo di M .

4. q_0 è la coppia (q_1, q_2) .

5. F è l'insieme delle coppie in cui l'uno o l'altro elemento è uno stato accettante di M_1 o M_2 . Possiamo scriverlo come:

$$F = \{(r_1, r_2) | r_1 \in F_1 \vee r_2 \in F_2\}$$

Questa espressione è uguale a $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$.

Teorema 1.1.2 – Chiusura rispetto alla concatenazione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione. In altre parole, se A_1 e A_2 sono linguaggi regolari, allora lo è anche $A_1 \circ A_2$.

Per provare questo teorema, proviamo qualcosa di simile alla prova del caso dell'unione. Come prima, possiamo iniziare con gli automi finiti M_1 ed M_2 che riconoscono i linguaggi regolari A_1 e A_2 . Ma ora, invece di costruire l'automa M che accetta il suo input se M_1 o M_2 lo accetta, esso deve accettare se il suo input può essere diviso in due parti, tali che M_1 accetta la prima parte ed M_2 accetta la seconda parte. Il problema è che M non sa dove dividere il suo input (cioè, dove finisce la prima parte e inizia la seconda). Per risolvere questo problema, introduciamo una nuova tecnica chiamata non determinismo.

1.2 Non determinismo

Il non determinismo è un concetto utile che ha avuto un grande impatto sulla teoria della computazione. Finora nella nostra discussione, ogni passo di una computazione seguiva univocamente dal passo precedente. In questo caso parliamo di computazione deterministica. In una macchina non deterministica, possono esistere diverse scelte per lo stato successivo in ogni punto.

Il non determinismo è una generalizzazione del determinismo, quindi ogni automa finito deterministico è automaticamente un automa finito non deterministico. Come mostra la Figura 1.4, gli automi finiti non deterministici possono avere ulteriori caratteristiche.

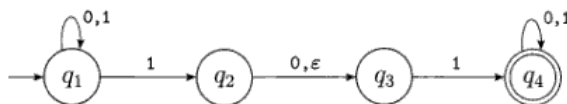


Figura 1.4: Automa finito non deterministico

La differenza tra un automa finito deterministico, indicato con l'acronimo DFA, e un automa finito non deterministico, indicato con l'acronimo NFA, è subito evidente. In un NFA, uno stato può avere zero, uno, o più archi uscenti per ogni simbolo dell'alfabeto e, in generale, può avere archi etichettati con elementi dell'alfabeto o ϵ .

Come computa un NFA? Supponiamo di eseguire un NFA su una stringa di input e di giungere in uno stato con più modi per procedere. La macchina si divide in più copie di sé stessa e segue tutte le possibilità in parallelo. Ogni copia della macchina prende una delle possibili direzioni per procedere. Se il simbolo di input successivo non compare su alcuno degli archi uscenti dallo stato occupato da una copia della macchina, quella copia della macchina "muore", insieme con il ramo della computazione a essa associato. Infine, se una qualunque di queste copie della macchina è in uno stato accettante alla fine dell'input, l'NFA accetta la stringa di input.

Un altro modo per considerare una computazione non deterministica è vederla come un albero di possibilità. La radice dell'albero corrisponde all'inizio della computazione. Ogni punto di ramificazione nell'albero corrisponde a un punto nella computazione in cui la macchina ha scelte multiple. La macchina accetta se almeno uno dei rami della computazione termina in uno stato accettante.

Nella descrizione formale, N_1 è $(Q, \Sigma, \delta, q_1, F)$ dove:

1. $Q = \{q_1, q_2, q_3, q_4\}$,
2. $\Sigma = \{0, 1\}$,

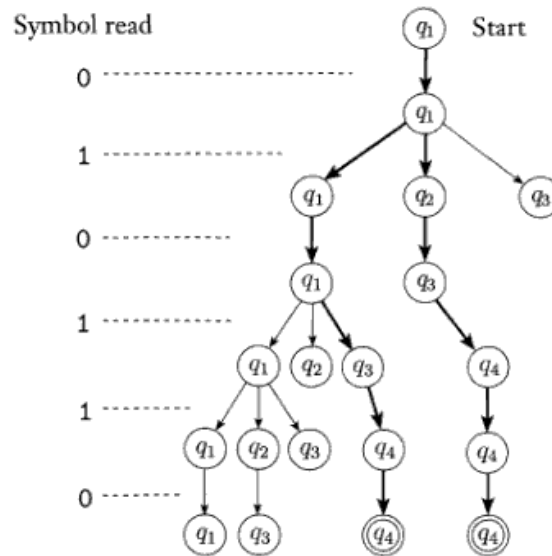


Figura 1.5: Computazione dell'automa 1.4 sull'input 010110

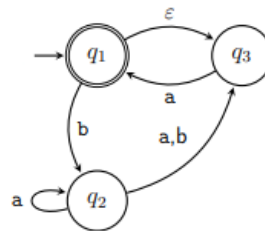
	0	1	ϵ
3. δ è data da	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
	$\{q_3\}$	$\emptyset$	$\{q_3\}$
	$\emptyset$	$\{q_4\}$	$\emptyset$
	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 è lo stato iniziale, ed

5. $F = \{q_4\}$.

Esempio 1.2.1 – Automa non deterministico

Costruire un NFA che accetta le stringhe ϵ , a , $baba$, e baa , ma che non accetta le stringhe b , bb , e $babba$.



1.2.1 Definizione formale di automa finito non deterministico

Definizione 1.2.1 – Automa finito non deterministico

Per un qualsiasi insieme Q denotiamo con $\mathcal{P}(Q)$ la collezione di tutti i sottoinsiemi di Q . In questo caso $\mathcal{P}(Q)$ è chiamato l'insieme potenza di Q . Per ogni alfabeto Σ scriviamo Σ_ϵ per denotare $\Sigma \cup \{\epsilon\}$.

Ora possiamo descrivere formalmente un automa finito non deterministico come una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

1. Q è un insieme finito di stati,
2. Σ è un alfabeto finito,
3. $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ è la funzione di transizione,
4. $q_0 \in Q$ è lo stato iniziale, ed

5. $F \subseteq Q$ l'insieme degli stati di accettazione.

Una configurazione per NFA N è $(q, z) \in Q \times \Sigma_\epsilon^*$. Diremo $(p, ax) \vdash_N (q, x) \iff q \in S(p, a), a \in \Sigma_\epsilon, x \in \Sigma_\epsilon^*$. Posso considerare la chiusura simmetrica e transitiva di $\vdash_N$, detta $\vdash_N^*$. N accetta $w \in \Sigma_\epsilon^* \iff \exists q \in F \mid (q_0, w) \vdash_N^* (q, \epsilon)$.

La definizione formale di computazione per un NFA è simile a quella per un DFA. Sia $N = (Q, \Sigma, \delta, q_0, F)$ un NFA e sia w una stringa sull'alfabeto Σ . Diciamo che N accetta w se possiamo scrivere w come $w = y_1 y_2 \dots y_m$, dove ciascun y_i è un elemento di Σ_ϵ ed esiste una sequenza di stati $r_0, r_1, \dots, r_m$ in Q con tre condizioni:

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$, for $i=0, \dots, m-1$, ed
3. $r_m \in F$.

La condizione 1 dice che la macchina inizia nello stato iniziale. La condizione 2 dice che lo stato r_{i+1} è uno dei possibili stati successivi quando N è nello stato r_i e sta leggendo y_{i+1} . Osserva che $\delta(r_i, y_{i+1})$ è l'insieme dei possibili stati successivi e quindi diciamo che r_{i+1} è un elemento di quell'insieme. Infine, la condizione 3 dice che la macchina accetta il suo input se l'ultimo stato è uno stato accettante.

1.2.2 Equivalenza tra gli NFA e i DFA

Gli automi finiti deterministici e quelli non deterministici riconoscono la stessa classe di linguaggi. Diciamo che due macchine sono equivalenti se esse riconoscono lo stesso linguaggio.

Teorema 1.2.1 – Equivalenza tra gli NFA e i DFA

Per ogni automa finito non deterministico esiste un automa finito deterministico equivalente.

Osservazione. Se un linguaggio è riconosciuto da un NFA, allora dobbiamo mostrare l'esistenza di un DFA che lo riconosce. L'idea è trasformare l'NFA in un DFA equivalente che simula l'NFA.

Ricorda la strategia "lettore come automa" per progettare automi finiti. Come simularesti l'NFA se stessi immaginando di essere un DFA? Cosa devi memorizzare dell'elaborazione della stringa di input? Se k è il numero degli stati dell'NFA, esso ha 2^k sottoinsiemi di stati. Ogni sottoinsieme corrisponde a una delle possibilità che il DFA deve ricordare, quindi il DFA che simula l'NFA avrà 2^k stati.

Dimostrazione. Sia $N = (Q, \Sigma, \delta, q_0, F)$ l'NFA che riconosce un linguaggio A . Costruiamo un DFA $M = (Q', \Sigma, \delta', q'_0, F')$ che riconosce A . Prima di fare la costruzione completa, consideriamo inizialmente il caso più semplice in cui N non ha ϵ -archi. In seguito considereremo gli ϵ -archi.

1. $Q' = \mathcal{P}(Q)$

Ogni stato di M è un insieme di stati di N . Ricorda che $\mathcal{P}(Q)$ è l'insieme dei sottoinsiemi di Q .

2. Per $R \in Q'$ e $a \in \Sigma$ sia $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ per qualche } r \in R\}$.

Se R è uno stato di M , esso è anche un insieme di stati di N . Quando M legge un simbolo a nello stato R , mostra dove a porta ogni stato in R . Poiché da ogni stato si può andare in un insieme di stati, prendiamo l'unione di tutti questi insiemi. Un altro modo per scrivere questa espressione è:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

3. $q'_0 = \{q_0\}$

M inizia nello stato corrispondente alla collezione che contiene solo lo stato iniziale di N .

4. $F' = \{R \in Q' \mid R \text{ contiene uno stato accettante di } N\}$.

La macchina M accetta se uno dei possibili stati in cui N potrebbe essere a quel punto è uno stato accettante.

Ora dobbiamo considerare gli ϵ -archi. Per farlo, introduciamo qualche ulteriore notazione. Per ogni stato R di M , definiamo $E(R)$ come la collezione di stati che possono essere raggiunti dagli elementi di R proseguendo solo con ϵ -archi, includendo gli stessi elementi di R . Formalmente, per $R \subseteq Q$ sia:

$$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso 0 o più } \epsilon\text{-archi}\}$$

Poi modifichiamo la funzione di transizione di M seguendo tutti gli stati che possono essere raggiunti proseguendo attraverso ϵ -archi dopo ogni passo. Sostituendo $\delta(r, a)$ con $E(\delta(r, a))$ realizziamo questo effetto. Quindi:

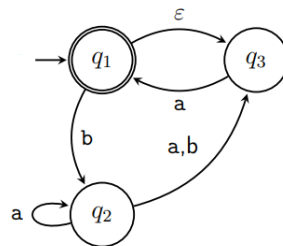
$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ per qualche } r \in R\}$$

Inoltre, dobbiamo modificare lo stato iniziale di M per seguire tutti i possibili stati che possono essere raggiunti dallo stato iniziale di N attraverso gli ϵ -archi. Cambiare q'_0 in $E(\{q_0\})$ realizza questo risultato. Ora abbiamo completato la costruzione del DFA M che simula l'NFA N .

Ovviamente la costruzione di M funziona correttamente. A ogni passo nella computazione di M su un input, chiaramente entra in uno stato che corrisponde al sottoinsieme di stati in cui N potrebbe essere a quel punto. Quindi la nostra prova è completa.

Esempio 1.2.2 – Trasformare un NFA in un DFA

Illustriamo la procedura che abbiamo dato nella prova del Teorema 1.2.2 per trasformare un NFA in un DFA.



Per costruire un DFA D equivalente innanzitutto determiniamo gli stati di D . L'NFA ha tre stati quindi costruiamo D con otto stati, uno per ogni sottoinsieme dell'insieme degli stati.

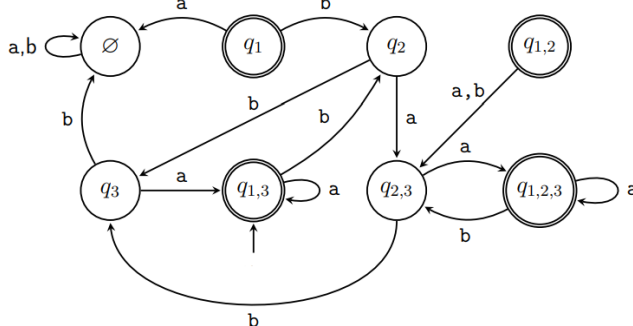
$$\{\emptyset, \{q_1\}, \{q_2\}, \{q_3\}, \{q_1, q_2\}, \{q_2, q_3\}, \{q_1, q_3\}, \{q_1, q_2, q_3\}\}$$

Poi, determiniamo lo stato iniziale e gli stati accettanti di D . Lo stato iniziale è $E(\{q_1\})$ l'insieme degli stati che sono raggiungibili da q_1 passando attraverso ϵ -archi, più q_1 stesso. Un ϵ -arco va da q_1 a q_3 , quindi $E(\{q_1\}) = \{q_1, q_3\}$. I nuovi stati accettanti sono quelli che contengono lo stato accettante dell'NFA quindi $\{\{q_1\}, \{q_1, q_2\}, \{q_1, q_3\}, \{q_1, q_2, q_3\}\}$. Infine, determiniamo la funzione di transizione di D . Ciascuno degli stati di D va in una posizione sull'input a e in una posizione sull'input b . Illustriamo il processo di determinare la posizione degli archi di transizione di D con pochi esempi.

In D , lo stato $\{q_2\}$ va in $\{q_2, q_3\}$ sull'input a perché nell'NFA, lo stato q_2 va sia in q_2 che in q_3 sull'input a e non possiamo andare più lontano da q_2 o q_3 attraverso ϵ -archi. Lo stato $\{q_2\}$ va nello stato $\{q_3\}$ sull'input b perché nell'NFA lo stato q_2 va solo nello stato q_3 sull'input b e da q_3 non possiamo andare più lontano attraverso ϵ -archi. Lo stato $\{q_1\}$ va in $\emptyset$ su a perché nessun arco etichettato con a esce da esso. Va in $\{q_2\}$ su b . Nota che la procedura nel Teorema 1.2.2 specifica che seguiamo gli ϵ -archi dopo ogni simbolo di input che viene letto. Una procedura alternativa basata sul seguire gli ϵ -archi prima di leggere ogni simbolo funziona ugualmente bene.

Lo stato $\{q_3\}$ va in $\{q_1, q_3\}$ su a perché nell'NFA, lo stato q_3 va in q_1 su a e q_1 a sua volta va in q_3 con un ϵ -arco. Lo stato $\{q_3\}$ su b va in $\emptyset$.

Lo stato $\{q_1, q_2\}$ su a va in $\{q_2, q_3\}$ perché q_1 non punta ad alcuno stato con a archi, q_2 punta sia a q_2 che a q_3 con a archi, e nessuno dei due punta altrove con ϵ -archi. Lo stato $\{q_1, q_2\}$ su b va in $\{q_2, q_3\}$. Continuando in questo modo, otteniamo il seguente diagramma per D .



1.2.3 Chiusura rispetto alle operazioni regolari

Ora torniamo alla chiusura della classe dei linguaggi regolari rispetto alle operazioni regolari. Il nostro scopo provare che l'unione, la concatenazione, e lo star di linguaggi regolari sono ancora regolari.

Innanzitutto, consideriamo di nuovo la chiusura rispetto all'unione. Prima abbiamo provato la chiusura rispetto all'unione simulando deterministicamente e simultaneamente entrambe le macchine, attraverso la costruzione di un prodotto cartesiano. Ora diamo una nuova prova per illustrare la tecnica del non determinismo.

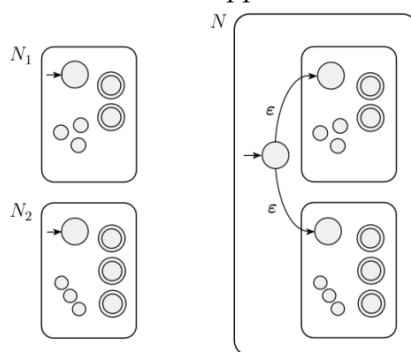
Teorema 1.2.2 – I linguaggi regolari sono chiusi rispetto all'unione

Abbiamo i linguaggi regolari A_1 e A_2 e vogliamo provare che $A_1 \cup A_2$ è regolare.

L'idea è prendere due NFA, N_1 ed N_2 per A_1 e A_2 e comporli in un nuovo NFA, N .

La macchina N deve accettare il suo input se N_1 o N_2 accetta questo input. La nuova macchina ha un nuovo stato iniziale che si dirama negli stati iniziali delle vecchie macchine con ϵ -archi. In questo modo, la nuova macchina ipotizza non deterministicamente quale delle due macchine accetta l'input. Se una di esse accetta l'input, anche N lo accetterà.

Rappresentiamo questa costruzione nella figura seguente dove mostriamo come comporre N_1 ed N_2 in N aggiungendo archi di transizione supplementari.



Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere $A_1 \cup A_2$:

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$.

Gli stati di N sono tutti gli stati di N_1 ed N_2 con l'aggiunta di un nuovo stato iniziale q_0 .

2. Lo stato q_0 è lo stato iniziale di N .

3. L'insieme degli stati accettanti $F = F_1 \cup F_2$.

Gli stati accettanti di N sono tutti gli stati accettanti di N_1 ed N_2 . In questo modo, N accetta se N_1 accetta o N_2 accetta.

4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$,

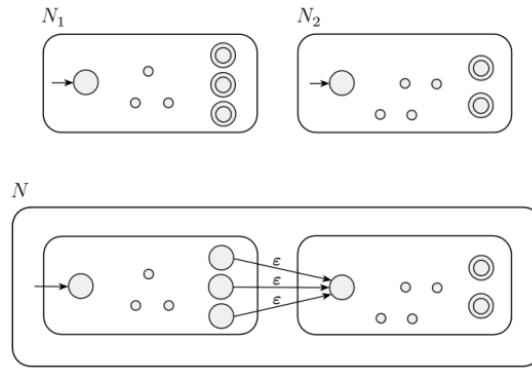
$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

Ora possiamo provare la chiusura rispetto alla concatenazione.

Teorema 1.2.3 – I linguaggi regolari sono chiusi rispetto alla concatenazione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

Osservazione. Abbiamo i linguaggi regolari A_1 e A_2 vogliamo provare che $A_1 \circ A_2$ è regolare. L'idea è prendere due NFA, N_1 ed N_2 per A_1 ed A_2 e combinarli in un nuovo NFA N come abbiamo fatto nel caso dell'unione, ma questa volta in un modo diverso.



Poniamo come stato iniziale di N lo stato iniziale di N_1 . Gli stati accettanti di N_1 hanno degli ulteriori ϵ -archi che non deterministicamente permettono di diramarsi in N_2 ogni volta che N_1 è in uno stato accettore, indicando che ha trovato un pezzo iniziale dell'input che costituisce una stringa in A_1 . Gli stati accettanti di N sono solo gli stati accettanti di N_2 . Quindi esso accetta quando l'input può essere diviso in due parti, la prima accettata da N_1 e la seconda da N_2 . Possiamo pensare che N non deterministicamente ipotizza dove effettuare la divisione.

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$.

Gli stati di N sono tutti gli stati di N_1 ed N_2 .

2. Lo stato q_1 è uguale allo stato iniziale di N_1 .

3. Gli stati accettanti F_2 sono uguali agli stati accettanti di N_2 .

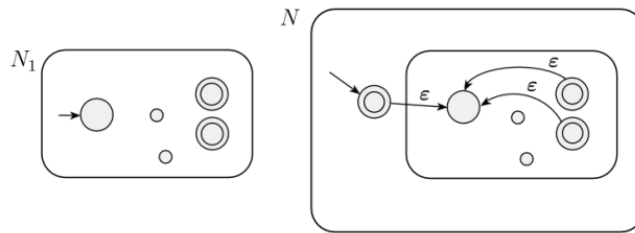
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

Teorema 1.2.4 – I linguaggi regolari sono chiusi rispetto all'operazione star

La classe dei linguaggi regolari è chiusa rispetto all'operazione star.

Osservazione. Abbiamo un linguaggio regolare A_1 e vogliamo provare che anche A_1^* è regolare. Prendiamo un NFA N_1 per A_1 e lo modifichiamo per riconoscere A_1^* . L'NFA N risultante accetterà il suo input quando esso può essere diviso in varie parti ed N_1 accetta ogni parte. Possiamo costruire N come N_1 con ϵ -archi supplementari che dagli stati accettanti ritornano allo stato iniziale. In questo modo, quando l'elaborazione giunge alla fine di una parte che N_1 accetta, la macchina N ha la scelta di tornare indietro allo stato iniziale per provare a leggere un'altra parte che N_1 accetta. Inoltre, dobbiamo modificare N in modo che accetti ϵ , che è sempre un elemento di A_1^* . Un'idea (leggermente cattiva) è semplicemente aggiungere lo stato iniziale all'insieme degli stati accettanti. Questa strategia certamente aggiunge ϵ al linguaggio riconosciuto, ma può anche aggiungere altre stringhe non volute. Il modo per correggerla è aggiungere un nuovo stato iniziale, che è anche uno stato accettore, e che ha un ϵ -arco entrante nel vecchio stato iniziale. Questa soluzione ha l'effetto desiderato di aggiungere ϵ al linguaggio senza aggiungere qualcos'altro.



Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 . Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere A_1^* .

1. $Q = \{q_0\} \cup Q_1$.

Gli stati di N sono gli stati di N_1 più un nuovo stato iniziale.

2. Lo stato q_0 è il nuovo stato iniziale.

3. $F = \{q_0\} \cup F_1$

Gli stati accettanti sono i vecchi stati accettanti più il nuovo stato iniziale.

4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

1.3 Espressioni regolari

Analogamente alle espressioni algebriche, possiamo usare le operazioni regolari per costruire espressioni che descrivono linguaggi, che sono chiamate espressioni regolari. Un esempio è $(0 \cup 1)0^*$.

Il valore di un'espressione regolare è un linguaggio. In questo caso, il valore è il linguaggio che consiste di tutte le stringhe che iniziano con uno 0 o un 1 seguito da un qualsiasi numero di simboli uguali a 0. Otteniamo questo risultato dividendo l'espressione nelle sue parti. In primo luogo, i simboli 0 e 1 sono abbreviazioni per gli insiemi $\{0\}$ e $\{1\}$. Quindi $(0 \cup 1)$ significa $(\{0\} \cup \{1\})$. Il valore di questa parte è il linguaggio $\{0, 1\}$. La parte 0^* denota $\{0\}^*$, e il suo valore è il linguaggio che consiste di tutte le stringhe contenenti un numero qualsiasi di simboli uguali a 0. Inoltre, come il simbolo $\times$ in algebra, il simbolo della concatenazione $\circ$ è spesso sottinteso nelle espressioni regolari.

Quindi $(0 \cup 1)0^*$ è in realtà un'abbreviazione per $(0 \cup 1) \circ 0^*$. La concatenazione connette le stringhe dalle due parti per ottenere il valore dell'intera espressione.

Le espressioni regolari hanno un ruolo importante nelle applicazioni dell'informatica. Nelle applicazioni che coinvolgono testo, gli utenti possono voler cercare stringhe che soddisfano alcuni schemi. Le espressioni regolari forniscono un metodo potente per descrivere tali schemi.

Nelle espressioni regolari, l'operazione star è eseguita per prima, seguita dalla concatenazione e infine dall'unione, a meno che delle parentesi non cambino l'ordine usuale.

Definizione 1.3.1 – Espressione regolare

Diciamo che R è un'espressione regolare se R è

1. a per qualche a nell'alfabeto Σ ,
2. ϵ ,
3. $\emptyset$,
4. $(R_1 \cup R_2)$ dove R_1 ed R_2 sono espressioni regolari,
5. $(R_1 \circ R_2)$ dove R_1 ed R_2 sono espressioni regolari, o
6. (R_1^*) dove R_1 è un'espressione regolare.

Nei punti 1 e 2, le espressioni regolari a e ϵ rappresentano i linguaggi $\{a\}$ e $\{\epsilon\}$, rispettivamente. Nel punto 3, l'espressione regolare $\emptyset$ rappresenta il linguaggio vuoto. Nei punti 4, 5, e 6, le espressioni rappresentano i linguaggi ottenuti prendendo l'unione o la concatenazione dei linguaggi R_1 ed R_2 o lo star del linguaggio R_1 rispettivamente.

Non confondere le espressioni regolari ϵ e $\emptyset$. L'espressione ϵ rappresenta il linguaggio che contiene una sola stringa ossia, la stringa vuota - mentre $\emptyset$ rappresenta il linguaggio che non contiene alcuna stringa.

Apparentemente, sembra che stiamo definendo la nozione di espressione regolare in termini di sé stessa. Se fosse vero, avremmo una definizione circolare, che non sarebbe valida. Comunque, R_1 ed R_2 sono sempre più piccole di R . Quindi in realtà stiamo definendo le espressioni regolari in termini di espressioni regolari più piccole, evitando in tal modo la circolarità. Una definizione di questo tipo è chiamata una definizione induttiva.

Esempio 1.3.1 – Espressioni regolari

Negli esempi seguenti, assumiamo che l'alfabeto Σ sia $\{0, 1\}$.

1. $0^*10^* = \{w | w \text{ contiene un solo } 1\}$.
2. $\Sigma^*1\Sigma^* = \{w | w \text{ ha almeno un } 1\}$.
3. $\Sigma^*001\Sigma^* = \{w | w \text{ contiene la stringa } 001 \text{ come sottostringa}\}$.
4. $(0 \cup \epsilon)(1 \cup \epsilon) = \{\epsilon, 0, 1, 01\}$
5. $1^*\emptyset = \emptyset$.

Concatenando l'insieme vuoto a un qualsiasi insieme si ottiene l'insieme vuoto.

6. $\emptyset^* = \{\epsilon\}$

L'operazione star concatena un numero qualsiasi di stringhe del linguaggio per ottenere una stringa nel risultato. Se il linguaggio è vuoto, l'operazione star può concatenare 0 stringhe, dando solo la stringa vuota.

1.3.1 Equivalenza con gli automi finiti

Le espressioni regolari e gli automi finiti sono equivalenti rispetto alla loro potenza descrittiva. Ogni espressione regolare può essere trasformata in un automa finito che riconosce il linguaggio che essa

descrive, e viceversa. Ricorda che un linguaggio regolare è un linguaggio che è riconosciuto da qualche automa finito.

Teorema 1.3.1 – Linguaggio ed espressione regolare

Un linguaggio è regolare se e solo se qualche espressione regolare lo descrive.

Questo teorema deve essere dimostrato in entrambe le direzioni. Noi enunciamo e proviamo ciascuna direzione in un lemma separato.

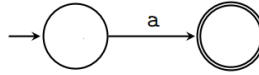
Lemma 1.3.1 – Espressione regolare implica linguaggio regolare

Se un linguaggio è descritto da un'espressione regolare, allora esso è regolare.

Osservazione. Supponiamo di avere un'espressione regolare R che descrive un linguaggio A . Mostriamo come trasformare R in un NFA che riconosce A . Se un NFA riconosce A allora A è regolare.

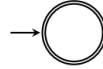
Dimostrazione. Trasformiamo R in un NFA N . Consideriamo i sei casi nella definizione di espressione regolare.

1. $R = a$ per qualche $a \in \Sigma$. Allora $L(R) = \{a\}$ e il seguente NFA riconosce $L(R)$.



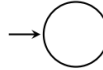
Formalmente, $N = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$, dove descriviamo δ dicendo che $\delta(q_1, a) = \{q_2\}$ e $\delta(r, b) = \emptyset$ per $r \neq q_1$ o $b \neq a$.

2. $R = \epsilon$. Allora $L(R) = \{\epsilon\}$ e il seguente NFA riconosce $L(R)$.



Formalmente, $N = (\{q_1\}, \Sigma, \delta, q_1, \{q_1\})$, dove $\delta(r, b) = \emptyset$ per ogni r e b .

3. $R = \emptyset$. Allora $L(R) = \emptyset$, e il seguente NFA riconosce $L(R)$.



Formalmente, $N = (\{q\}, \Sigma, \delta, q, \emptyset)$, dove $\delta(r, b) = \emptyset$ per ogni r e b .

4. $R = R_1 \cup R_2$.
5. $R = R_1 \circ R_2$.
6. $R = R_1^*$.

Per gli ultimi tre casi, usiamo le costruzioni date nelle prove che la classe dei linguaggi regolari è chiusa rispetto alle operazioni regolari. In altre parole, costruiamo l'NFA per R dagli NFA per R_1 ed R_2 (o solo R_1 nel caso 6) e mediante l'appropriata costruzione della chiusura.

Questo termina la prima parte della prova del Teorema 1.3.1, dando la direzione più facile della condizione "se e solo se".

Definizione 1.3.2 – Automa finito non deterministico generalizzato

Un automa finito non deterministico generalizzato è una quintupla, $(Q, \Sigma, \delta, q_{start}, q_{accept})$, dove

1. Q è l'insieme finito degli stati,
2. Σ è l'alfabeto di input,
3. $\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \longrightarrow \mathcal{R}$ è la funzione di transazione,

4. q_{start} è lo stato iniziale e
5. q_{accept} è lo stato accettante.

Gli automi finiti non deterministici generalizzati sono semplicemente automi finiti non deterministici nei quali gli archi delle transizioni possono avere espressioni regolari come etichette, invece che solo elementi dell'alfabeto o ϵ . Il GNFA legge blocchi di simboli dall'input, non necessariamente solo un simbolo alla volta come in un comune NFA. Il GNFA si muove lungo un arco di transizione che collega due stati leggendo un blocco di simboli dall'input che formano una stringa descritta dall'espressione regolare su quell'arco. Un GNFA è non deterministico e quindi può avere diversi modi di elaborare la stessa stringa di input. Esso accetta il suo input se la sua elaborazione può far sì che il GNFA sia in uno stato accettante alla fine dell'input.

Un GNFA accetta una stringa w in Σ^* se $w = w_1w_2...w_k$ dove ogni w_i è in Σ^* ed esiste una sequenza di stati tale che $q_0, q_1, \dots, q_k$ tale che

1. $q_0 = q_{start}$ è lo stato iniziale,
2. $q_k = q_{accept}$ lo stato accettante e
3. per ogni i , risulta $w_i \in L(R_i)$ dove $R_i = \delta(q_{i-1}, q_i)$; in altre parole, R_i è l'espressione sull'arco da q_{i-1} a q_i .

Lemma 1.3.2 – Linguaggio regolare implica espressione regolare

Se un linguaggio è regolare, allora è descritto da un'espressione regolare.

Osservazione. Dobbiamo mostrare che se un linguaggio A è regolare, allora un'espressione regolare lo descrive. Poiché A è regolare, esso è accettato da un DFA. Descriviamo una procedura per trasformare i DFA in espressioni regolari equivalenti.

Dividiamo questa procedura in due parti, usando un nuovo tipo di automa finito chiamato automa finito non deterministico generalizzato, GNFA. Prima mostriamo come trasformare un DFA in un GNFA e poi come trasformare un GNFA in un'espressione regolare. Per comodità, richiediamo che i GNFA abbiano sempre una forma speciale che soddisfi le seguenti condizioni.

- Lo stato iniziale ha archi di transizione uscenti verso un qualsiasi altro stato ma nessun arco entrante proveniente da un qualsiasi altro stato.
- Esiste un solo stato accettante, ed esso ha archi entranti provenienti da un qualsiasi altro stato ma nessun arco uscente verso un qualsiasi altro stato. Inoltre, lo stato accettante non è uguale allo stato iniziale.
- Eccetto che per lo stato iniziale e lo stato accettante, un arco va da ogni stato ad ogni altro stato e anche da ogni stato in sé stesso.

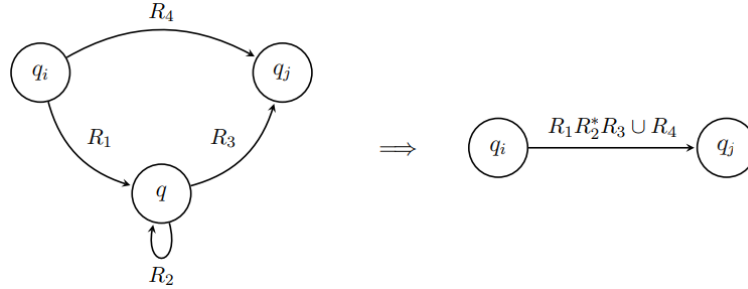
Possiamo facilmente trasformare un DFA in un GNFA nella forma speciale. Aggiungiamo semplicemente un nuovo stato iniziale con un ϵ -arco che entra nel vecchio stato iniziale e un nuovo stato accettante con ϵ -archi entranti, provenienti dai vecchi stati accettanti. Se alcuni archi hanno più etichette (o se ci sono più archi che collegano gli stessi due stati nella stessa direzione), sostituiamo ognuno di essi con un solo arco la cui etichetta è l'unione delle precedenti etichette. Infine, aggiungiamo archi con etichetta $\emptyset$ tra stati che non hanno archi. Questo ultimo passo non cambierebbe il linguaggio riconosciuto perché una transizione etichettata con $\emptyset$ non può mai essere usata. Da qui in poi assumiamo che tutti i GNFA sono nella forma speciale.

Ora mostriamo come trasformare un GNFA in un'espressione regolare. Supponiamo che il GNFA abbia k stati. Allora, poiché un GNFA deve avere uno stato iniziale e uno stato accettante ed essi devono essere diversi tra loro, sappiamo che $k \geq 2$. Se $k > 2$, costruiamo un GNFA equivalente con $k - 1$ stati. Questo passo può essere ripetuto sul nuovo GNFA fino a quando esso è ridotto a due stati. Se $k = 2$, il GNFA ha un solo arco che va dallo stato iniziale allo stato accettante. L'etichetta di questo arco è l'espressione regolare equivalente.

Il passo cruciale è costruire un GNFA equivalente con uno stato in meno quando $k > 2$. Lo facciamo scegliendo uno stato, estraendolo dalla macchina, e modificando il resto in modo che

sia ancora riconosciuto lo stesso linguaggio. Ogni stato può essere scelto, purché non sia lo stato iniziale o lo stato accettante. Siamo certi che un tale stato esiste perché $k > 2$. Chiamiamo q_{rip} lo stato rimosso.

Dopo aver rimosso q_{rip} modifichiamo la macchina cambiando le espressioni regolari che etichettano ciascuno dei restanti archi. Le nuove etichette controbilanciano l'assenza di q_{rip} reintegrando le computazioni perse. La nuova etichetta che va da uno stato q_i a uno stato q_j è un'espressione regolare che descrive tutte le stringhe che porterebbero la macchina da q_i a q_j o direttamente o tramite q_{rip} . Illustriamo questa strategia:



Nella vecchia macchina, se

1. q_i va a q_{rip} con un arco etichettato R_1 ,
2. q_{rip} va in sé stesso con un arco etichettato R_2 ,
3. q_{rip} va a q_j con un arco etichettato R_3 ,
4. q_i va a q_j con un arco etichettato R_4 ,

allora nella nuova macchina, l'arco da q_i a q_j riceve l'etichetta

$$(R_1)(R_2)^*(R_3) \cup (R_4)$$

Facciamo questa modifica per ogni arco che va da un qualsiasi stato q_i a un qualsiasi stato q_j includendo il caso in cui $q_i = q_j$. La nuova macchina riconosce il linguaggio di partenza.

Dimostrazione. Formalizziamo questa idea. In primo luogo, per rendere più facile la prova, definiamo formalmente il nuovo tipo di automa introdotto. Un GNFA è simile a un automa finito non deterministico tranne che per la funzione di transizione, che ha la forma

$$\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \longrightarrow \mathcal{R}$$

Il simbolo $\mathcal{R}$ è la collezione di tutte le espressioni regolari sull'alfabeto Σ , e q_{start} e q_{accept} sono gli stati iniziale e accettante. Se $\delta(q_i, q_j) = \mathcal{R}$, l'arco dallo stato q_i allo stato q_j ha come etichetta l'espressione regolare $\mathcal{R}$. Il dominio della funzione di transizione è $(Q - \{q_{accept}\}) \times (Q - \{q_{start}\})$ perché un arco collega ogni stato a un qualsiasi altro stato, tranne che nessun arco è uscente da q_{accept} o è entrante in q_{start} .

Sia M il DFA per il linguaggio A . Allora trasformiamo M in un GNFA G aggiungendo un nuovo stato iniziale e un nuovo stato accettante e i necessari archi di transizione supplementari. Usiamo la procedura $\text{CONVERT}(G)$ (vedi Algoritmo 1.3.1).

Algoritmo 1.3.1 – $\text{CONVERT}(G)$

$\text{CONVERT}(G)$ prende un GNFA e restituisce un'espressione regolare equivalente. Questa procedura usa la ricorsione, che significa che essa chiama sé stessa. Evitiamo un ciclo infinito perché la procedura chiama sé stessa solo per elaborare un GNFA che ha uno stato in meno. Il caso in cui il GNFA ha due stati è trattato senza ricorsione.

1. Sia k il numero di stati di G .
2. Se $k = 2$, allora G deve consistere di uno stato iniziale, uno stato accettante, e un singolo arco che li collega ed è etichettato con un'espressione regolare $\mathcal{R}$. Restituisce

l'espressione $\mathcal{R}$.

3. Se $k > 2$ scegliamo un qualsiasi stato $q_{rip} \in Q$ diverso da q_{start} e q_{accept} e sia G' il GNFA $(Q', \Sigma, \delta', q_{start}, q_{accept})$, dove

$$Q' = Q - \{q_{rip}\}$$

e per ogni $q_i \in Q' - \{q_{accept}\}$ e ogni $q_j \in Q' - \{q_{start}\}$ poniamo

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$$

dove $R_1 = \delta(q_i, q_{rip})$, $R_2 = \delta(q_{rip}, q_{rip})$, $R_3 = \delta(q_{rip}, q_j)$ ed $R_4 = \delta(q_i, q_j)$.

4. Calcola $\text{CONVERT}(G')$ e restituisce questo valore.

Teorema 1.3.2 – $\text{CONVERT}(G)$ è equivalente al GNFA

Per ogni GNFA G , $\text{CONVERT}(G)$ è equivalente a G .

Proviamo quest'affermazione per induzione su k , il numero di stati del GNFA.

Dimostrazione.

- *Passo base:* Proviamo che l'affermazione è vera per $k = 2$ stati. Se G ha solo due stati, esso può avere solo un singolo arco, che va dallo stato iniziale allo stato accettante. L'espressione regolare che è l'etichetta di quest'arco descrive tutte le stringhe che permettono a G di raggiungere lo stato accettante. Quindi quest'espressione è equivalente a G .
- *Passo induttivo:* Assumiamo che l'affermazione sia vera per $k - 1$ stati e usiamo questa ipotesi per provare che l'affermazione è vera per k stati. In primo luogo mostriamo che G e G' riconoscono lo stesso linguaggio. Supponiamo che G accetti un input w . Allora in un ramo accettante della computazione, G entra in una sequenza di stati:

$$q_{start}, q_1, q_2, q_3, \dots, q_{accept}$$

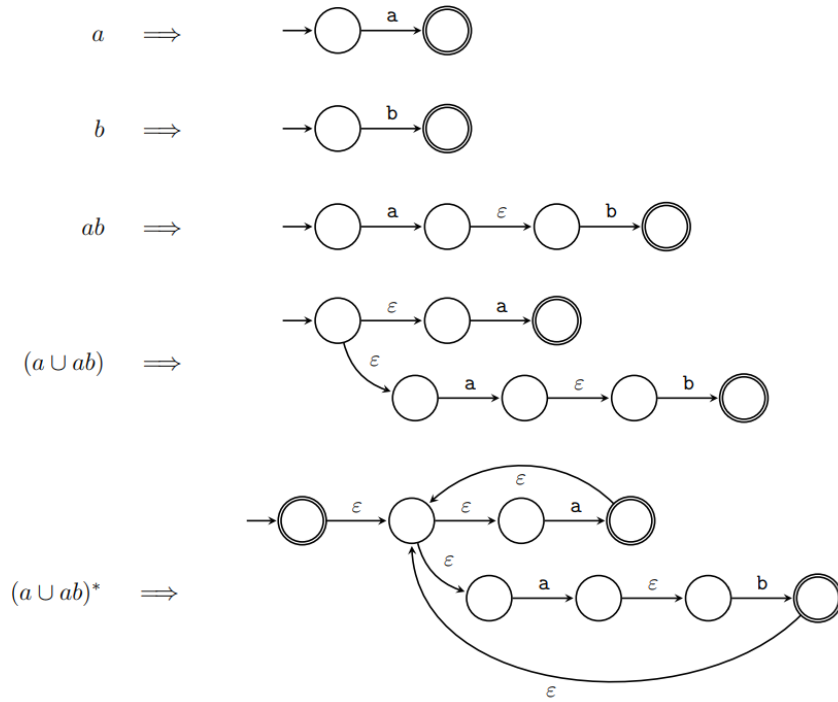
Se nessuno di essi è lo stato rimosso q_{rip} , evidentemente anche G' accetta w . La ragione è che ciascuna delle nuove espressioni regolari che etichettano gli archi di G' contiene le vecchie espressioni regolari come parte di un'unione. Se q_{rip} è presente, eliminando ogni sequenza di stati q_{rip} consecutivi creiamo una computazione accettante per G' . Gli stati q_i e q_j che raggruppano una tale sequenza hanno una nuova espressione sull'arco tra essi, la quale descrive tutte le stringhe che portano da q_i a q_j , attraverso q_{rip} in G . Quindi G' accetta w .

Viceversa, supponiamo che G' accetti un input w . Poiché ciascun arco tra due qualsiasi stati q_i e q_j in G' descrive la collezione delle stringhe che portano da q_i a q_j in G , o direttamente o attraverso q_{rip} , anche G deve accettare w . Quindi G e G' sono equivalenti.

L'ipotesi induttiva afferma che quando l'algoritmo chiama sé stesso ricorsivamente sull'input G , il risultato è un'espressione regolare che è equivalente a G' perché G' ha $k - 1$ stati. Quindi anche questa espressione regolare è equivalente a G , e abbiamo provato che l'algoritmo è corretto.

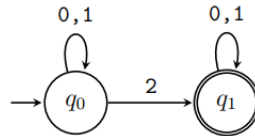
Esempio 1.3.2 – Espressione regolare in NFA

Trasformiamo l'espressione regolare $(a \cup ab)^*$ in un NFA.

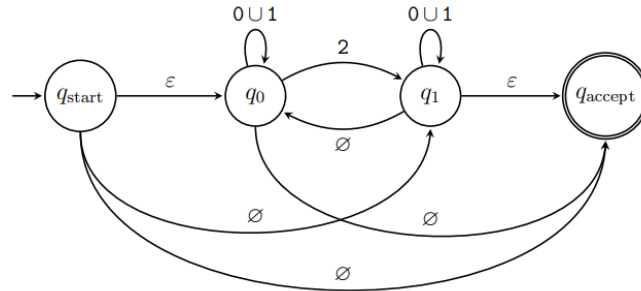


Esempio 1.3.3 – Conversione DFA in espressione regolare

Consideriamo il seguente DFA:

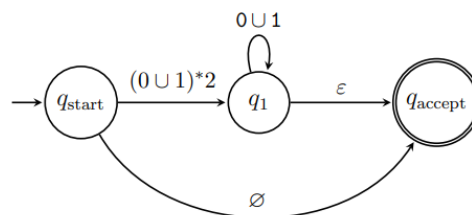


Il suo GNFA equivalente corrisponde a:



Rimuoviamo quindi lo stato q_0 calcolando le nuove transizioni:

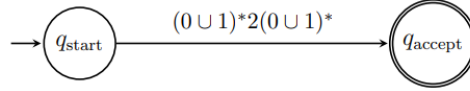
$$\begin{aligned}
 \delta'(q_{start}, q_1) &= \delta(q_{start}, q_0) \delta(q_0, q_0)^* \delta(q_0, q_1) \cup \delta(q_{start}, q_1) = \epsilon(0 \cup 1)^* 2 \cup \emptyset = (0 \cup 1)^* 2 \\
 \delta'(q_{start}, q_{accept}) &= \delta(q_{start}, q_0) \delta(q_0, q_0)^* \delta(q_0, q_{accept}) \cup \delta(q_{start}, q_{accept}) = \epsilon(0 \cup 1)^* \emptyset \cup \emptyset = \emptyset \\
 \delta'(q_1, q_1) &= \delta(q_1, q_0) \delta(q_0, q_0)^* \delta(q_0, q_1) \cup \delta(q_1, q_1) = \emptyset(0 \cup 1)^* 2 \cup (0 \cup 1) = 0 \cup 1 \\
 \delta'(q_1, q_{accept}) &= \delta(q_1, q_0) \delta(q_0, q_0)^* \delta(q_0, q_{accept}) \cup \delta(q_1, q_{accept}) = \emptyset(0 \cup 1)^* \emptyset \cup \epsilon = \epsilon
 \end{aligned}$$



Infine, rimuoviamo lo stato q_1 calcolando le nuove transizioni:

$$\delta''(q_{start}, q_{accept}) = \delta'(q_{start}, q_1) \delta'(q_1, q_1)^* \delta'(q_1, q_{accept}) \cup \delta'(q_{start}, q_{accept}) = (0 \cup 1)^* 2 (0 \cup 1)^* \epsilon \cup \emptyset = (0 \cup 1)^* 2 (0 \cup 1)^*$$

Il GNFA minimale, dunque, corrisponde a:



1.4 Linguaggi non regolari

Per comprendere il potere computazionale degli automi finiti dobbiamo anche comprenderne i limiti. In questa sezione, mostriamo come provare che alcuni linguaggi non possono essere riconosciuti da alcun automa finito.

Consideriamo il linguaggio $B = \{0^n 1^n \mid n \geq 0\}$. Se cerchiamo di trovare un DFA che riconosca B , scopriamo che la macchina sembra aver bisogno di ricordare quanti simboli uguali a 0 ha visto fin quando legge l'input. Poiché il numero di simboli uguali a 0 non è limitato, la macchina dovrà tenere traccia di un numero infinito di possibilità. Ma non può farlo con un numero finito di stati.

Di seguito presentiamo un metodo per provare che linguaggi come B non sono regolari.

1.4.1 Il pumping lemma per i linguaggi regolari

La nostra tecnica per provare la non regolarità deriva da un teorema sui linguaggi regolari, tradizionalmente chiamato il pumping lemma. Questo teorema afferma che tutti i linguaggi regolari hanno una proprietà speciale. Se noi possiamo mostrare che un linguaggio non ha questa proprietà, siamo sicuri che esso non è regolare. La proprietà afferma che tutte le stringhe nel linguaggio possono essere "replicate" se la loro lunghezza raggiunge almeno uno specifico valore speciale, chiamato la lunghezza del pumping. Questo significa che ogni tale stringa contiene una parte che può essere ripetuta un numero qualsiasi di volte ottenendo una stringa che appartiene ancora al linguaggio.

Teorema 1.4.1 – Pumping lemma

Se A è un linguaggio regolare, allora esiste un numero p (la lunghezza del pumping) tale che se s è una qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in tre parti, $s = xyz$ soddisfacenti le seguenti condizioni:

1. per ogni $i \geq 0$, $xy^i z \in A$,
2. $|y| > 0$, e
3. $|xy| \leq p$

Ricordiamo che $|s|$ rappresenta la lunghezza della stringa s , y^i indica i copie di y concatenate insieme, e y^0 è uguale a ϵ .

Quando s è divisa in xyz , x o z potrebbe essere ϵ , ma la condizione 2 dice che $y \neq \epsilon$. Osserva che senza la condizione 2 il teorema sarebbe banalmente vero. La condizione 3 afferma che le parti x e y insieme hanno lunghezza al più p . E' una condizione tecnica supplementare che ogni tanto troviamo utile quando dimostriamo che alcuni linguaggi non sono regolari.

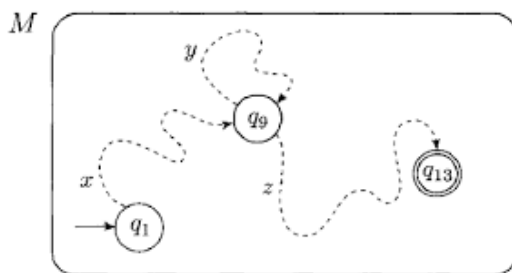
Osservazione. Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A . Assegniamo alla lunghezza del pumping p il numero degli stati di M . Mostriamo che ogni stringa s in A di lunghezza almeno p può essere divisa nelle tre parti xyz , soddisfacenti le nostre tre condizioni. Cosa accade se nessuna stringa in A è di lunghezza almeno p ? Allora il nostro compito è perfino più facile perché il teorema diventa banalmente vero: ovviamente le tre condizioni valgono per tutte le stringhe di lunghezza almeno p se non vi è alcuna di tali stringhe.

Se s in A ha lunghezza almeno p , consideriamo la sequenza di stati che M attraversa nella computazione con input s . Inizia con lo stato iniziale q_1 poi va per esempio in q_3 poi per

esempio in q_{20} poi in q_9 , e così via, finché raggiunge la fine di s nello stato q_{13} . Se s è in A , sappiamo che M accetta s , quindi q_{13} è uno stato accettante.

Se supponiamo che n sia la lunghezza di s , la sequenza di stati $q_1, q_3, q_{20}, q_9, \dots, q_{13}$ ha lunghezza $n + 1$. Poiché n è almeno p , sappiamo $n + 1$ è più grande di p , il numero di stati di M . Quindi, la sequenza deve contenere uno stato che si ripete. Questo risultato è un esempio del principio della piccionaia, un nome di fantasia per il fatto piuttosto ovvio che se p piccioni sono messi in una piccionaia con meno di p nicchie, qualche nicchia deve avere più di un piccione in essa. Lo stato q_9 è quello che si ripete.

Ora dividiamo s nelle tre componenti x , y e z . La componente x è la parte di s che compare prima di q_9 , la componente y è la parte tra le due occorrenze di q_9 e la componente z è la parte restante di s , che viene dopo la seconda occorrenza di q_9 . Quindi x porta M dallo stato q_1 a q_9 , y riporta M da q_9 a q_9 e z porta M da q_9 allo stato accettante q_{13} .



Vediamo perché questa divisione di s soddisfa le tre condizioni. Supponiamo di eseguire M sull'input $xyyz$. Sappiamo che x porta M da q_1 a q_9 , e poi il primo y lo riporta da q_9 a q_9 , come fa il secondo y e poi z lo porta in q_{13} . Essendo q_{13} uno stato accettante, M accetta l'input $xyyz$. Analogamente, esso accetterà xy^iz per ogni $i > 0$. Nel caso $i = 0$, $xy^iz = xz$, che è accettata per ragioni analoghe. Questo prova la condizione 1.

Per verificare la condizione 2, vediamo che $|y| > 0$, poiché era la parte di s tra due diverse occorrenze dello stato q_9 .

Per ottenere la condizione 3, ci assicuriamo che q_9 sia la prima ripetizione nella sequenza. Per il principio della piccionaia, i primi $p + 1$ stati nella sequenza devono contenere una ripetizione. Quindi, $|xy| \leq p$.

Dimostrazione. Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A e sia p il numero di stati di M .

Sia $s = s_1s_2\dots s_n$ una stringa in A di lunghezza n , dove $n \geq p$. Sia $r_1, \dots, r_{n+1}$ la sequenza di stati attraversati da M mentre elabora s , quindi $r_{i+1} = \delta(r_i, s_i)$ per $1 \leq i \leq n$. Questa sequenza ha lunghezza $n + 1$ che è almeno $p + 1$. Due tra i primi $p + 1$ elementi nella sequenza devono essere lo stesso stato, per il principio della piccionaia. Chiamiamo il primo di questi r_j e il secondo r_l . Poiché r_l si presenta tra le prime $p + 1$ posizioni in una sequenza che inizia in r_1 , abbiamo $l \leq p + 1$. Ora sia $x = s_1\dots s_{j-1}$, $y = s_j\dots s_{l-1}$ e $z = s_l\dots s_n$.

Poiché x porta M da r_1 a r_j , y porta M da r_j a r_j e z porta M da r_j a r_{n+1} che è uno stato accettante, M deve accettare xy^iz per $i \geq 0$. Sappiamo che $j \neq l$, perciò $|y| > 0$, e $l \leq p + 1$, perciò $|xy| \leq p$. Quindi tutte le condizioni del pumping lemma sono rispettate.

Per usare il pumping lemma per provare che un linguaggio B non è regolare, in primo luogo si assuma che B sia regolare per ottenere una contraddizione. Poi si usi il pumping lemma per assicurare l'esistenza di una lunghezza del pumping p tale che tutte le stringhe di lunghezza maggiore o uguale a p in B possano essere iterate. In seguito, si trovi una stringa s in B che ha lunghezza maggiore o uguale a p , ma che non può essere iterata. Infine, si dimostri che s non può essere iterata considerando tutti i modi di dividere s in x , y e z (prendendo in considerazione la condizione 3 del pumping lemma se è utile) e, per ogni tale divisione, trovando un valore i tale che $xy^iz \notin B$. Questo passo finale spesso comporta il dover raggruppare i vari modi di dividere s in diversi casi e l'analizzarli individualmente. L'esistenza di s contraddirebbe il pumping lemma se B fosse regolare. Quindi B non può essere regolare.

Trovare s a volte richiede un po' di ragionamento creativo. Potresti dover cercare tra diversi candidati per s prima di scoprirne uno che funzioni. Prova con elementi di B che sembrano esibire l'"essenza" della non regolarità di B .

Esempio 1.4.1 – Pumping lemma

Sia B il linguaggio $\{0^n 1^n | n \geq 0\}$. Usiamo il pumping lemma per provare che B non è regolare. La prova è per assurdo.

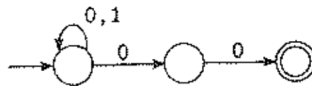
Assumiamo al contrario che B sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Scegliamo s uguale alla stringa $0^p 1^p$. Poiché s è un elemento di B ed s ha lunghezza maggiore di p , il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z$ è in B . Consideriamo tre casi per mostrare che questo risultato è impossibile.

1. La stringa y consiste solo di simboli uguali a 0. In questo caso, la stringa $xyyz$ ha più simboli uguali a 0 che simboli uguali a 1 e quindi non è un elemento di B , violando la condizione 1 del pumping lemma. Questo caso porta a una contraddizione.
2. La stringa y consiste solo di simboli uguali a 1. Anche questo caso porta a una contraddizione.
3. La stringa y consiste sia di simboli uguali a 0 che di simboli uguali a 1. In questo caso, la stringa $xyyz$ può avere lo stesso numero di simboli uguali a 0 e simboli uguali a 1, ma essi non saranno nell'ordine corretto, con qualche 1 prima di qualche 0. Quindi non è un elemento di B , il che è una contraddizione.

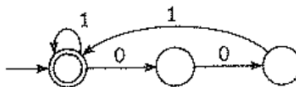
Pertanto una contraddizione è inevitabile se assumiamo che B sia regolare, quindi B non è regolare.

1.5 Esercizi svolti**Esercizio 1.5.1**

Costruire un NFA per il linguaggio $L = \{w \in \{0,1\}^* : w \text{ termina con } 00\}$ con 3 stati.

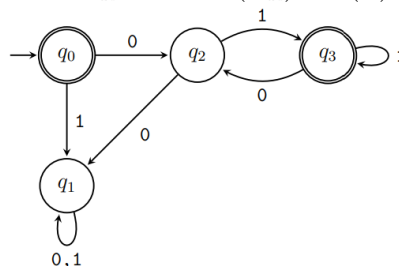
**Esercizio 1.5.2**

Sia $r = 1^*(001^+)^*$, dove $+$ significa "almeno un'occorrenza". Costruire un DFA tale che $L(D) = L(r)$ con 3 stati e alfabeto $\{0,1\}$.

**Esercizio 1.5.3**

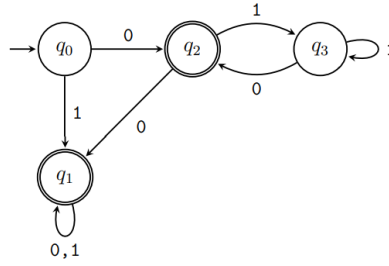
Data l'espressione regolare $R = (01^+)^*$, costruire il DFA D tale che $L(D) = \{w \in \{0,1\}^* | w \notin L(R)\}$.

Prima di tutto, costruiamo un DFA D_R tale che $L(D_R) = L(R)$.



A questo punto, ci basta costruire il DFA D tale che $L(D) = \overline{L(D_R)}$ utilizzando la chiusura del

complemento in REG.



Esercizio 1.5.4

Dato il linguaggio $L = \{w \in \{0,1\}^* : |w|_0 = |w|_1\}$, dimostrare che L non è regolare.

Dimostrazione. Supponiamo per assurdo che L sia regolare, implicando che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping.

Consideriamo quindi la stringa $w = 0^p 1^p \in L$. Poiché $|w| \geq p$, possiamo suddividerla in tre sottostringhe $x, y, z \in \Sigma^*$ tali che $w = xyz$. Poiché la terza condizione del pumping lemma impone che $|xy| \leq p$ e poiché $w := 0^p 1^p$, ne segue che $xy = 0^m$ e $z = 0^{p-m} 1^p$, dove $m \in [1, p]$. Inoltre, per la seconda condizione, si ha che $|y| > 0$, dunque necessariamente si ha che $x = 0^{m-k}$ e $y = 0^k$, dove $k \in [1, m]$. A questo punto, consideriamo la stringa xy^0z . Notiamo immediatamente che

$$xy^0z = 0^{m-k}(0^k)^0 0^{p-m} 1^p = 0^{m-k} 0^{p-m} 1^p = 0^{p-k} 1^p \implies |xy^0z|_0 \neq |xy^0z|_1 \implies xy^0z \notin L$$

contraddicendo la prima condizione del lemma per cui si ha che $\forall i \geq 0, xy^i z \in L$. Dunque, ne segue necessariamente che L non sia regolare.

Esercizio 1.5.5

Dato il linguaggio $L = \{1^{n^2} | n \geq 0\}$, dimostrare che L non è regolare.

Dimostrazione. Supponiamo per assurdo che L sia regolare, implicando che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping.

Consideriamo quindi la stringa $w := 1^{p^2} \in L$. Poiché $|w| \geq p$, possiamo suddividerla in tre sottostringhe $x, y, z \in \Sigma^*$ tali che $w = xyz$. Poiché la terza condizione del lemma impone che $|xy| \leq p$ e poiché $w := 1^{p^2}$, ne segue che $xy = 1^m$ e $z = 1^{p^2-m}$, dove $m \in [1, p]$. Inoltre, per la seconda condizione del lemma, si ha che $|y| > 0$, dunque necessariamente si ha che $x = 1^{m-k}$ e $y = 1^k$, dove $k \in [1, m]$. A questo punto, consideriamo la stringa xy^0z . Notiamo immediatamente che

$$xy^0z = 1^{m-k}(1^k)^0 1^{p^2-m} = 1^{p^2-k}$$

Tuttavia, poiché $k \in [1, p]$, ne segue che $\nexists n \geq 0 | n^2 = p^2 - k$, implicando dunque che $xy^0z \notin L$, contraddicendo la prima condizione del lemma per cui si ha che $\forall i \geq 0, xy^i z \in L$. Dunque, ne segue necessariamente che L non sia regolare.

Capitolo 2

Linguaggi context-free

In questo capitolo presentiamo grammatiche context-free, un metodo più potente per descrivere linguaggi. Queste grammatiche possono descrivere alcuni aspetti che hanno una struttura ricorsiva. Ciò le rende utili in una varietà di applicazioni.

Un'importante applicazione delle grammatiche context-free si presenta nella specifica e nella compilazione dei linguaggi di programmazione. Progettisti di compilatori e interpreti per linguaggi di programmazione spesso iniziano con l'ottenere una grammatica per il linguaggio. La maggior parte dei compilatori e degli interpreti contiene una componente chiamata parser che estrae il significato di un programma prima di generare il codice compilato o di eseguire il programma interpretato. Alcuni strumenti generano perfino automaticamente il parser dalla grammatica.

I linguaggi associati alle grammatiche context-free sono chiamati linguaggi context-free. La classe dei linguaggi context-free include tutti i linguaggi regolari e molti altri linguaggi.

2.1 Grammatiche context-free

Un esempio di grammatica context-free, che chiamiamo G_1 , è il seguente

$$A \rightarrow 0A1$$

$$A \rightarrow B$$

$$B \rightarrow \#$$

Una grammatica consiste di un insieme di regole di sostituzione, anche chiamate produzioni. Ogni regola appare come una linea nella grammatica, costituita da un simbolo e una stringa separati da una freccia. Il simbolo è chiamato una variabile. La stringa consiste di variabili e altri simboli chiamati terminali. Le variabili sono spesso rappresentate da lettere maiuscole. I terminali sono analoghi ai simboli dell'alfabeto di input e sono spesso rappresentati da lettere minuscole, numeri o simboli speciali. Una delle variabili è chiamata la variabile iniziale. Essa generalmente si trova sul lato sinistro della regola più in alto. Per esempio, la grammatica G_1 ha tre regole. Le variabili di G_1 sono A e B , e A è la variabile iniziale. I suoi terminali sono 0 , 1 e $\#$.

Una grammatica può essere usata per descrivere un linguaggio generando ogni stringa del linguaggio nel seguente modo.

1. Scrivi la variabile iniziale. È la variabile sul lato sinistro della regola più in alto, a meno che non sia diversamente specificato.
2. Trova una variabile che è stata scritta e una regola che inizia con quella variabile. Sostituisci la variabile scritta con il lato destro di quella regola.
3. Ripeti il passo 2 fino a quando non vi sono più variabili.

Per esempio, la grammatica G_1 genera la stringa $000\#111$. Una sequenza di sostituzioni per ottenere una stringa è chiamata una derivazione. Una derivazione della stringa $000\#111$ nella grammatica G_1 è:

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

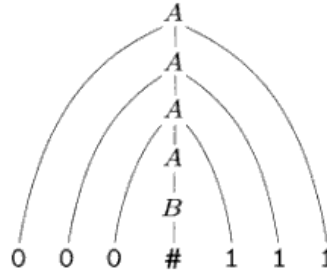


Figura 2.1: Albero sintattico per 000#111 nella grammatica G_1

La stessa informazione può essere anche rappresentata mediante un disegno con un albero sintattico (parse tree - vedi Figura 2.1).

Tutte le stringhe generate in questo modo costituiscono il linguaggio della grammatica. Denoteremo con $L(G_1)$ il linguaggio della grammatica G_1 . Alcune prove con la grammatica G_1 mostrano che $L(G_1)$ è $(0^n \# 1^n | n \geq 0)$. Ogni linguaggio che può essere generato da una grammatica context-free è chiamato un linguaggio context-free (CFL). Nel presentare una grammatica context-free, è conveniente abbreviare le diverse regole con la stessa variabile a sinistra, come $A \rightarrow 0A1$ e $A \rightarrow B$ in una singola linea $A \rightarrow 0A1 | B$, usando il simbolo "|" come un "oppure".

Formalizziamo la nostra nozione di grammatica context-free (CFG).

Definizione 2.1.1 – Grammatica context-free

Una grammatica context-free è una quadrupla (V, Σ, R, S) , dove

1. V è un insieme finito i cui elementi sono chiamati variabili,
2. Σ è un insieme finito, disgiunto da V , i cui elementi sono chiamati terminali,
3. R è un insieme finito di regole, dove ciascuna regola è una variabile e una stringa di variabili e terminali, ed
4. $S \in V$ è la variabile iniziale.

Se u, v e w sono stringhe di variabili e terminali e $A \rightarrow w$ è una regola della grammatica, diciamo che uAv produce uww , e lo denotiamo con $uAv \Rightarrow uww$. Diciamo che u deriva v , e lo denotiamo con $u \Rightarrow v$ se $u = v$ o se esiste una sequenza $u_1, u_2, \dots, u_k$ con $k \geq 0$ e

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v$$

Il linguaggio della grammatica è $\{w \in \Sigma^* | S \Rightarrow^* w\}$.

Nella grammatica G_1 : $V = \{A, B\}$, $\Sigma = \{0, 1, \#\}$, $S = A$ ed R è l'insieme delle tre regole.

2.1.1 Progettare grammatiche context-free

Come con la progettazione di automi finiti, la progettazione di grammatiche context-free richiede creatività. Le tecniche seguenti sono utili, singolarmente o combinate, quando affrontiamo il problema di costruire una CFG.

Innanzitutto, molti CFL sono l'unione di CFL più semplici. Se devi costruire una CFG per un CFL che puoi dividere in componenti più semplici, fallo e poi costruisci grammatiche separate per ciascuna componente. Queste singole grammatiche possono facilmente essere fuse in una grammatica per il linguaggio iniziale unendo le loro regole e poi aggiungendo la nuova regola $S \rightarrow S_1 | S_2 | \dots | S_k$ dove le variabili S_i sono le variabili iniziali per le grammatiche individuali. Risolvere diversi problemi più semplici è spesso più facile che risolvere un problema complicato.

Esempio 2.1.1 – Grammatiche context-free - Unione

Per esempio, per ottenere una grammatica per il linguaggio $\{0^n 1^n | n \geq 0\} \cup \{1^n 0^n | n \geq 0\}$

costruiamo prima la grammatica

$$S_1 \rightarrow 0S_11|\epsilon$$

per il linguaggio $\{0^n1^n | n \geq 0\}$ e la grammatica

$$S_2 \rightarrow 1S_20|\epsilon$$

per il linguaggio $\{1^n0^n | n \geq 0\}$ e poi aggiungiamo la regola $S \rightarrow S_1|S_2$ ottenendo la grammatica

$$S \rightarrow S_1|S_2$$

$$S_1 \rightarrow 0S_11|\epsilon$$

$$S_2 \rightarrow 1S_20|\epsilon$$

In secondo luogo, costruire una CFG per un linguaggio che sia regolare è facile se si può prima costruire un DFA per quel linguaggio. Si può trasformare un DFA in una CFG equivalente nel modo seguente. Si introduca una variabile R_i per ogni stato q_i del DFA. Si aggiunga la regola $R_i \rightarrow aR_j$ alla CFG se $\delta(q_i, a) = q_j$ è una transizione nel DFA. Si aggiunga la regola $R_i \rightarrow \epsilon$ se q_i è uno stato accetante del DFA. Si assuma R_0 come variabile iniziale della grammatica, dove q_0 è lo stato iniziale della macchina. Si può verificare che la CFG risultante genera lo stesso linguaggio di quello riconosciuto dal DFA.

Inoltre, alcuni linguaggi context-free contengono stringhe con due sottostringhe che sono "collegate", nel senso che una macchina per un tale linguaggio avrebbe bisogno di ricordare una quantità non limitata di informazione su una delle sottostringhe per verificare che essa corrisponde correttamente all'altra sottostringa. Questa situazione si verifica nel linguaggio $\{0^n1^n | n \geq 0\}$ poiché una macchina avrebbe bisogno di ricordare il numero di uguali a 0 per verificare che esso è uguale al numero di simboli uguali a 1. Si può costruire una CFG per gestire questa situazione usando una regola della forma $R \rightarrow uRv$ che genera stringhe nelle quali la porzione contenente le u corrisponde alla porzione contenente le v .

Infine, in linguaggi più complessi, le stringhe possono contenere alcune strutture che compaiono ricorsivamente come parte di altre strutture (o della stessa).

2.1.2 Ambiguità

Qualche volta una grammatica può generare la stessa stringa in più modi diversi. Una tale stringa avrà diversi alberi sintattici e quindi diversi significati.

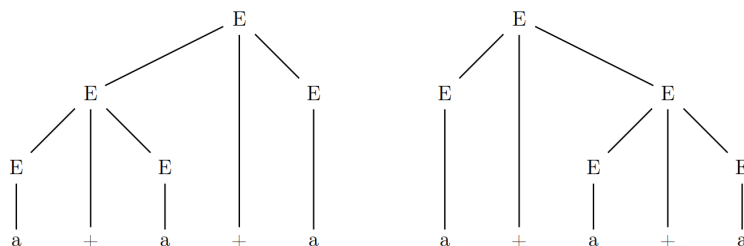
Se una grammatica genera la stessa stringa in più modi diversi, diciamo che la stringa è derivata ambigualmente in quella grammatica. Se una grammatica genera alcune stringhe ambigualmente, diciamo che la grammatica è ambigua.

Esempio 2.1.2 – Grammatica ambigua

Per esempio, consideriamo la grammatica:

$$E \rightarrow E + E | E \times E | (E) | a$$

Questa grammatica genera la stringa $a + a \times a$ ambigualmente. La figura seguente mostra i due diversi alberi sintattici.



Questa grammatica non tiene conto delle usuali regole di precedenza e quindi può raggruppare $+$ prima di $\times$ o viceversa.

Ora formalizziamo la nozione di ambiguità. Quando diciamo che una grammatica genera ambigualmente una stringa, intendiamo che la stringa ha due diversi alberi sintattici, non due differenti derivazioni. Due derivazioni possono differire solo nell'ordine in cui esse sostituiscono le variabili ma non nella loro struttura complessiva. Per concentrarci sulla struttura, definiamo un tipo di derivazione che sostituisce le variabili con un ordine stabilito. Una derivazione di una stringa w in una grammatica G è una derivazione a sinistra (leftmost derivation) se a ogni passo la variabile sostituita è quella che si trova più a sinistra.

Definizione 2.1.2 – Derivazione ambigua

Una stringa w è derivata ambigualmente in una grammatica context-free G se essa ha due o più diverse derivazioni a sinistra. Una grammatica G è ambigua se essa genera qualche stringa ambigualmente.

Qualche volta quando abbiamo una grammatica ambigua, possiamo trovare una grammatica non ambigua che genera lo stesso linguaggio. Tuttavia, alcuni linguaggi context-free possono essere generati solo da grammatiche ambigue. Tali linguaggi sono chiamati inerentemente ambigui.

2.1.3 Forma normale di Chomsky

Quando si lavora con le grammatiche context-free, è spesso conveniente averle in forma semplificata. Una delle forme più semplici e utili è chiamata la forma normale di Chomsky. La forma normale di Chomsky è utile nel dare algoritmi che lavorano con grammatiche context-free.

Definizione 2.1.3 – Forma normale di Chomsky

Una grammatica context-free è in forma normale di Chomsky se ogni regola è della forma

$$\begin{aligned} A &\rightarrow BC \\ A &\rightarrow a \end{aligned}$$

dove a è un terminale e A , B e C sono variabili qualsiasi tranne che B e C non possono essere la variabile iniziale. Inoltre, permettiamo la regola $S \rightarrow \epsilon$, dove S è la variabile iniziale.

Teorema 2.1.1 – CFG in forma normale di Chomsky

Ogni linguaggio context-free è generato da una grammatica context-free in forma normale di Chomsky.

Osservazione. Possiamo trasformare ogni grammatica G in forma normale di Chomsky. La trasformazione ha diversi passi nei quali le regole che violano le condizioni sono rimpiazzate con regole equivalenti che sono adeguate. Innanzitutto, aggiungiamo una nuova variabile iniziale. Poi, eliminiamo tutte le ϵ -regole della forma $A \rightarrow \epsilon$. Eliminiamo anche tutte le regole unitarie della forma $A \rightarrow B$. In entrambi i casi, modifichiamo la grammatica in modo da essere sicuri che generi ancora lo stesso linguaggio. Infine, trasformiamo le restanti regole nella forma adeguata.

Dimostrazione. In primo luogo, aggiungiamo una nuova variabile iniziale S_0 e la regola $S_0 \rightarrow S$ dove S era la variabile iniziale di partenza. Questo cambiamento garantisce che la variabile iniziale non compare sul lato destro di una regola.

In secondo luogo, ci occupiamo di tutte le ϵ -regole. Eliminiamo una ϵ -regola $A \rightarrow \epsilon$ dove A non è la variabile iniziale. Poi, per ogni occorrenza di A sul lato destro di una regola, aggiungiamo una nuova regola con quell'occorrenza cancellata. In altre parole, se $R \rightarrow uAv$ è una regola in cui u e v sono stringhe di variabili e terminali, aggiungiamo la regola $R \rightarrow uv$. Facciamo questo per ogni occorrenza di A , in modo che la regola $R \rightarrow uAvAw$ faccia sì che vengano aggiunte $R \rightarrow uvAw$, $R \rightarrow uAvw$ ed $R \rightarrow uvw$. Se abbiamo la regola $R \rightarrow A$ aggiungiamo $R \rightarrow \epsilon$ a meno che non avevamo precedentemente rimosso la regola $R \rightarrow \epsilon$. Ripetiamo questi passi fino a eliminare tutte le ϵ -regole che non coinvolgono la variabile

iniziale.

Inoltre, ci occupiamo delle regole unitarie. Eliminiamo una regola unitaria $A \rightarrow B$. Poi, per ogni regola $B \rightarrow u$ aggiungiamo la regola $A \rightarrow u$ a meno che questa non sia una regola unitaria precedentemente cancellata. Come prima, u è una stringa di variabili e terminali. Ripetiamo questi passi fino a eliminare tutte le regole unitarie.

Infine, trasformiamo tutte le restanti regole nella forma appropriata. Rimpiazziamo ogni regola $A \rightarrow u_1 u_2 \dots u_k$, dove $k \geq 3$ e ciascun u_i è una variabile o un simbolo terminale, con le regole $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2$, $A_2 \rightarrow u_3 A_3, \dots$ and $A_{k-2} \rightarrow u_{k-1} u_k$. Le A_i sono nuove variabili. Rimpiazziamo ogni terminale u_i nelle precedenti regole (o regola) con la nuova variabile U_i e aggiungiamo la regola $U_i \rightarrow u_i$.

Esempio 2.1.3 – Trasformazione CFG in forma normale di Chomsky

Consideriamo la seguente CFG G_6 e trasformiamola in forma normale di Chomsky. La sequenza di grammatiche presentate illustra i passi nella conversione.

1. La CFG iniziale G_6 è mostrata sulla sinistra. Il risultato dell'applicazione del primo passo, che crea una nuova variabile iniziale, è mostrato sulla destra.

$$\begin{array}{ll} S \rightarrow ASA|aB & S_0 \rightarrow S \\ A \rightarrow B|S & S \rightarrow ASA|aB \\ B \rightarrow b|\epsilon & A \rightarrow B|S \\ & B \rightarrow b|\epsilon \end{array}$$

2. Eliminiamo le ϵ -regole $B \rightarrow \epsilon$ come mostrato sulla sinistra, e $A \rightarrow \epsilon$ come mostrato sulla destra.

$$\begin{array}{ll} S_0 \rightarrow S & S_0 \rightarrow S \\ S \rightarrow ASA|ab|a & S \rightarrow ASA|aB|a|SA|AS|S \\ A \rightarrow B|S|\epsilon & A \rightarrow B|S| \notin \\ B \rightarrow b| \notin & B \rightarrow b \end{array}$$

- 3a. Eliminiamo le regole unitarie $S \rightarrow S$ come mostrato sulla sinistra, e $S_0 \rightarrow S$ come mostrato sulla destra.

$$\begin{array}{ll} S_0 \rightarrow S & S_0 \rightarrow \notin|ASA|aB|a|SA|AS \\ S \rightarrow ASA|aB|a|SA|AS| \notin & S \rightarrow ASA|aB|a|SA|AS \\ a \rightarrow B|S & A \rightarrow B|S \\ B \rightarrow b & B \rightarrow b \end{array}$$

- 3b. Eliminiamo le regole unitarie $A \rightarrow B$ e $A \rightarrow S$.

$$\begin{array}{ll} S_0 \rightarrow ASA|aB|a|SA|AS & S_0 \rightarrow ASA|aB|a|SA|AS \\ S \rightarrow ASA|aB|a|SA|AS & S \rightarrow ASA|aB|a|SA|AS \\ A \rightarrow \notin| \notin|b & A \rightarrow S|b|ASA|aB|a|SA|AS \\ B \rightarrow b & B \rightarrow b \end{array}$$

4. Trasformiamo le rimanenti regole nella forma corretta, aggiungendo variabili e regole supplementari. La grammatica finale in forma normale di Chomsky è equivalente a G_6 (In realtà la procedura fornita nel Teorema 2.1.3 crea diverse variabili U_i e diverse regole $U_i \rightarrow a$. Abbiamo semplificato la grammatica risultante usando una sola variabile U e

una sola regola $U \rightarrow a$).

$$\begin{aligned} S_0 &\rightarrow AA_1|UB|a|SA|AS \\ S &\rightarrow AA_1|UB|a|SA|AS \\ A &\rightarrow b|AA_1|UB|a|SA|AS \\ A_1 &\rightarrow SA \\ U &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

2.2 Automi a pila

In questa sezione introduciamo un nuovo tipo di modello computazionale chiamato automa a pila (pushdown automata). Questi automi sono come gli automi finiti non deterministici ma hanno una componente in più chiamata pila (stack). La pila fornisce memoria aggiuntiva oltre alla quantità finita di essa disponibile nel controllo. La pila consente a tali automi di riconoscere alcuni linguaggi non regolari.

Gli automi a pila sono computazionalmente equivalenti alle grammatiche context-free. Questa equivalenza è utile poiché essa ci dà due alternative per provare che un linguaggio è context free. Possiamo dare una grammatica context-free che lo genera oppure un automa a pila che lo riconosce. Alcuni linguaggi sono più facilmente descritti in termini di generatori, mentre altri sono più facilmente descritti da riconoscitori.

Un automa a pila (PDA) può scrivere simboli nella pila e rileggerli in seguito. Scrivere un simbolo "spinge giù" tutti gli altri simboli nella pila. In un qualunque momento il simbolo sulla cima (top) della pila può essere letto e rimosso. I rimanenti simboli allora ritornano più in alto. L'operazione di scrivere un simbolo sulla pila è spesso chiamata push, quella di eliminare un simbolo dalla pila è spesso chiamata pop. Nota che ogni accesso alla pila, sia per leggere che per scrivere, può essere fatto solo sulla sommità. In altre parole, una pila è un dispositivo di memoria "last in, first out" (ultimo entrato, primo uscito). Se alcuni dati sono scritti nella pila e dati aggiuntivi sono scritti in seguito, i dati precedenti diventano inaccessibili fino a quando quelli successivi vengono rimossi.

Una pila è molto importante perché può mantenere una quantità non limitata di informazioni. Quindi, la natura non limitata di una pila permette al PDA di memorizzare numeri di dimensioni non limitate. La seguente descrizione informale mostra come opera l'automa per questo linguaggio.

2.2.1 Definizione formale di automa a pila

La definizione di automa a pila è simile a quella di un automa finito, tranne che per la pila. La pila è un dispositivo che contiene simboli presi da un qualche alfabeto. La macchina può usare differenti alfabeti per il suo input e la sua pila, quindi ora specifichiamo sia un alfabeto di simboli di input Σ sia un alfabeto di pila Γ .

Nel cuore di ogni definizione di automa c'è la funzione di transizione, che descrive il suo comportamento. Ricordiamo che $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ e $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$. Il dominio della funzione di transizione è $Q \times \Sigma_\epsilon \times \Gamma_\epsilon$. Quindi lo stato corrente, il prossimo simbolo di input letto, e il simbolo sulla cima della pila determinano la mossa seguente di un automa a pila. L'uno o l'altro simbolo può essere ϵ , il che determina che la macchina si muova senza leggere un simbolo di input o senza leggere un simbolo dalla pila.

Per quanto riguarda il codominio della funzione di transizione, dobbiamo considerare cosa può fare l'automa quando è in una specifica situazione. Esso può trovarsi in un nuovo stato ed eventualmente scrivere un simbolo sulla cima della pila. La funzione δ può indicare quest'azione restituendo un elemento di Q insieme a un elemento di Γ_ϵ , cioè un elemento di $Q \times \Gamma_\epsilon$. Poiché permettiamo il non determinismo in questo modello, una situazione può avere diverse mosse successive lecite. La funzione di transizione ingloba il non determinismo nel modo usuale, restituendo un insieme di elementi di $Q \times \Gamma_\epsilon$, cioè un elemento di $\mathcal{P}(Q \times \Gamma_\epsilon)$. Mettendo tutto questo insieme, la nostra funzione di transizione δ assume la forma $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$.

Definizione 2.2.1 – Automa a pila

Un automa a pila è una sestupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove Q, Σ, Γ ed F sono tutti insiemi finiti, e

1. Q è l'insieme degli stati,
2. Σ è l'alfabeto dell'input,
3. Γ è l'alfabeto della pila,
4. $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ è la funzione di transizione.
5. $q_0 \in Q$ è lo stato iniziale, ed
6. $F \subseteq Q$ l'insieme degli stati accettanti.

Dato $(q, c) \in \delta(p, a, b)$ dove δ è la funzione di transizione di un PDA, si ha che:

- Se $b, c = \epsilon$ (dunque $a; \epsilon \rightarrow \epsilon$) allora l'automa leggerà a dalla stringa e passerà direttamente dallo stato p allo stato q , senza modificare lo stack;
- Se $b = \epsilon$ e $c \neq \epsilon$ (dunque $a; \epsilon \rightarrow c$) allora l'automa leggerà a dalla stringa, passerà direttamente dallo stato p allo stato q e in cima allo stack viene aggiunto il simbolo c (push);
- Se $b \neq \epsilon$ e $c = \epsilon$ (dunque $a; b \rightarrow \epsilon$) allora l'automa leggerà a e se in cima allo stack vi è b , l'automa passerà dallo stato p allo stato q e rimuoverà b dalla cima dello stack (pop).

Un automa a pila $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ computa come segue. Accetta un input w se w può essere $w = w_1 w_2 \dots w_m$, dove ciascun $w_i \in \Sigma_\epsilon$ ed esistono sequenze di stati $r_0, r_1, \dots, r_m \in Q$ di stringhe $s_0, s_1, \dots, s_m \in \Gamma^*$ che soddisfano le tre condizioni seguenti. Le stringhe s_i rappresentano la sequenza del contenuto della pila che M ha su un ramo accettante della computazione.

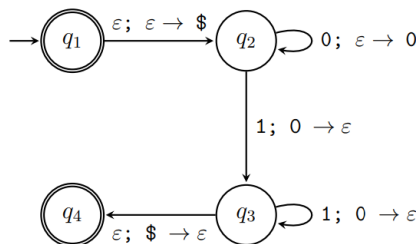
1. $r_0 = q_0$ and $s_0 = \epsilon$. Questa condizione significa che M inizia correttamente, nello stato iniziale e con una pila vuota.
2. Per $i = 0, \dots, m-1$ abbiamo $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, dove $s_i = at$ ed $s_{i+1} = bt$ qualche $a, b \in \Gamma_\epsilon$ e $t \in \Gamma^*$. Questa condizione che muove correttamente in base allo stato, al simbolo di pila, e al prossimo simbolo in input.
3. $r_m \in F$. Questa condizione afferma che alla fine dell'input M si trova in uno stato accettante.

Esempio 2.2.1 – Automa a pila

Descrivere un PDA che riconosce il linguaggio $L = \{0^n 1^n | n \geq 0\}$.

Un PDA è in grado di riconoscere questo linguaggio poiché può usare la sua pila per memorizzare il numero di simboli uguali a 0 che ha visto.

Legge i simboli in input. Scrive ciascun 0 letto sulla pila. Non appena vede simboli uguali a 1, cancella uno 0 dalla pila per ogni 1 letto. Se la lettura dell'input termina esattamente quando la pila diventa priva di simboli uguali a 0, accetta l'input. Se la pila si svuota ma restano simboli uguali a 1 o se i simboli uguali a 1 sono finiti ma la pila contiene ancora simboli uguali a 0 o se qualche 0 appare nell'input dopo i simboli uguali a 1, rifiuta l'input.

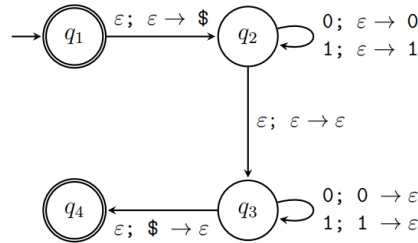


Esempio 2.2.2 – Automa a pila

Descrivere un PDA che riconosce il linguaggio $L = \{ww^R\} | w \in \{0,1\}^*$.

Ricordiamo che w^R significa w scritta al contrario.

Inizia inserendo nella pila i simboli letti. In qualunque momento ipotizza non deterministicamente di aver raggiunto la prima metà della stringa e quindi cambia azione, eliminando dalla pila un simbolo per ciascun simbolo letto, se essi coincidono. Se essi coincidono sempre e la pila si svuota nello stesso momento in cui l'input è terminato, accetta; altrimenti rifiuta.

**2.2.2 Equivalenza con le grammatiche context-free**

Ora mostriamo che grammatiche context-free e automi a pila sono computazionalmente equivalenti. Entrambi sono in grado di descrivere la classe dei linguaggi context-free. Mostreremo come trasformare ogni grammatica context-free in un automa a pila che riconosce lo stesso linguaggio e viceversa. Ricordiamo che abbiamo definito un linguaggio context-free come un linguaggio che può essere descritto con una grammatica context-free. Quindi, il nostro obiettivo è il teorema seguente.

Teorema 2.2.1 – Linguaggio context-free e automa a pila

Un linguaggio è context-free se e solo se esiste un automa a pila che lo riconosce.

Come al solito, per i teoremi "se e solo se", abbiamo due enunciati da provare, nelle due direzioni. Innanzitutto trattiamo la direzione più facile, la diretta.

Lemma 2.2.1 – Linguaggio context-free implica automa a pila

Se un linguaggio è context-free, allora esiste un automa a pila che lo riconosce.

Osservazione. Sia A un CFL. Dalla definizione sappiamo che esiste una CFG, G , che genera A . Mostriamo come trasformare G in un PDA equivalente, che chiamiamo P .

Il PDA P che ora descriviamo, opererà accettando il suo input w , se G genera tale input, determinando se esiste una derivazione per w . Ricordiamo che una derivazione è semplicemente la sequenza di sostituzioni fatte nel processo di generazione di una stringa mediante una grammatica. Ogni passo di una derivazione produce una stringa intermedia di variabili e terminali. Noi progettiamo P in modo che possa stabilire se una serie di sostituzioni che usano le regole di G possa condurre dalla variabile iniziale a w .

Il non determinismo di un PDA gli consente di ipotizzare la sequenza di sostituzioni giuste. In ogni passo della derivazione, una delle regole per una particolare variabile è scelta non deterministicamente e usata per sostituire quella variabile.

Il PDA P inizia scrivendo la variabile iniziale sulla sua pila. Esso passa attraverso una serie di stringhe intermedie, facendo una sostituzione dietro l'altra. Infine può giungere a una stringa che contiene solo simboli terminali, il che significa che ha usato la grammatica per derivare una stringa. Allora P accetta se questa stringa è identica alla stringa che ha ricevuto in input.

Il PDA può accedere solo al simbolo sulla cima della pila e questo potrebbe essere un simbolo terminale e non una variabile. Il modo per aggirare questo problema è mantenere solo parte della stringa intermedia sulla pila: i simboli che iniziano con la prima variabile nella stringa intermedia. Tutti i simboli terminali che compaiono prima della prima variabile sono subito abbinati con i simboli nella stringa di input.

1. Inserisce il simbolo marcatore $\$$ e la variabile iniziale sulla pila.

2. Ripete i seguenti passi.

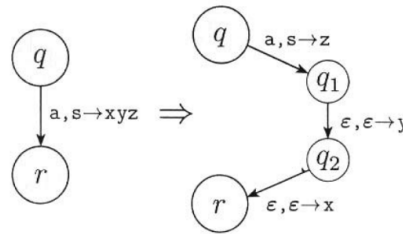
- Se sulla cima della pila c'è il simbolo di una variabile A , sceglie non deterministicamente una delle regole per A e sostituisce A con la stringa sul lato destro della regola.
- Se sulla cima della pila c'è un simbolo terminale a , legge il simbolo seguente dall'input e lo confronta con a . Se essi sono uguali, ripete. Se essi non sono uguali, rifiuta su questo ramo del non determinismo.
- Se sulla cima della pila c'è il simbolo $\$,$ entra nello stato accettante. In questo modo accetta l'input se esso è stato completamente letto.

Dimostrazione. Ora diamo i dettagli formali della costruzione dell'automa a pila $P = (Q, \Sigma, \Gamma, \delta, q_{start}, F)$. Per rendere più chiara la costruzione, usiamo una notazione abbreviata per la funzione di transizione.

Siano q ed r stati del PDA e siano $a \in \Sigma_\epsilon$, ed $s \in \Gamma_\epsilon$. Supponiamo di volere che il PDA vada da q a r quando legge a ed elimina s . Inoltre, vogliamo che esso inserisca l'intera stringa $u = u_1 \dots u_l$ sulla pila simultaneamente. Possiamo eseguire questa azione introducendo nuovi stati $q_1, \dots, q_{l-1}$ definendo la funzione di transizione come segue:

$$\begin{aligned} \delta(q, a, s) &\text{ contiene } (q_1, u_1), \\ \delta(q_1, \epsilon, \epsilon) &= \{(q_2, u_{l-1})\}, \\ \delta(q_2, \epsilon, \epsilon) &= \{(q_3, u_{l-2})\}, \\ &\dots \\ \delta(q_{l-1}, \epsilon, \epsilon) &= \{(r, u_1)\} \end{aligned}$$

Useremo la notazione (r, u) e $\delta(q, a, s)$ per denotare che quando q è lo stato in cui si trova l'automa, a è il prossimo simbolo di input ed s è il simbolo sulla cima della pila, il PDA può leggere a ed eliminare s , poi inserire la stringa u nella pila e passare nello stato r . La figura seguente ne mostra la realizzazione.



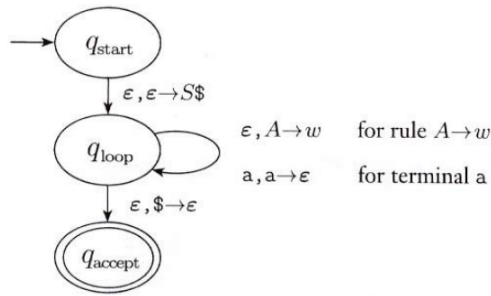
Gli stati di P sono $Q = \{q_{start}, q_{loop}, q_{accept}\} \cup E$, dove E è l'insieme degli stati necessari per realizzare l'abbreviazione appena descritta. Lo stato iniziale è q_{start} . L'unico stato accettante è q_{accept} .

La funzione di transizione è definita come segue. Cominciamo inizializzando la pila inserendo i simboli $\$$ and S , realizzando così il passo 1 nella descrizione informale: $\delta(q_{start}, \epsilon, \epsilon) = \{(q_{loop}, S\$)\}$ Poi aggiungiamo le transizioni per il ciclo principale del passo 2.

In primo luogo, trattiamo il caso (a) in cui la cima della pila contiene una variabile. Poniamo $\delta(q_{loop}, \epsilon, A) = \{(q_{loop}, w) \mid \text{dove } A \rightarrow w \text{ è una regola in } R\}$.

In secondo luogo, trattiamo il caso (b) in cui la cima della pila contiene un terminale. Poniamo $\delta(q_{loop}, a, a) = \{(q_{loop}, \epsilon)\}$.

Infine, trattiamo il caso scelto per indicare la pila vuota $\$$ è sulla cima della pila. Poniamo $\delta(q_{loop}, \epsilon, \$) = \{(q_{accept}, \epsilon)\}$.

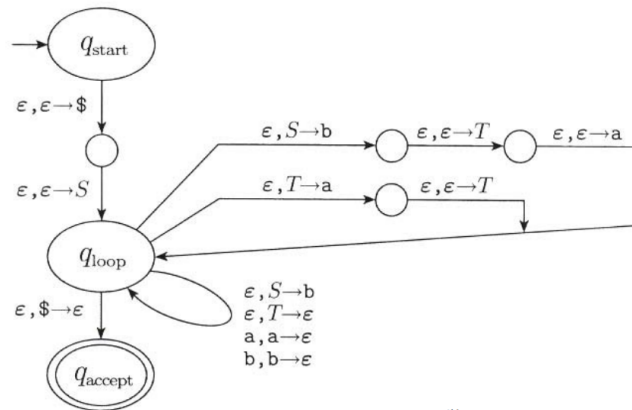


Esempio 2.2.3 – Linguaggio context-free implica automa a pila

Usiamo la procedura sviluppata nel Lemma 2.2.2 per costruire una PDA dalla seguente CFG G :

$$S \rightarrow aTb|b$$

$$T \rightarrow Ta|\epsilon$$



Ora dimostriamo la direzione inversa del Teorema 2.2.2. Ora vogliamo trasformare un PDA in una CFG.

Lemma 2.2.2 – Automa a pila implica linguaggio context-free

Se un linguaggio è riconosciuto da un automa a pila, allora esso è context-free.

Osservazione. Abbiamo un PDA P e vogliamo costruire una CFG G che genera tutte le stringhe che P accetta. In altre parole, G dovrebbe generare una stringa se quella stringa fa andare il PDA dal suo stato iniziale a uno stato accettante.

Per raggiungere questo risultato, progettiamo una grammatica che fa un pò in più. Per ciascuna coppia di stati p e $q \in P$, la grammatica avrà una variabile A_{pq} . Questa variabile genera tutte le stringhe che possano portare P da p con pila vuota a q con pila vuota. Si osservi che tali stringhe possono anche condurre P da p a q , indipendentemente dal contenuto della pila in p , lasciando la pila in q nella stessa condizione in cui era in p .

Innanzitutto, semplifichiamo il nostro compito modificando leggermente P per munirlo delle seguenti tre caratteristiche.

1. Ha un unico stato accettante, q_{accept} .
2. Svuota la sua pila prima di accettare.
3. Ciascuna transizione inserisce un simbolo sulla pila (effettua un push) o ne elimina uno dalla pila (effettua un pop), ma non fa entrambe le azioni contemporaneamente.

Dare a P le caratteristiche 1 e 2 è facile. Per munirlo della caratteristica 3, sostituiamo ciascuna transizione che contemporaneamente elimina ed inserisce simboli con una sequenza di due transizioni che attraversa un nuovo stato, e sostituiamo ogni transizione che non elimina né inserisce simboli con una sequenza di due transizioni che inserisce e poi elimina un simbolo arbitrario della pila.

Per progettare G in modo che A_{pq} generi tutte le stringhe che portano P da p a q , iniziando e terminando con una pila vuota, dobbiamo capire come P agisce su queste stringhe. Per ognuna di tali stringhe x , la prima mossa di P su x deve essere un push, poiché ogni mossa è un push o un pop e P non può eliminare da una pila vuota. Analogamente, l'ultima mossa su x deve essere un pop perché alla fine la pila deve essere vuota.

Durante la computazione di P su x si presentano due eventualità. O il simbolo eliminato alla fine è il simbolo che era stato inserito all'inizio, oppure no. Nel primo caso, la pila potrebbe essere vuota solo all'inizio e alla fine della computazione di P su x . Altrimenti, il simbolo inizialmente inserito deve essere stato eliminato in qualche punto prima della fine di x e quindi la pila si svuota in questo punto. Simuliamo la prima possibilità con la regola $A_{pq} \rightarrow aA_{rs}b$, dove a è l'input letto nella prima mossa, b è l'input letto nell'ultima mossa, r è lo stato che segue p ed s è lo stato che precede q . Simuliamo la seconda possibilità con la regola $A_{pq} \rightarrow A_{pr}A_{rq}$, dove r è lo stato in cui la pila diventa vuota.

Dimostrazione. Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{accept}\})$ e costruiamo G . L'insieme delle variabili di G è $\{A_{pq} | p, q \in Q\}$. La variabile iniziale è $A_{q_0, q_{accept}}$. Ora descriviamo le regole nei seguenti tre punti

1. Per ogni $p, q, r, s \in Q$, $u \in \Gamma$ e $a, b \in \Sigma$, se $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) , poni la $A_{pq} \rightarrow aA_{rs}b$ in G .
2. Per ogni $p, q, r \in Q$ poni la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
3. Infine, per ogni $p \in Q$ poni la regola $A_{pp} \rightarrow \epsilon$ in G .

Si può intuire questa costruzione dalle seguenti figure.

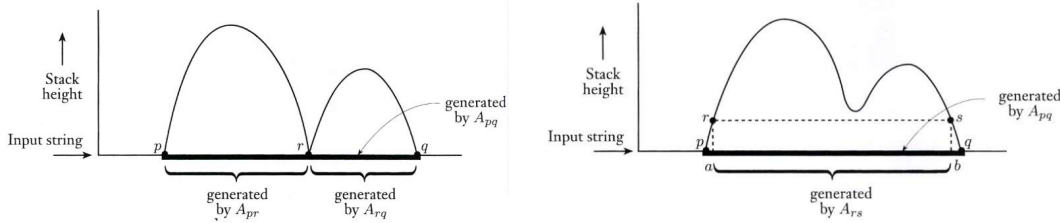


Figura 2.2: PDA corrispondenti alle regole $A_{pq} \rightarrow A_{pr}A_{rq}$ e $A_{pq} \rightarrow aA_{rs}b$

Ora proviamo che questa costruzione funziona dimostrando che A_{pq} genera x se e solo se x porta P da p con la pila vuota a q con la pila vuota. Consideriamo ogni direzione del "se e solo se" come un enunciato separato.

Lemma 2.2.3 – A_{pq} genera $x \implies x$ porta P da p a q

Se A_{pq} genera x , allora x può portare P da p con la pila vuota a q con la pila vuota.

Dimostriamo questo enunciato per induzione sul numero dei passi nella derivazione di x da A_{pq} .

- *Passo base:* La derivazione è in un solo passo. Una derivazione in un solo passo deve usare una regola il cui lato destro non contenga variabili. Le uniche regole in G con nessuna variabile sul lato destro sono $A_{pp} \rightarrow \epsilon$. Ovviamente, l'input ϵ porta P da p con la pila vuota a p con la pila vuota quindi la base è dimostrata.
- *Passo induttivo:* Assumiamo che l'enunciato sia vero per le derivazioni di lunghezza al più k , dove $k \geq 1$ e proviamo che esso è vero per le derivazioni di lunghezza $k + 1$.

Supponiamo che $A_{pq} \Rightarrow^* x$ in $k + 1$ passi. Il primo passo in questa derivazione è $A_{pq} \Rightarrow aA_{rs}b$ oppure $A_{pq} \Rightarrow A_{pr}A_{rq}$. Trattiamo questi due casi separatamente.

Nel primo caso, consideriamo la parte y di x che A_{rs} genera, quindi $x = ayb$. Poiché $A_{rs} \Rightarrow^* y$ in k passi, l'ipotesi induttiva ci dice che P può andare da r con la pila vuota a s con la pila vuota. Poiché $A_{pq} \rightarrow aA_{rs}b$ è una regola di G , $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) per qualche simbolo di pila u . Quindi, se P inizia in p con la pila vuota, dopo aver letto a può andare nello stato r e inserire u in cima alla pila. Poi, la lettura della stringa y può portarlo in se lasciare u sulla pila. In seguito dopo aver letto b può andare nello stato q ed eliminare u dalla pila. Pertanto, x può portare P da p con la pila vuota a q con la pila vuota.

Nel secondo caso, consideriamo le parti y e z di x che A_{pr} e A_{rq} rispettivamente generano, quindi $x = yz$. Poiché $A_{pr} \Rightarrow^* y$ in al più k passi e $A_{rq} \Rightarrow^* z$ in al più k passi, l'ipotesi induttiva ci dice che y può portare P da r a q , con la pila vuota all'inizio e alla fine della computazione. Quindi z può portare P da p con la pila vuota a q con la pila vuota. Questo completa la prova del passo induttivo.

Lemma 2.2.4 – x porta P da p a $q \implies A_{pq}$ genera x

Se x può portare P da p con la pila vuota a q con la pila vuota, A_{pq} genera x . Dimostriamo questo enunciato per induzione sul numero dei passi nella computazione di P da p a q con pile vuote sull'input x .

- *Passo base:* La computazione è in 0 passi.

Se una computazione è in 0 passi, essa inizia e termina nello stesso stato diciamo, p . Quindi dobbiamo mostrare che $A_{pp} \Rightarrow^* x$. P non può leggere alcun carattere in 0 passi, quindi $x = \epsilon$. Per costruzione, G ha la regola $A_{pp} \rightarrow \epsilon$, perciò la base è dimostrata.

- *Passo induttivo:* Assumiamo l'enunciato vero per le computazioni di lunghezza al più k , dove $k \geq 0$ e dimostriamo che è vero per le computazioni di lunghezza $k + 1$.

Supponiamo che P abbia una computazione dove x porta da p a q con pile vuote in $k + 1$ passi. O la pila è vuota solo all'inizio e alla fine di questa computazione oppure essa si svuota anche altrove.

Nel primo caso, il simbolo che è stato inserito nella prima mossa deve essere lo stesso simbolo che è stato rimosso nell'ultima mossa. Chiamiamo u questo simbolo. Sia a il simbolo di input letto nella prima mossa, b il simbolo di input letto nell'ultima mossa, r lo stato dopo la prima mossa ed s lo stato prima dell'ultima mossa. Allora $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) e quindi la regola $A_{pq} \rightarrow aA_{rs}b$ è in G . Sia y la parte di x senza a e b , quindi $x = ayb$. L'input y può portare P da r a s senza toccare il simbolo u che è sulla cima della pila e quindi P può andare da r a s con una pila vuota sull'input y . Abbiamo eliminato il primo e l'ultimo passo dei $k + 1$ passi nella computazione iniziale su x , perciò la computazione su y ha $(k + 1) - 2 = k - 1$ passi. Quindi l'ipotesi induttiva ci dice che $A_{rs} \Rightarrow^* y$. Allora $A_{pq} \Rightarrow^* x$.

Nel secondo caso, sia r uno stato in cui la pila oltre che all'inizio o alla fine della computazione su x . Allora le parti della computazione da p a r e da r a q contengono al più k passi. Sia y l'input letto nella prima parte e sia z l'input letto nella seconda parte. L'ipotesi induttiva ci dice che $A_{pr} \Rightarrow^* y$ e $A_{rq} \Rightarrow^* z$. Poiché la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ è in G , $A_{pq} \Rightarrow^* x$ e la dimostrazione è completa.

2.3 Linguaggi non context-free

In questa sezione presentiamo una tecnica per dimostrare che alcuni linguaggi non sono context-free. A questo punto presentiamo un pumping lemma per i linguaggi context-free. Esso stabilisce che per ogni linguaggio context-free esiste un particolare valore chiamato la lunghezza del pumping tale che tutte le stringhe più lunghe nel linguaggio possono essere "iterate". Significa che la stringa

può essere divisa in cinque parti in modo tale che la seconda e la quarta parte possono insieme essere replicate un numero qualsiasi di volte ottenendo una stringa che appartiene ancora al linguaggio.

2.3.1 Pumping lemma per i linguaggi context-free

Teorema 2.3.1 – Pumping lemma per i linguaggi context-free

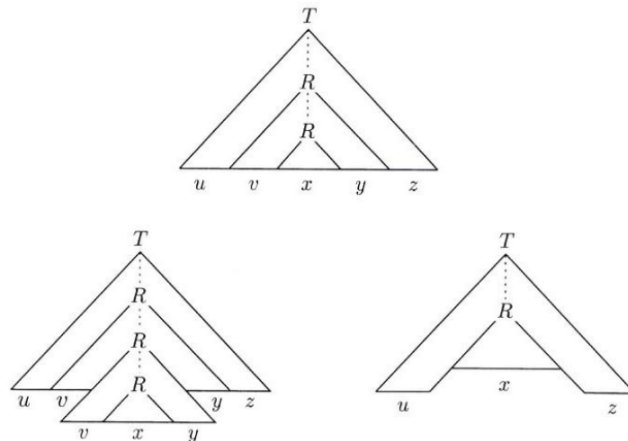
Se A è un linguaggio context-free, allora esiste un numero p (la lunghezza del pumping) tale che, se s è una qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in cinque parti $s = uvxyz$ che soddisfano le condizioni

1. per ogni $i \geq 0$, $uv^i xy^i z \in A$,
2. $|vy| > 0$ e
3. $|vxy| \leq p$.

Quando s è divisa in $uvxyz$, la condizione 2 afferma che u oppure y non è la stringa vuota. Altrimenti il teorema sarebbe banalmente vero. La condizione 3 afferma che le parti v , x e y insieme hanno lunghezza al più p .

Osservazione. Sia A un CFL e sia G una CFG che lo genera. Dobbiamo mostrare che ogni stringa sufficientemente lunga s in A può essere iterata e restare in A . L'idea dietro questa strategia è semplice.

Sia s una stringa molto lunga in A . Poiché s è in A , essa è derivabile da G e quindi ha un albero sintattico. L'albero sintattico per s deve essere molto alto perché s è molto lunga. Cioè l'albero sintattico deve contenere un cammino lungo dalla variabile alla radice dell'albero a uno dei simboli terminali su una foglia. Per il principio della piccioniatura, qualche simbolo di variabile R si deve ripetere in questo cammino lungo. Questa ripetizione ci permette di sostituire il sottoalbero sotto la seconda occorrenza di R con il sottoalbero sotto la prima occorrenza di R e ottenere ancora un albero sintattico consentito. Pertanto, possiamo dividere s in cinque parti $uvxyz$ e possiamo replicare il secondo e quarto pezzo e ottenere una stringa ancora nel linguaggio. In altre parole, $uv^i xy^i z$ è in A per ogni $i \geq 0$.



Veniamo ora ai dettagli per ottenere tutte e tre le condizioni del pumping lemma. Mostriamo anche come calcolare la lunghezza del pumping p .

Dimostrazione. Sia G una CFG per il CFL A . Sia b il massimo numero di simboli nel lato destro di una regola (assumiamo che sia almeno 2). Sappiamo che, in ogni albero sintattico costruito usando questa grammatica, un nodo non può avere più di b figli. In altre parole, ci sono al più b foglie in un passo dalla variabile iniziale; ci sono al più b^2 foglie in 2 passi dalla variabile iniziale; e ci sono al più b^h foglie in h passi dalla variabile iniziale. Quindi, se l'altezza dell'albero sintattico è al più h , la lunghezza della stringa generata è al più b^h . Viceversa, se una stringa generata ha lunghezza maggiore o uguale a $b^h + 1$ ciascuno dei suoi alberi sintattici deve avere un'altezza maggiore o uguale a $h + 1$. Sia $|V|$ il numero delle variabili in G . Poniamo p , la lunghezza del pumping, uguale a $b^{|V|+1}$. Ora se s è una stringa

in A e la sua lunghezza è maggiore o uguale a p , il suo albero sintattico deve avere altezza maggiore o uguale a $|V| + 1$ poiché $b^{(|V|+1)} \geq b^{|V|} + 1$.

Per vedere come iterare una tale stringa s , sia τ uno dei suoi alberi sintattici. Se s ha diversi alberi sintattici, scegliamo un albero sintattico τ che abbia il più piccolo numero di nodi. Sappiamo che τ deve avere altezza maggiore o uguale a $|V| + 1$ quindi il suo cammino più lungo dalla radice a una foglia ha lunghezza almeno $|V| + 1$. Questo cammino ha almeno $|V| + 2$ nodi; uno etichettato da un terminale, gli altri etichettati da variabili. Quindi questo cammino ha almeno $|V| + 1$ variabili. Poiché G ha solo $|V|$ variabili, qualche variabile R è presente più di una volta su questo cammino. Per un utilizzo successivo, scegliamo R in modo che sia una variabile che si ripete tra le $|V| + 1$ variabili più in basso su questo cammino.

Dividiamo s in $uvxyz$ come nella figura. Ogni occorrenza di R ha un sottoalbero sotto essa che genera una parte della stringa s . L'occorrenza più in alto di R ha un sottoalbero più grande e genera vxy , mentre l'occorrenza più in basso genera solo x con un sottoalbero più piccolo. Entrambi questi sottoalberi sono generati dalla stessa variabile, quindi possiamo sostituire l'uno con l'altro e ottenere ancora un albero sintattico corretto. Sostituire ripetutamente il più piccolo con il più grande fornisce gli alberi sintattici per le stringhe $uv^i xy^i z$ per ogni $i > 1$. Sostituire il più grande con il più piccolo genera la stringa uxz . Questo dimostra la condizione 1 del lemma. Ora veniamo alle condizioni 2 e 3.

Per ottenere la condizione 2, dobbiamo essere sicuri che v e y non sono entrambe ϵ . Se lo fossero, l'albero sintattico ottenuto sostituendo il più piccolo sottoalbero al più grande avrebbe meno nodi di τ e genererebbe ancora s . Questo non è possibile perché abbiamo scelto τ in modo che sia un albero sintattico per s con il più piccolo numero di nodi. Questa è la ragione per aver selezionato così τ .

Per ottenere la condizione 3, dobbiamo essere sicuri che vxy ha lunghezza al più p . Nell'albero sintattico per s l'occorrenza più in alto di R genera vxy . Abbiamo scelto R in modo che entrambe le occorrenze di essa cadano nelle $|V| + 1$ variabili più in basso del cammino e abbiamo scelto il più lungo cammino nell'albero sintattico, in modo che il sottoalbero in cui R genera vxy sia alto al più $|V| + 1$. Un albero con questa altezza può generare una stringa di lunghezza al più $b^{(|V|+1)} = p$.

Esempio 2.3.1 – Pumping lemma per i linguaggi context-free

Consideriamo il linguaggio $L = \{0^n 1^n 2^n \mid n \geq 0\}$ e dimostriamo che non è un CFL.

Supponiamo per assurdo che L sia un CFL. In tal caso, ne segue che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping. Consideriamo quindi la stringa $w := 0^p 1^p 2^p$. Poiché $|w| \geq p$, possiamo suddividerla in cinque sottostringhe $u, v, x, y, z \in \Sigma^*$ tali che $w = uvxyz$.

Poiché la terza condizione del pumping lemma impone che $|vxy| \leq p$, le uniche possibilità sono:

1. Se $vxy = 0^m$ con $1 \leq m \leq p$, si ha che $u = 0^h$ e $z = 0^{p-m-h} 1^p 2^p$, dove $1 \leq m + h \leq p$. Inoltre, poiché la seconda condizione impone che $|vy| > 0$, si ha che v e/o y contengono almeno uno 0.
2. Se $vxy = 1^m$ con $1 \leq m \leq p$, si ha che $u = 0^p 1^h$ e $z = 1^{p-m-h} 2^p$, dove $1 \leq m + h \leq p$. Inoltre, poiché la seconda condizione impone che $|vy| > 0$, si ha che v e/o y contengono almeno un 1.
3. Se $vxy = 2^m$ con $1 \leq m \leq p$, si ha che $u = 0^p 1^p$ e $z = 2^{p-m-h}$, dove $2 \leq m + h \leq p$. Inoltre, poiché la seconda condizione impone che $|vy| > 0$, si ha che v e/o y contengono almeno un 2.
4. Se $vxy = 0^m 1^h$ con $1 \leq m + h \leq p$, si ha che $u = 0^{p-m}$ e $z = 1^{p-h} 2^p$. Inoltre, poiché la seconda condizione impone che $|vy| > 0$, si ha che v contiene almeno uno 0 e/o y contiene almeno un 1.
5. Se $vxy = 1^m 2^h$ con $1 \leq m + h \leq p$, si ha che $u = 0^p 1^{p-m}$ e $z = 2^{p-h}$. Inoltre, poiché la seconda condizione impone che $|vy| > 0$, si ha che v contiene almeno uno 1 e/o y contiene almeno un 2.

In tutti i casi possibili descritti, risulta automatico che

$$\nexists n \geq 0 |n = |uv^0xy^0z|_0 = |uv^0xy^0z|_1 = |uv^0xy^0z|_2 \implies uv^0xy^0z \notin L$$

contraddicendo quindi la prima condizione del pumping lemma. Di conseguenza, ne segue necessariamente che L non è un CFL.

Esempio 2.3.2 – Pumping lemma per i linguaggi context-free

Consideriamo il linguaggio $L = \{wv|w \in \{0,1\}^*\}$ e dimostriamo che non è un CFL.

Supponiamo per assurdo che L sia un CFL. In tal caso, ne segue che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping. Consideriamo quindi la stringa $w := 0^p100^p1^p$. Poiché $|w| \geq p$, possiamo suddividerla in cinque sottostringhe $u, v, x, y, z \in \Sigma^*$ tali che $w = uvxyz$.

Poiché la terza condizione del pumping lemma impone che $|vxy| \leq p$, le uniche possibilità sono:

1. Se $u = 0^h$, $vxy = 0^m$ e $z = 0^{p-m-h}1^p0^p1^p$, dove $1 \leq m+h \leq p$, poiché la seconda condizione impone che $|vy| > 0$, si ha che v e/o y contengono almeno uno 0, dunque si ha che:

$$\exists k < m |v^0xy^0 = 0^k \implies uv^0xy^0z = 0^h0^k0^{p-m-h}1^p0^p1^p = 0^{p-m+k}1^p0^p1^p$$

dove $k < m \implies p-m-k < p$ e dunque che $uv^0xy^0z \notin L$.

2. Se $u = 0^p1^p0^h$, $vxy = 0^m$ e $z = 0^{p-m-h}1^p$, dove $1 \leq m+h \leq p$, procedendo analogamente al caso (1) otteniamo che $uv^0xy^0z \notin L$.
3. Se $u = 0^p1^h$, $vxy = 1^m$ e $z = 1^{p-m-h}0^p1^p$, dove $1 \leq m+h \leq p$, procedendo analogamente al caso (1) otteniamo che $uv^0xy^0z \notin L$.
4. Se $u = 0^p1^p0^p1^h$, $vxy = 1^m$ e $z = 1^{p-m-h}$, dove $1 \leq m+h \leq p$, procedendo analogamente al caso (1) otteniamo che $uv^0xy^0z \notin L$.
5. Se $u = 0^{p-h}$, $vxy = 0^h1^m$ e $z = 1^{p-m}0^p1^p$, dove $1 \leq m+h \leq p$, poiché la seconda condizione impone che $|vy| > 0$, si ha che v contiene almeno uno 0 e/o y contiene almeno un 1, dunque si ha che:

$$\exists j < h, j < m |v^0xy^0 = 0^j1^k \implies uv^0xy^0z = 0^{p-h}0^j1^k1^{p-m}0^p1^p = 0^{p-h+j}1^{p-m+k}0^p1^p$$

dove $j < h, k < m \implies p-h+j, p-m+k < p$ e dunque che $uv^0xy^0z \notin L$.

6. Se $u = 0^p1^p0^{p-h}$, $vxy = 0^h1^m$ e $z = 1^{p-m}$, dove $1 \leq m+h \leq p$, procedendo analogamente al caso (5) otteniamo che $uv^0xy^0z \notin L$.
7. Se $u = 0^p1^{p-h}$, $vxy = 1^h0^m$ e $z = 0^{p-m}1^p$, dove $1 \leq m+h \leq p$, poiché la seconda condizione impone che $|vy| > 0$, si ha che v contiene almeno uno 1 e/o y contiene almeno un 0, dunque si ha che:

$$\exists j < h, j < m |v^0xy^0 = 1^j0^k \implies uv^0xy^0z = 0^p1^{p-h}1^j0^k0^{p-m}1^p = 0^p1^{p-h+j}0^{p-m+k}1^p$$

dove $j < h, k < m \implies p-h+j, p-m+k < p$ e dunque che $uv^0xy^0z \notin L$.

Di conseguenza, poiché il pump down non può essere effettuato nè in un blocco di soli 0 o soli 1 (casi 1, 2, 3, 4), nè a cavallo tra degli 0 ed 1 (casi 5, 6), nè al centro della stringa (caso 7), ne segue che la prima condizione del pumping lemma venga contraddetta. Di conseguenza, ne segue necessariamente che L non è un CFL.

Esempio 2.3.3 – Chiusura della concatenazione in CFL

Consideriamo i seguenti linguaggi $L_1 = \{0^n 1^n \mid n \geq N\}$ e $L_2 = \{1^m 2^m \mid m \geq 0\}$. Consideriamo quindi le due grammatiche:

$$\begin{aligned} G_1 : A &\rightarrow 0A1 \mid \epsilon \\ G_2 : B &\rightarrow 1A0 \mid \epsilon \end{aligned}$$

tali che $L_1 = L(G_1)$ e $L_2 = L(G_2)$.

La grammatica G tale che $L(G) = L_1 \cup L_2$, corrisponderà a:

$$\begin{aligned} G : S &\rightarrow A \mid B \\ A &\rightarrow 0A1 \mid \epsilon \\ B &\rightarrow 0B1 \mid \epsilon \end{aligned}$$

La grammatica G' tale che $L(G') = L_1 \circ L_2$, corrisponderà a:

$$\begin{aligned} G : S &\rightarrow AB \\ A &\rightarrow 0A1 \mid \epsilon \\ B &\rightarrow 0B1 \mid \epsilon \end{aligned}$$

Esempio 2.3.4 – Non chiusura del complemento in CFL

Consideriamo il seguente linguaggio $L = \{a, b\}^* - \{ww \mid w \in \{a, b\}^*\}$. Consideriamo quindi la seguente grammatica

$$\begin{aligned} G : S &\rightarrow A \mid B \mid AB \mid BA \\ A &\rightarrow a \mid aAa \mid aAb \mid bAa \mid bAb \\ B &\rightarrow b \mid aBa \mid aBb \mid bBa \mid bBb \end{aligned}$$

Data $x \in L$ tale che $|x|$ sia dispari, notiamo che se il simbolo centrale di x è a , allora $S \Rightarrow A \Rightarrow^* x$, se il simbolo centrale di x è b , allora $S \Rightarrow B \Rightarrow^* x$, dunque ne segue che $x \in L(G)$. Viceversa, data $x \in L(G)$ tale che $|x|$ sia dispari, ne segue immediatamente che $\nexists w \in \{a, b\}^* \mid x = ww \Rightarrow x \in L$. Sia quindi $x \in L$ tale che $|x|$ sia pari. Dati $x_1, \dots, x_n \in a, b$ tali che $x = x_1 \dots x_n$, ne segue che:

$$x \in L \Rightarrow \exists i \in [1, n] \mid x_i \neq x_{\frac{n}{2}+i}$$

Siano quindi $u := x_1 \dots x_{\frac{n}{2}-1}$ e $v := x_{\frac{n}{2}+1} \dots x_n$. Notiamo che il simbolo centrale di u corrisponde a $x_{\frac{1+\frac{n}{2}-1}{2}}$, mentre quello di v corrisponde a $x_{\frac{2i+n}{2}} = x_{\frac{n}{2}+i}$, da cui traiamo che:

$$x_i \neq x_{\frac{n}{2}+i} \Rightarrow x_{\frac{1+\frac{n}{2}-1}{2}} = x_i \neq x_{\frac{2i+n}{2}} \Rightarrow u \neq v$$

Inoltre, notiamo che $|u|$ e $|v|$ siano dispari, dunque si ha che $u, v \in L(G)$. Di conseguenza, otteniamo che:

$$S \Rightarrow AB \Rightarrow^* uv = x \text{ oppure } S \Rightarrow BA \Rightarrow^* uv = x$$

implicando quindi che $x \in L(G)$.

Sia quindi $x \in L(G)$ tale che $|x|$ sia pari. Poiché $|x|$ è pari, ne segue necessariamente che:

$$S \Rightarrow AB \Rightarrow^* x \text{ oppure } S \Rightarrow BA \Rightarrow^* x$$

Poiché i due casi sono analoghi, senza perdita di generalità consideriamo il caso in cui $S \Rightarrow AB \Rightarrow^* x$. Siano quindi $u := x_1 \dots x_k$ e $v := x_{k+1} \dots x_n$ tali che $x = uv$, $S \Rightarrow A \Rightarrow^* u$ e $S \Rightarrow B \Rightarrow^* v$. Poiché $S \Rightarrow A \Rightarrow^* u$ e $S \Rightarrow B \Rightarrow^* v$, otteniamo che: $|u| = k$ e $|v| = n - k$ sono dispari; $S \Rightarrow A \Rightarrow^* u$ implica che il simbolo centrale di u sia a , ossia che $x_{\frac{1+k}{2}} = a$; $S \Rightarrow B \Rightarrow^* v$ implica che il simbolo centrale di v sia b , ossia che $x_{\frac{k+1+n}{2}} = b$.

Siano quindi che $w := w_1 \dots w_h$ e $w' := w'_1 \dots w'_h$ tali che $|w| = |w'| = h$ e che $u = ww'$, implicando dunque che $h = \frac{n}{2}$. Per il risultato precedente, ne segue automaticamente che:

$$w_{\frac{1+h}{2}} = x_{\frac{1+k}{2}} = a \neq b = x_{\frac{k+1+n}{2}} = w'_{\frac{1+h}{2}} \implies w \neq w' \implies x = ww' \in L$$

Dunque, abbiamo ottenuto $L = L(G) \in CFL$. Il complemento di tale linguaggio risulta essere $\bar{L} = \{ww | w \in \{a, b\}^*\}$, il quale abbiamo già dimostrato non essere un linguaggio acontestuale. Di conseguenza, concludiamo che $L \in CFL$, ma $\bar{L} \notin CFL$.

Parte II

Teoria della computabilità

Capitolo 3

La tesi di Church-Turing

3.1 Macchine di Turing

Introduciamo ora un modello molto più potente, proposto inizialmente da Alan Turing nel 1936, chiamato macchina di Turing. Simile ad un automa finito, ma con una memoria illimitata e senza restrizioni, una macchina di Turing è un modello molto più preciso di un computer. Una macchina di Turing può fare tutto ciò che può fare un computer reale.

Una macchina di Turing utilizza un nastro infinito come propria memoria illimitata. Ha una testina che è in grado di leggere e scrivere simboli ed è libera di muoversi lungo il nastro. Inizialmente il nastro contiene solo la stringa di input e tutto il resto è vuoto. Se la macchina deve memorizzare un'informazione, la può scrivere sul nastro. Per leggere l'informazione che ha scritto, la macchina può spostare la testina su di essa. La macchina continua a computare finché non decide di produrre un output. Gli output "accetta" e "rifiuta" sono ottenuti occupando appositi stati di accettazione e di rifiuto. Se non raggiunge uno stato di accettazione o di rifiuto, la macchina andrà avanti per sempre, senza mai fermarsi.

Quindi una macchina di Turing:

1. può sia scrivere che leggere sul nastro;
2. ha una testina di lettura-scrittura può muoversi sia verso sinistra che verso destra;
3. ha un nastro infinito;
4. ha degli stati speciali di accettazione e rifiuto che hanno effetto immediato.

Esempio 3.1.1 – Macchina di Turing

Introduciamo una macchina di Turing M_1 per testare l'appartenenza al linguaggio $B = \{w\#w \mid w \in \{0,1\}^*\}$.

Vogliamo che M_1 accetti se l'input è un elemento di B e che rifiuti altrimenti, cioè se l'input comprende due stringhe identiche separate da un simbolo $\#$. La strategia ovvia è quella di muoversi a zig-zag in maniera simmetrica attorno al simbolo $\#$ e stabilire se gli elementi nelle posizioni di ugual indice su i due lati corrispondono. Contrassegnate poi il nastro per tenere traccia degli elementi che corrispondono.

Progettiamo M_1 in modo da lavorare in questo modo. Esegue più passaggi sulla stringa in input con la testina di lettura-scrittura. Ad ogni passaggio verifica la corrispondenza di un carattere ad ogni lato del simbolo $\#$. Per tenere traccia dei simboli già controllati, M_1 barra ogni simbolo già esaminato. Se ha barrato tutti i simboli, significa che tutti i simboli corrispondevano ed M_1 va in uno stato di accettazione. Se si accorge di una mancata corrispondenza, M_1 entra in uno stato di rifiuto.

3.1.1 Definizione formale di macchina di Turing

Il cuore della definizione di una macchina di Turing è la funzione di transizione, perché essa ci dice come la macchina effettua un passo. Per una macchina di Turing, δ prende la forma $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. Cioè, quando la macchina occupa un certo stato q e la testina punta alla casella del

nastro contenente un simbolo a , e se $\delta(q, a) = (r, b, L)$ la macchina scrive il simbolo b al posto di a , e passa nello stato r . La terza componente è L oppure R ed indica se la testina si sposta verso sinistra o verso destra dopo la scrittura. In questo caso la L indica una mossa a sinistra.

Definizione 3.1.1 – Macchina di Turing

Una macchina di Turing è una 7-tupla, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, dove Q, Σ, Γ sono tutti insiemi finiti e

1. Q è l'insieme degli stati,
2. Σ è l'alfabeto di input non contenente il simbolo blank $\sqcup$,
3. Γ è l'alfabeto del nastro, con $\sqcup \in \Gamma$ e $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ è la funzione di transizione,
5. $q_0 \in Q$ è lo stato iniziale,
6. $q_{\text{accept}} \in Q$ è lo stato di accettazione e
7. $q_{\text{reject}} \in Q$ è lo stato di rifiuto, con $q_{\text{reject}} \neq q_{\text{accept}}$.

Una macchina di Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ computa nel seguente modo. Inizialmente M riceve il suo input $w = w_1 w_2 \dots w_n \in \Sigma^*$ sulle n celle più a sinistra del nastro, mentre il resto del nastro è vuoto (cioè, contiene tutti simboli blank). La testina parte dalla posizione più a sinistra del nastro. Si noti che Σ non contiene il simbolo blank, in tal modo il primo simbolo blank che compare sul nastro segna la fine dell'input. Quando M inizia la computazione, questa procede secondo le regole descritte dalla funzione di transizione. La computazione prosegue fin quando la macchina raggiunge lo stato di accettazione oppure di rifiuto, a tal punto si ferma. Se nessuno dei due stati viene raggiunto, M va avanti per sempre.

Durante la computazione di una macchina di Turing, si verificano cambiamenti dello stato corrente, del contenuto corrente del nastro, e della posizione corrente della testina. Un'impostazione di questi tre elementi è chiamata una configurazione della macchina di Turing. Le configurazioni sono spesso rappresentate in modo speciale. Per uno stato q e due stringhe u e v sull'alfabeto Γ del nastro, scriviamo uqv per indicare la configurazione, dove lo stato corrente è q , il contenuto corrente del nastro è uv e la posizione attuale della testina è il primo simbolo di v . Dopo l'ultimo simbolo di v , il nastro contiene solo simboli blank.

Si dice che la configurazione C_1 produce la configurazione C_2 se la macchina di Turing può passare da C_1 a C_2 in un unico passo. Definiamo formalmente questa nozione nel seguente modo. Supponiamo di avere a, b e c in Γ , così come u e v in Γ^* e gli stati q_i e q_j . In tal caso $uaq_i bu$ e $uq_j acv$ sono due configurazioni. Diciamo che

$$uaq_i bv \text{ produce } uq_j acv$$

se nella funzione di transizione $\delta(q_i, b) = (q_j, c, L)$. Questo nel caso in cui la macchina di Turing effettua verso sinistra. Per uno spostamento a destra, diciamo che

$$uaq_i bv \text{ produce } uacq_j v \tag{3.1}$$

Una macchina di Turing M accetta l'input w se esiste una sequenza di configurazioni $C_1, C_2, \dots, C_k$ tale che

1. C_1 è la configurazione iniziale di M su input w ,
2. ogni C_i produce C_{i+1} e
3. C_k è una configurazione di accettazione.

L'insieme di stringhe che M accetta rappresenta il linguaggio di M , o il linguaggio riconosciuto da M , denotato con $L(M)$.

Definizione 3.1.2 – Linguaggio Turing-riconoscibile

Un linguaggio si dice Turing-riconoscibile se esiste una macchina di Turing che lo riconosce.

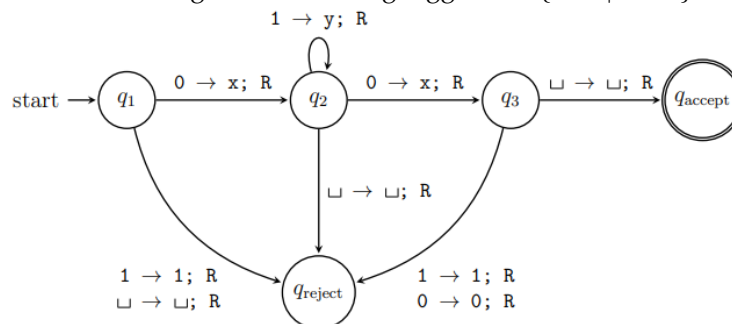
Quando attiviamo una macchina di Turing su un input, tre risultati sono possibili. La macchina può accettare, rifiutare, oppure andare in loop (ciclare). Per loop si intende semplicemente che la macchina non si ferma. Un ciclo, in quanto tale, comporta un qualche comportamento semplice o complesso che non porta ad uno stato di arresto. Una macchina che non va in loop è detta decisore perché prende in ogni caso una decisione, sia essa di accettare o di rifiutare.

Definizione 3.1.3 – Linguaggio Turing-decidibile

Un linguaggio si dice Turing-decidibile o semplicemente decidibile se esiste una macchina di Turing che lo decide.

Esempio 3.1.2 – Linguaggio Turing-riconoscibile

La seguente macchina di Turing riconosce il linguaggio $L = \{01^n0 | n \geq 0\}$.



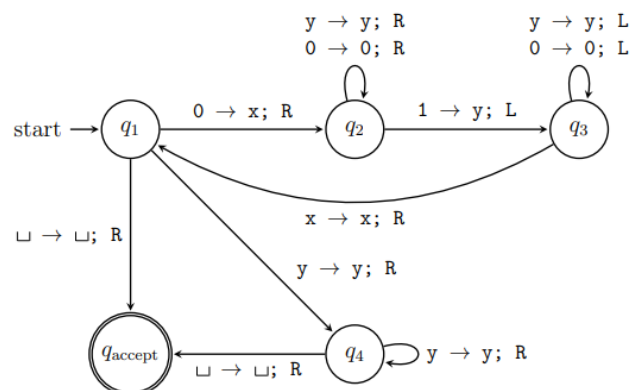
Infatti, durante la lettura della stringa 01110, la macchina assume le seguenti configurazioni:

```

q1 0 1 1 1 0
x q2 1 1 1 0
x y q2 1 1 0
x y y q2 1 0
x y y y q2 0
x y y y x q3 ␣
x y y y x ␣ qaccept ␣
  
```

Esempio 3.1.3 – Linguaggio Turing-riconoscibile

La seguente macchina di Turing riconosce il linguaggio $L = \{0^n1^n | n \geq 0\}$.



Infatti, durante la lettura della stringa 000111, la macchina assume le seguenti configurazioni:

$q_1 0 0 0 1 1 1$	$x q_1 x 0 y 1 1$	$x x q_1 0 y y 1$	
$x q_2 0 0 1 1 1$	$x x q_2 0 y 1 1$	$x x x q_2 y y 1$	$x x x q_3 y y y$
$x 0 q_2 0 1 1 1$	$x x 0 q_2 y 1 1$	$x x x y q_2 y 1$	$x x x y q_4 y y$
$x 0 0 q_2 1 1 1$	$x x 0 y q_2 1 1$	$x x x y y q_2 1$	$x x x y y q_4 y$
$x 0 q_3 0 y 1 1$	$x x 0 q_3 y y 1$	$x x x y q_3 y y$	$x x x y y y q_4 \sqcup$
$x q_3 0 0 y 1 1$	$x x q_3 0 y y 1$	$x x x q_3 y y y$	$x x x y y y \sqcup q_{accept} \sqcup$
$q_3 x 0 0 y 1 1$	$x q_3 x 0 y y 1$	$x x q_3 x y y y$	

3.2 Varianti di macchine di turing

Esistono molte definizioni alternative di macchine di Turing, comprese le versioni multinastro o non deterministiche. Sono dette varianti del modello macchina di Turing. Il modello originale e le sue ragionevoli varianti hanno tutti lo stesso potere computazionale riconoscono la stessa classe di linguaggi.

Per illustrare la robustezza del modello di macchina di Turing cerchiamo di variare il tipo di funzione di transizione consentito. Nella nostra definizione, la funzione di transizione forza la testina a spostarsi verso sinistra o destra ad ogni passo; la testina non può semplicemente restare ferma. Supponiamo di aver permesso alla macchina di Turing la capacità di restare ferma. La funzione di transizione avrebbe allora la forma $\delta : Q \times Q \times \Gamma \times \{L, R, S\}$. Siamo in grado di convertire qualsiasi TM con la possibilità di "restar ferma" in una che non ha tale capacità. Lo facciamo sostituendo ogni transizione "resta ferma" con due transizioni, una che sposta la testina a destra e una che la riporta a sinistra. Questo piccolo esempio contiene la chiave per mostrare l'equivalenza di varianti di TM. Per dimostrare che due modelli sono equivalenti abbiamo semplicemente bisogno di dimostrare che ognuno dei due può simulare l'altro.

3.2.1 Macchina di Turing multinastro

Una macchina di Turing multinastro è come una normale macchina di Turing con vari nastri. Ogni nastro ha la sua testina per la lettura e la scrittura. Inizialmente l'input si trova sul nastro 1, mentre gli altri nastri sono vuoti. La funzione di transizione viene modificata per consentire la lettura, la scrittura e lo spostamento della testina contemporaneamente su alcuni o tutti i nastri. Formalmente,

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

dove k è il numero di nastri. L'espressione

$$\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, \dots, b_k, L, R, \dots, L)$$

significa che, se la macchina si trova nello stato q_i e le testine da 1 a k leggono i simboli da a_1 ad a_k , la macchina va nello stato q_j scrive i simboli da b_1 a b_k e muove ogni testina a sinistra o a destra, o la fa restare ferma, come specificato.

Teorema 3.2.1 – Turing multinastro è equivalente a Turing singolo nastro

Per ogni macchina di Turing multinastro esiste una macchina di Turing a nastro singolo equivalente.

Dimostrazione. Mostriamo come convertire una macchina di Turing multinastro M in una TM S equivalente a nastro singolo. L'idea chiave è quella di mostrare come simulare M con S . Supponiamo che M abbia k nastri. Allora S simula l'effetto di k nastri memorizzando le loro informazioni sul suo singolo nastro. Essa utilizza il nuovo simbolo $\#$ come delimitatore per separare i contenuti dei diversi nastri. Oltre al contenuto dei nastri, S deve tenere traccia delle posizioni delle testine. Lo fa scrivendo un simbolo con un punto sopra per contrassegnare la posizione in cui si troverebbe la testina di quel nastro. Pensate a questi come nastri e testine "virtuali". Come in precedenza, i simboli del nastro "puntati" sono semplicemente simboli nuovi che sono stati aggiunti all'alfabeto del nastro.

$S = \text{"Su input } w = w_1 \dots w_n:$

1. Inizialmente S mette il suo nastro nel formato che rappresenta tutti i k nastri di M . Il

nastro formattato contiene

$$\# \overset{\bullet}{w}_1 \overset{\bullet}{w}_2 \dots \overset{\bullet}{w}_n \# \square \square \square \dots \square$$

2. Per simulare una singola mossa, S scansiona il suo nastro dal primo $\#$, che segna l'estremità sinistra, al $(k + 1)$ mo $\#$, che segna l'estremità di destra, per determinare i simboli puntati dalle testine virtuali. Successivamente S fa un secondo passaggio per aggiornare i nastri in accordo alla funzione di transizione di M .
3. Se in qualsiasi momento S sposta una delle testine virtuali a destra su un $\#$, questa azione significa che M ha spostato la testina corrispondente sulla parte di nastro vuota non letta in precedenza. Quindi S scrive un simbolo blank in questa cella del nastro e sposta il contenuto del nastro, da questa cella fino al simbolo $\#$ più a destra, di una unità a destra. Poi prosegue la simulazione come prima."

Corollario 3.2.1 – Linguaggio Turing-riconoscibile e Turing multinastro

Un linguaggio è Turing-riconoscibile se e solo se qualche macchina di Turing multinastro lo riconosce.

Dimostrazione. Un linguaggio Turing-riconoscibile è riconosciuto da una macchina di Turing ordinaria (ad un solo nastro), che è un caso particolare di una macchina di Turing multinastro. Questo prova un verso di questo corollario. L'altro verso segue dal Teorema 3.2.1.

3.2.2 Macchina di Turing non deterministica

In una macchina di Turing non deterministica, in qualsiasi punto della computazione, la macchina può procedere effettuando varie scelte. La funzione di transizione di una macchina di Turing non deterministica è della forma

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

La computazione di una macchina di Turing non deterministica è un albero i cui rami corrispondono alle diverse scelte possibili previste per la macchina. Se qualche ramo della computazione porta allo stato di accettazione, allora la macchina accetta l'input. Ora mostriamo che il non determinismo non influisce sulla potenza di calcolo del modello macchina di Turing.

Teorema 3.2.2 – Turing non deterministica è equivalente a Turing deterministica

Per ogni macchina di Turing non deterministica esiste una macchina di Turing deterministica equivalente.

Osservazione. Possiamo simulare qualsiasi TM non deterministica N con una TM deterministica D . L'idea di base della simulazione consiste nel provare tutte le possibili scelte che può fare N durante la sua computazione non deterministica. Se D trova lo stato di accettazione su uno qualsiasi di questi rami, allora D accetta. Altrimenti, la simulazione di D non terminerà. Guardiamo alla computazione di N su un input w come ad un albero. Ogni ramo dell'albero rappresenta una scelta non deterministica. Ogni nodo dell'albero è una configurazione di N . La radice dell'albero è la configurazione iniziale. La TM D esplora questo albero alla ricerca di una configurazione di accettazione. Eseguire accuratamente questa ricerca è cruciale perché con una ricerca in profondità D potrebbe non visitare l'intero albero, entrando in un loop. Quindi progettiamo D in modo da esplorare l'albero utilizzando, invece una visita in ampiezza (breadth-first search). Questa strategia esplora tutti i cammini che terminano alla stessa profondità prima di andare a esplorare ogni cammino che termina alla profondità successiva. Questo metodo garantisce che D visita ogni nodo dell'albero finché non incontra una configurazione di accettazione.

Dimostrazione. La TM D deterministica che simula ha tre nastri. La macchina D utilizza i suoi tre nastri in maniera particolare. Il nastro 1 contiene sempre la stringa di input e non viene mai modificato. Il nastro 2 mantiene una copia del nastro di N corrispondente a qualche diramazione della sua computazione non deterministica. Il nastro 3 tiene traccia della posizione di D nell'albero delle computazioni di N .

1. Inizialmente il nastro 1 contiene l'input w e i nastri 2 e 3 sono vuoti.
2. Copia il nastro 1 sul nastro 2 ed inizializza la stringa sul nastro 3 a ϵ .
3. Utilizza il nastro 2 per simulare N con input w su una ramificazione della sua computazione non deterministica. Prima di ogni passo di N , consulta il simbolo successivo sul nastro 3 per determinare quale scelta fare tra quelle consentite dalla funzione di transizione di n . Se non rimangono più simboli sul nastro 3 o se questa scelta non deterministica non è valida, interrompe questo cammino andando alla fase 4. Va alla fase 4 anche quando si verifica una configurazione di rifiuto. Se incontra una configurazione di accettazione, allora accetta l'input.
4. Sostituisce la stringa sul nastro 3 con la stringa successiva rispetto all'ordine sulle stringhe. Simula la ramificazione successiva della computazione di N andando al passo 2.

3.2.3 Enumeratori

Definito in modo informale, un enumeratore è una macchina di Turing con una stampante collegata. La macchina di Turing può utilizzare tale stampante come dispositivo di output per stampare stringhe. Ogni volta che la macchina di Turing vuole aggiungere una stringa alla lista, invia la stringa alla stampante. Un enumeratore E inizia con un nastro di input vuoto. Se l'enumeratore non si ferma, esso può stampare un elenco infinito di stringhe. Inoltre, E potrebbe generare le stringhe del linguaggio in qualsiasi ordine, eventualmente anche con ripetizioni.

Teorema 3.2.3 – Linguaggio Turing-riconoscibile $\iff \exists$ un enumeratore

Un linguaggio è Turing-riconoscibile se e solo se esiste un enumeratore che lo enumera.

Dimostrazione. Come prima cosa, dimostriamo che se abbiamo un enumeratore E che enumera un linguaggio A , allora esiste una TM M che riconosce A . La TM M funziona come segue.

$M =$ "Su input w :

1. Esegue E . Ogni volta che E genera una stringa, la confronta con w .
2. Se w appare nell'output di E , accetta."

Chiaramente, M accetta quelle stringhe che compaiono sulla lista di E ". Ora occupiamoci dell'altra direzione. Se la TM M riconosce un linguaggio A , possiamo costruire il seguente enumeratore E per A . Sia $s_1, s_2, s_3, \dots$ una lista di tutte possibili stringhe in Σ^* .

$E =$ "Ignora l'input.

1. Ripete i seguenti passi per $i = 1, 2, 3, \dots$
2. Esegue M per i passi su ogni input, $s_1, s_2, \dots, s_i$.
3. Se una qualche computazione accetta, stampa la corrispondente s_j ."

Se M accetta una particolare stringa s , alla fine s apparirà nella lista generata da E . In realtà, essa apparirà nella lista un numero infinito di volte, perché M ritorna all'inizio su ogni stringa per ogni ripetizione del passo 1. Questa procedura fa sì che M lavori in parallelo su tutte le possibili stringhe di input.

Capitolo 4

Dedicibilità

In questo capitolo iniziamo ad investigare la potenza degli algoritmi nella risoluzione di problemi. Dimosteremo che alcuni problemi possono essere risolti in maniera algoritmica ed altri no. Il nostro obiettivo è esplorare i limiti della risolvibilità algoritmica dei problemi.

4.1 Linguaggi decidibili

In questa sezione diamo alcuni esempi di linguaggi che sono decidibili mediante algoritmi.

4.1.1 Problemi decidibili relativi a linguaggi regolari

Iniziamo con alcuni problemi computazionali relativi ad automi finiti.

Ad esempio, il problema dell'accettazione per DFA consiste nel testare se un particolare automa finito deterministico accetta una data stringa, può essere espresso come un linguaggio, A_{DFA} . Questo linguaggio contiene le codifiche di tutti i DFA con le stringhe che essi accettano. Definiamo

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ è un DFA che accetta la stringa di input } w \}$$

Il problema di verificare se un DFA B accetta l'input w coincide con il problema di verificare se $\langle B, w \rangle$ è un elemento del linguaggio A_{DFA} . Mostrare che il linguaggio è decidibile equivale a mostrare che il problema computazionale è decidibile.

Teorema 4.1.1 – A_{DFA} è un linguaggio decidibile

A_{DFA} è un linguaggio decidibile.

Osservazione. Abbiamo semplicemente bisogno di presentare una TM M che decide A_{DFA} . $M =$ "Su input $\langle B, w \rangle$, dove B è un DFA e w è una stringa:

1. Simula B su input w .
2. Se la simulazione termina in uno stato di accettazione, accetta. Se non termina in uno stato di accettazione, rifiuta."

Dimostrazione. In primo luogo, esaminiamo l'input $\langle B, w \rangle$. Si tratta di una rappresentazione di un DFA B insieme ad una stringa w . Una rappresentazione ragionevole di B è semplicemente una lista delle sue cinque componenti: Q, Σ, δ, q_0 , ed F . Quando M riceve il suo input, per prima cosa verifica se esso rappresenta correttamente un DFA B ed una stringa w . In caso contrario, M rifiuta. Poi M effettua direttamente la simulazione. Tiene traccia dello stato corrente di B e della posizione corrente di B nell'input w scrivendo queste informazioni sul suo nastro. Inizialmente, lo stato corrente di B è q_0 e la posizione corrente dell'input di B è il simbolo più a sinistra di w . Gli stati e le posizioni vengono aggiornati in base alla funzione di transizione specificata δ . Quando M termina l'elaborazione dell'ultimo simbolo di w , M accetta l'input se B è in uno stato di accettazione; M rifiuta l'input se B non è in uno stato di accettazione.

Possiamo dimostrare un teorema simile per automi a stati finiti non deterministici. Dato

$$A_{NFA} = \{ \langle B, w \rangle \mid B \text{ è un NFA che accetta la stringa di input } w \}$$

Teorema 4.1.2 – A_{NFA} è un linguaggio decidibile

A_{NFA} è un linguaggio decidibile.

Dimostrazione. Presentiamo una TM N che decide A_{NFA} . Potremmo progettare N in modo che operi come M , simulando un NFA invece di un DFA. Invece, procederemo in modo diverso per illustrare una nuova idea: N usa M come una sottoprocedura. Poiché M è progettata per funzionare con un DFA, N prima converte l'NFA che riceve come input in un DFA e poi lo passa ad M .

In modo analogo, possiamo determinare se un'espressione regolare genera una data stringa

$$A_{REG} = \{ \langle R, w \rangle \mid R \text{ è un'espressione regolare che genera la stringa } w \}$$

Teorema 4.1.3 – A_{REG} è un linguaggio decidibile

A_{REG} è un linguaggio decidibile.

Dimostrazione. La seguente TM P decide A_{REG} .

$P =$ "Su input $\langle R, w \rangle$, dove R è un'espressione regolare e w è una stringa:

1. Converti l'espressione regolare R in un NFA A equivalente, mediante la procedura di conversione.
2. Esegui la TM N su input $\langle A, w \rangle$.
3. Se N accetta, accetta; se N rifiuta, rifiuta."

I Teoremi 4.1.1, 4.1.1 e 4.1.1 ci dicono che, ai fini della decidibilità, è equivalente presentare alla macchina di Turing un DFA, un NFA, oppure un'espressione regolare, perché la macchina è in grado di convertire una codifica nell'altra.

Ora affrontiamo un diverso tipo di problema concernente gli automi a stati finiti: il test del vuoto per il linguaggio di un automa finito. Dobbiamo determinare se un automa finito accetta una qualche stringa. Consideriamo

$$E_{DFA} = \{ \langle A \rangle \mid A \text{ è un DFA e } L(A) = \emptyset \}$$

Teorema 4.1.4 – E_{DFA} è un linguaggio decidibile

E_{DFA} è un linguaggio decidibile.

Dimostrazione. Il DFA accetta almeno una stringa se e solo se dallo stato iniziale può raggiungere uno stato di accettazione percorrendo il verso delle frecce del DFA. Per verificare questa condizione possiamo progettare una TM T che utilizza un algoritmo di marcatura.

$T =$ "Su input $\langle A \rangle$, dove A è un DFA:

1. Marca lo stato iniziale di A .
2. Ripete fino a quando non vengono più marcati nuovi stati: marca qualsiasi stato che ha una transizione proveniente da uno stato già marcato.
3. Se nessuno stato di accettazione risulta marcato, accetta; altrimenti, rifiuta."

Il prossimo teorema afferma che determinare se due DFA riconoscono lo stesso linguaggio è decidibile. Sia

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ sono DFA e } L(A) = L(B) \}$$

Teorema 4.1.5 – $E_{Q_{DFA}}$ è un linguaggio decidibile

$E_{Q_{DFA}}$ è un linguaggio decidibile.

Dimostrazione. Per dimostrare questo teorema usiamo il Teorema 4.1.1. Costruiamo un nuovo DFA C a partire da A e B , dove C accetta solo quelle stringhe che sono accettate da A o da B , ma non da entrambi. In particolare, se A e B riconoscono lo stesso linguaggio, C non accetterà nulla. Il linguaggio di C è

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Questa espressione è a volte denotata come differenza simmetrica di $L(A)$ e $L(B)$. Qui $\overline{L(A)}$ denota il complemento di $L(A)$. La differenza simmetrica è utile perché $L(C) = \emptyset$ se e solo se $L(A) = L(B)$. Una volta che abbiamo costruito C possiamo usare il Teorema 4.1.1 per verificare se $L(C)$ è vuoto o meno. Se è vuoto, $L(A)$ e $L(B)$ devono essere uguali.

$F =$ "Su input $\langle A, B \rangle$, dove A e B sono DFA:

1. Costruisce il DFA C come descritto.
2. Esegue la TM T del Teorema 4.1.1 su input $\langle C \rangle$.
3. Se T accetta, accetta. Se T rifiuta, rifiuta."

4.1.2 Problemi decidibili relativi a linguaggi context-free

Descriviamo ora algoritmi per determinare se una CFG genera una particolare stringa e per determinare se il linguaggio di una CFG è vuoto. Consideriamo

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ è una CFG che genera la stringa } w \}$$

Teorema 4.1.6 – A_{CFG} è un linguaggio decidibile

A_{CFG} è un linguaggio decidibile.

Osservazione. Per la CFG G e la stringa w vogliamo determinare se G genera w . Un'idea è quella di utilizzare G per passare attraverso tutte le derivazioni per determinare se ne esiste una di w . Quest'idea non funziona, poichè bisognerebbe provare infinite derivazioni. Se G fosse in forma normale di Chomsky, qualsiasi derivazione di w avrebbe $2n - 1$ passi, dove n è la lunghezza di w . In tal caso sarebbe sufficiente controllare solo derivazioni di $2n - 1$ passi per determinare se G genera w . Possiamo convertire G in forma normale di Chomsky.

Dimostrazione. Diamo la TM S per A_{CFG} .

$S =$ "Su input $\langle G, w \rangle$ dove G è una CFG e w è una stringa:

1. Converte G in una grammatica equivalente in forma normale di Chomsky.
2. Lista tutte le derivazioni di $2n - 1$ passi, dove n è la lunghezza di w ; tranne se $n = 0$ in tal caso lista tutte le derivazioni di un passo.
3. Se una di tali derivazioni genera w , accetta; altrimenti, rifiuta."

Come abbiamo fatto per i DFA, possiamo dimostrare che il problema di determinare se una CFG genera almeno una stringa è decidibile. Sia

$$E_{CFG} = \{ \langle G \rangle \mid G \text{ è una CFG ed } L(G) = \emptyset \}$$

Teorema 4.1.7 – E_{CFG} è un linguaggio decidibile

E_{CFG} è un linguaggio decidibile.

Osservazione. Per determinare se il linguaggio di una grammatica è vuoto, abbiamo bisogno di testare se la variabile iniziale può generare una stringa di terminali. L'algoritmo fa questo risolvendo un problema più generale. Determina per ogni variabile se essa è in grado di

generare una stringa di terminali. Quando l'algoritmo ha determinato che una variabile può generare qualche stringa di terminali, l'algoritmo tiene traccia di queste informazioni marcando tale variabile. Dapprima, l'algoritmo marca tutti i simboli terminali della grammatica. Poi, scandisce tutte le regole della grammatica. Se trova una regola che consente a qualche variabile di essere sostituita da una stringa di simboli, che sono già tutti marcati, l'algoritmo sa che anche questa variabile può essere marcata. L'algoritmo continua in questo modo finché non può marcare ulteriori variabili. La TM R implementa questo algoritmo.

Dimostrazione. $R =$ "Su input $\langle G \rangle$, dove G è una CFG:

1. Marca tutti i simboli terminali in G .
2. Ripete fin quando nessuna nuova variabile viene marcata: marca una qualsiasi variabile A tale che G ha una regola $A \rightarrow U_1 U_2 \dots U_k$ ed ogni simbolo $U_1, \dots, U_k$ è già stato marcato.
3. Se la variabile iniziale non è segnata, accetta; altrimenti, rifiuta."

Consideriamo ora il problema di determinare se due grammatiche context-free generano lo stesso linguaggio. Sia

$$EQ_{CFG} = \{ \langle G, H \rangle \mid G \text{ e } H \text{ sono CFG e } L(G) = L(H) \}$$

La classe dei linguaggi context-free non è chiusa per complemento o intersezione. Infatti, EQ_{CFG} non è decidibile.

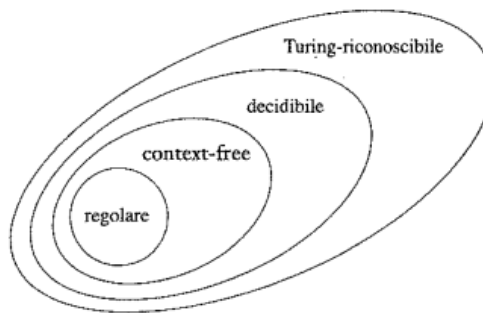


Figura 4.1: Relazioni tra le classi di linguaggi

4.2 Indecidibilità

Ora affronteremo il nostro primo teorema che stabilisce l'indecidibilità di uno specifico linguaggio: il problema di determinare se una macchina di Turing accetta una determinata stringa in input. Chiamiamo tale linguaggio A_{TM} . Sia

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è una TM e } M \text{ accetta } w \}$$

Teorema 4.2.1 – A_{TM} è indecidibile

A_{TM} è indecidibile.

Dimostrazione. Prima di vedere la dimostrazione, osserviamo che A_{TM} è Turing-riconoscibile. Quindi questo teorema mostra che i riconoscitori sono più potenti dei decisori. La seguente macchina di Turing U riconosce A_{TM} .

$U =$ "Su input $\langle M, w \rangle$, dove M è una TM e w è una stringa:

1. Simula M su input w .
2. Se durante la computazione M entra nello stato di accettazione, accetta; se M entra nello stato di rifiuto, rifiuta."

Si noti che questa macchina cicla su input $\langle M, w \rangle$ se M cicla su w , questo è il motivo per cui

non decide A_{TM} . Se l'algoritmo avesse modo di determinare che M non si ferma su w , sarebbe in tal caso in grado di rifiutare w .

La macchina di Turing U è, di per sé, interessante. È un esempio della prima macchina universale di Turing introdotta da Alan Turing nel 1936. Questa macchina è chiamata universale perché è in grado di simulare qualsiasi altra macchina di Turing a partire dalla descrizione di tale macchina.

4.2.1 Metodo della diagonalizzazione

La dimostrazione dell'indcidibilità di A_{TM} utilizza una tecnica chiamata diagonalizzazione.

Teorema 4.2.2 – $\mathcal{R}$ non è numerabile

$\mathcal{R}$ è non numerabile.

Dimostrazione. Per dimostrare che $\mathcal{R}$ è non numerabile, mostriamo che non esiste una biezione tra $\mathcal{N}$ e $\mathcal{R}$. La dimostrazione è per assurdo. Supponiamo quindi che esista una biezione f tra $\mathcal{N}$ ed $\mathcal{R}$. Il nostro compito è quello di dimostrare che f non funziona come dovrebbe. Per essere una biezione, f deve associare tutti gli elementi di $\mathcal{N}$ con tutti gli elementi di $\mathcal{R}$. Tuttavia troveremo un numero X in $\mathcal{R}$ che non è accoppiato con alcun elemento in $\mathcal{N}$, il che rappresenterà la contraddizione cercata. Un metodo per trovare questo numero x è quello di costruirlo. Scegliamo ogni cifra di x in modo da renderlo differente da ogni numero reale accoppiato con un elemento di $\mathcal{N}$. Alla fine saremo sicuri che x risulta diverso da ogni numero reale accoppiato con un elemento di $\mathcal{N}$. Possiamo illustrare questa idea con un esempio. Supponiamo che la biezione f esista. Prendiamo $f(1) = 3.14159\dots$, $f(2) = 55.55555\dots$, $f(3) = \dots$ e così via. Allora f accoppia il numero 1 con $3.14159\dots$, il numero 2 con $55.55555\dots$ e così via.

Costruiamo il numero desiderato, dandone la rappresentazione decimale. Si tratta di un numero compreso tra 0 e 1, quindi tutte le sue cifre significative sono cifre frazionarie che seguono la virgola decimale. Il nostro obiettivo è quello di garantire che $x \neq f(n)$ per ogni n . Per garantire $x \neq f(1)$, scegliamo la prima cifra di x come una qualsiasi cifra diversa dalla prima cifra decimale 1 di $f(1) = 3.14159\dots$. Per assicurarci che $x \neq f(2)$ scegliamo la seconda cifra di x diversa dalla seconda cifra decimale 5 di $f(2) = 55.55555\dots$. Continuando in questo modo lungo la diagonale della tavola per f , otteniamo tutte le cifre di x . Sappiamo che x non è $f(n)$ per ogni n perché differisce da $f(n)$ nell' n -esima cifra frazionaria.

Teorema 4.2.3 – Linguaggi non Turing-riconoscibili

Alcuni linguaggi non sono Turing-riconoscibili.

Dimostrazione. Per dimostrare che l'insieme di tutte le macchine di Turing è numerabile dobbiamo prima osservare che l'insieme di tutte le stringhe Σ^* è numerabile, per ogni alfabeto Σ . Avendo solo un numero finito di stringhe di ogni lunghezza, possiamo formare una lista di Σ^* scrivendo tutte le stringhe di lunghezza 0, lunghezza 1, lunghezza 2, e così via. L'insieme di tutte le macchine di Turing è numerabile perché ogni macchina di Turing M può essere codificata con una stringa $\langle M \rangle$. Se ci limitiamo a tralasciare quelle stringhe che non sono la codifica di una macchina di Turing, possiamo ottenere una lista di tutte le macchine di Turing. Per mostrare che l'insieme di tutti i linguaggi è non numerabile, dobbiamo prima osservare che l'insieme delle sequenze binarie infinite è non-numerabile.

Una sequenza binaria infinita è una sequenza senza fine di 0 e 1. Sia $\mathcal{B}$ l'insieme di tutte le sequenze binarie infinite. Possiamo dimostrare che $\mathcal{B}$ è non numerabile utilizzando una dimostrazione mediante diagonalizzazione. Sia $\mathcal{L}$ l'insieme di tutti i linguaggi sull'alfabeto Σ . Mostriamo che $\mathcal{L}$ è non-numerabile dando una corrispondenza con $\mathcal{B}$, dimostrando così che i due insiemi hanno la stessa cardinalità. Sia $\Sigma^* = \{s_1, s_2, s_3, \dots\}$. Ogni linguaggio $A \in \mathcal{L}$ ha un'unica sequenza in $\mathcal{B}$. Il bit i -esimo della sequenza è un 1 se $s_i \in A$ ed è uno 0 se $s_i \notin A$ questa è chiamata la sequenza caratteristica di A . Per esempio, se A fosse il linguaggio di tutte

le stringhe che iniziano con uno 0 sull'alfabeto $\{0, 1\}$ la sua sequenza caratteristica $\mathcal{X}_A$ sarebbe

$$\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$$

$$A = \{0, 00, 01, 000, 001, \dots\}$$

$$XA = 010110011\dots$$

La funzione $f : \mathcal{L} \rightarrow \mathcal{B}$ dove $f(A)$ è uguale alla sequenza caratteristica di E , è sia iniettiva che suriettiva, quindi è una biezione. Pertanto, essendo $\mathcal{B}$ non numerabile, anche $\mathcal{L}$ è non numerabile.

Abbiamo così dimostrato che l'insieme di tutti i linguaggi non può essere messo in corrispondenza biunivoca con l'insieme di tutte le macchine di Turing. Concludiamo quindi che alcuni linguaggi non sono riconosciuti da una macchina di Turing.

4.2.2 Linguaggio indecidibile

Siamo ora pronti a dimostrare il teorema dell'ind decidibilità del linguaggio

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ è una TM ed } M \text{ accetta } w\}$$

Teorema 4.2.4 – A_{TM} è indecidibile

A_{TM} è indecidibile.

Dimostrazione. Assumiamo che A_{TM} è decidibile, per poi ottenere una contraddizione. Supponiamo che H sia un decisore per A_{TM} . Sull'input $\langle M, w \rangle$, dove M è una TM e w è una stringa, H si ferma ed accetta se M accetta w . Inoltre, H si ferma e rifiuta se M non accetta w . In altri termini, assumiamo che H è una TM, dove

$$H(\langle M, w \rangle) \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ non accetta } w \end{cases}$$

Ora costruiamo una nuova macchina di Turing D avente H come sottoprocedura. Questa nuova TM chiama H per determinare cosa fa M quando l'input di M è la sua stessa descrizione $\langle M \rangle$. Una volta che D ha determinato questa informazione, essa fa il contrario. Cioè, rifiuta se M accetta ed accetta se M non accetta. Diamo ora una descrizione di D .

$D =$ "Su input $\langle M \rangle$, dove M è una TM:

1. Esegue H su input $\langle M, \langle M \rangle \rangle$.
2. Dà in output l'opposto di ciò che H dà in output. Cioè, se H accetta, rifiuta; e se H rifiuta, accetta."

Non lasciatevi confondere dall'idea di attivare una macchina sulla sua stessa descrizione! Ciò è simile all'esecuzione di un programma che chiama se stesso in input, qualcosa che si fa a volte nella pratica. Ricapitolando,

$$D(\langle M \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ non accetta } \langle M \rangle \\ \text{rifiuta} & \text{se } M \text{ accetta } \langle M \rangle \end{cases}$$

Cosa succede quando eseguiamo D con la sua stessa descrizione $\langle D \rangle$ in input? In tal caso, otteniamo

$$D(\langle D \rangle) = \begin{cases} \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \\ \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle \end{cases}$$

Indipendentemente da ciò che D fa, essa è costretta a fare il contrario, il che è ovviamente una contraddizione. Quindi, né la TM D né la TM H possono esistere.

La diagonalizzazione diviene evidente quando si esaminano le tavole del comportamento delle TM H e D . In queste tavole indicizziamo le righe con tutte le TM $M_1, M_2, \dots$ e indicizziamo le colonne con le descrizioni $\langle M_1 \rangle, \langle M_2 \rangle, \dots$ di tali macchine. Le entrate dicono se la macchina

di una determinata riga accetta l'input di una data colonna. L'entrata è accetta se la macchina accetta l'input, ma è vuota, se rifiuta o entra in un loop su quell'input.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	<u>accetta</u>	rifiuta	accetta	rifiuta		accetta	
M_2	accetta	<u>accetta</u>	accetta	accetta	...	accetta	...
M_3	rifiuta	rifiuta	<u>rifiuta</u>	rifiuta		rifiuta	
M_4	accetta	accetta	rifiuta	<u>rifiuta</u>		accetta	
...	...	...	...	...	...	...	...
D	rifiuta	rifiuta	accetta	accetta		?	
...	...	...	...	...	...	...	...

4.2.3 Linguaggio non Turing-riconoscibile

Il teorema seguente mostra che, se un linguaggio ed il suo complemento sono entrambi Turing-riconoscibili, il linguaggio è decidibile. Quindi, per qualsiasi linguaggio non decidibile, o esso non è Turing-riconoscibile oppure il suo complemento non è Turing-riconoscibile. Ricordiamo che il complemento di un linguaggio è il linguaggio costituito da tutte le stringhe che non sono nel linguaggio. Diciamo che un linguaggio è coTuring-riconoscibile se esso è il complemento di un linguaggio Turing-riconoscibile.

Teorema 4.2.5 – Linguaggio decidibile $\iff$ Turing-riconoscibile e coTuring riconoscibile

Un linguaggio è decidibile se e solo se è Turing-riconoscibile e coTuring-riconoscibile.

In altre parole, un linguaggio è decidibile esattamente quando sia esso che il suo complemento sono Turing-riconoscibili.

Dimostrazione. Abbiamo due direzioni da dimostrare. In primo luogo, se A è decidibile, possiamo facilmente vedere che sia A che il suo complemento $\bar{A}$ sono Turing-riconoscibili. Qualsiasi linguaggio decidibile è Turing-riconoscibile, e il complemento di un linguaggio decidibile è a sua volta decidibile. Per la direzione inversa, se entrambi A e $\bar{A}$ sono Turing-riconoscibili, indichiamo con M_1 il riconoscitore per A e con M_2 il riconoscitore per $\bar{A}$. La seguente macchina di Turing M è un decisore per A .

$M =$ "Su input w :

1. Esegue sia M_1 che M_2 su input w in parallelo.
2. Se M_1 accetta, accetta; se M_2 accetta, rifiuta."

L'esecuzione di due macchine in parallelo significa che M ha due nastri, uno per simulare M_1 e l'altro per simulare M_2 . In questo caso M alterna la simulazione di passo di M_1 con un passo di M_2 e continua finché una delle due accetta. Ora dimostriamo che M decide A . Ogni stringa w è in A , oppure in $\bar{A}$. Pertanto una tra M_1 ed M_2 deve accettare w . Poiché M si ferma ogni volta che M_1 accetta oppure M_2 accetta, allora M si ferma sempre quindi è un decisore. Inoltre, accetta tutte le stringhe in A e respinge tutte le stringhe che non sono in A . Quindi M è un decisore per A , e pertanto A è decidibile.

Corollario 4.2.1 – $\overline{A_{TM}}$ non è Turing-riconoscibile

$\overline{A_{TM}}$ non è Turing-riconoscibile.

Dimostrazione. Sappiamo che A_{TM} è Turing-riconoscibile. Se anche $\overline{A_{TM}}$ fosse Turing-riconoscibile, A_{TM} sarebbe decidibile. Il Teorema 4.2 ci dice che A_{TM} non è decidibile, quindi $\overline{A_{TM}}$ non può essere Turing-riconoscibile.

Capitolo 5

Riducibilità

In questo capitolo esamineremo vari altri problemi irrisolvibili. Nel far ciò introdurremo il metodo principale per dimostrare che alcuni problemi sono computazionalmente irrisolvibili. Tale metodo si chiama *riducibilità*.

Una riduzione è un modo di convertire un problema in un altro problema in modo tale che una soluzione al secondo problema può essere usata per risolvere il primo problema. La riducibilità coinvolge sempre due problemi, che noi chiamiamo A e B . Se A si riduce a B , possiamo usare una soluzione per B per risolvere A . Notate che la riducibilità non dice nulla circa la soluzione dei problemi A o B individualmente, ma soltanto qualcosa circa la risolubilità di A quando abbiamo una soluzione per B .

Quando A è riducibile a B , trovare la soluzione di A non può essere più difficile di risolvere B perché una soluzione per B offre una soluzione ad A . In termini di teoria della computabilità, se A è riducibile a B e B è decidibile, allora anche A è decidibile. Equivalentemente, se A è indecidibile e riducibile a B , B è indecidibile.

In breve, il nostro metodo per dimostrare che un problema è indecidibile sarà quello di mostrare che qualche altro problema già noto per essere indecidibile si riduce ad esso.

5.1 Problemi indecidibili dalla teoria dei linguaggi

Abbiamo già stabilito l'indecidibilità di A_{TM} , il problema di determinare se una macchina di Turing accetta un dato input. Consideriamo ora un problema affine, $HALT_{TM}$, il problema di determinare se una macchina di Turing si ferma (accettando o rifiutando) su un dato input. Questo problema è generalmente noto come il problema della fermata. Usiamo l'indecidibilità di A_{TM} per dimostrare l'indecidibilità del problema della fermata riducendo A_{TM} a $HALT_{TM}$. Sia

$$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ è una TM e } M \text{ si ferma su input } w \}$$

Teorema 5.1.1 – $HALT_{TM}$ è indecidibile

$HALT_{TM}$ è indecidibile.

Osservazione. Questa dimostrazione è per assurdo. Assumiamo che $HALT_{TM}$ è decidibile ed utilizziamo questa assunzione per dimostrare che A_{TM} è decidibile, contraddicendo il Teorema 4.2. L'idea chiave è quella di mostrare che A_{TM} è riducibile ad $HALT_{TM}$.

Supponiamo di avere una TM R che decide $HALT_{TM}$. Usiamo quindi R per costruire S , una TM che decide A_{TM} . S potrebbe non essere in grado di determinare se M è entrata in un ciclo, in questo caso la simulazione non terminerà. Questo non va bene perché S è un decisore e non vi è permesso ciclare. Quindi questa idea non funziona. Utilizziamo, invece, l'ipotesi che avete una TM R che decide $HALT_{TM}$. Con R , potete verificare se M si ferma su w . Se R indica che M non si ferma su w , rifiutate perché $\langle M, w \rangle$ non è in A_{TM} . Tuttavia, se R indica che M si ferma su w , potete fare la simulazione senza alcun pericolo di ciclare. Così, se la TM R esistesse, potremmo decidere A_{TM} , ma sappiamo che A_{TM} è indecidibile. In virtù di questa contraddizione possiamo concludere che R non esiste. Pertanto $HALT_{TM}$ è indecidibile.

Dimostrazione. Assumiamo al fine di ottenere una contraddizione che la TM R decide

$HALT_{TM}$. Costruiamo la TM S per decidere A_{TM} , che opera come segue.
 $S =$ "Su input $\langle M, w \rangle$, una codifica di una TM M ed una stringa w

1. Esegue la TM R su input $\langle M, w \rangle$.
2. Se R rifiuta, rifiuta.
3. Se R accetta, simula M su w finché non si ferma.
4. Se M ha accettato, accetta: se M ha rifiutato, rifiuta."

Chiaramente, se R decide $HALT_{TM}$, allora S decide A_{TM} . Poiché A_{TM} indecidibile, allora anche $HALT_{TM}$ deve essere indecidibile.

Presentiamo ora vari altri teoremi e le loro dimostrazioni come ulteriori esempi dell'uso della riducibilità per dimostrare l'indecidibilità. Sia

$$E_{TM} = \{ \langle M \rangle \mid M \text{ è una TM e } L(M) = \emptyset \}$$

Teorema 5.1.2 – E_{TM} è indecidibile

E_{TM} è indecidibile.

Osservazione. Assumiamo che E_{TM} è decidibile e mostriamo che A_{TM} è decidibile una contraddizione. Sia R una TM che decide E_{TM} . Utilizziamo R per costruire la TM S che decide A_{TM} . Come funzionerà S quando riceve in input $\langle M, w \rangle$?

Un'idea è che S esegue R su input $\langle M \rangle$ e vede se R accetta. Se lo fa, sappiamo che $L(M)$ è vuoto e quindi che M non accetta w . Ma, se R rifiuta $\langle M \rangle$, tutto quello che sappiamo è che $L(M)$ non è vuoto e, di conseguenza, che M accetta qualche stringa ma ancora non sappiamo se M accetta la stringa w in particolare. Quindi abbiamo bisogno di utilizzare un'idea diversa. Invece di eseguire R su $\langle M \rangle$, eseguiamo R su una modifica di $\langle M \rangle$. Modifichiamo $\langle M \rangle$ così da garantire che M rifiuta tutte le stringhe tranne w , ma su input w , funziona come al solito. A questo punto, utilizziamo R per determinare se la macchina modificata riconosce il linguaggio vuoto. L'unica stringa che adesso la macchina può accettare è w , per cui il suo linguaggio sarà non vuoto se accetta w . Se R accetta quando riceve in input la descrizione della macchina modificata, sappiamo che la macchina modificata non accetta nulla e che M non accetta w .

Dimostrazione. Descriviamo la macchina modificata già descritta nell'idea di dimostrazione usando la nostra notazione standard. Chiamiamola M_1 .

$M_1 =$ "Su input x :

1. $x \neq w$ rifiuta.
2. $x = w$ esegue M su input w e accetta se M accetta."

Questa macchina ha la stringa w come parte della sua descrizione. Essa verifica se $x = w$ nel modo ovvio, attraverso la scansione dell'input e confrontandolo carattere per carattere con w per determinare se coincidono. Mettendo insieme tutto questo, assumiamo che la TM R decide E_{TM} e costruiamo la TM S che decide A_{TM} come segue.

$S =$ "Su input $\langle M, w \rangle$ una codifica di una TM M e una stringa w :

1. Usa la descrizione di M e w per costruire la TM M_1 descritta sopra.
2. Esegue R su input $\langle M_1 \rangle$.
3. Se R accetta, rifiuta; se R rifiuta, accetta."

Notate che S deve essere effettivamente in grado di calcolare una descrizione di M_1 da una descrizione di M e w . Può farlo, perché ha bisogno solo di aggiungere ad M alcuni stati in più che svolgono il test $x = w$. Se R fosse un decisore per E_{TM} , S sarebbe un decisore per A_{TM} . Un decisore per A_{TM} non può esistere, quindi sappiamo che E_{TM} deve essere indecidibile.

5.2 Riducibilità mediante funzione

In questa sezione formalizziamo il concetto di riducibilità. Informalmente, essere in grado di ridurre il problema A al problema B utilizzando una riduzione mediante funzione significa che esiste una funzione calcolabile che trasforma istanze del problema A in istanze del problema B .

5.2.1 Funzioni calcolabili

Una macchina di Turing calcola una funzione iniziando con l'input della funzione sul nastro e terminando con l'output della funzione sul nastro.

Definizione 5.2.1 – Funzione calcolabile

Una funzione $f : \Sigma \rightarrow \Sigma$ è una funzione calcolabile se esiste una macchina di Turing M che, su qualsiasi input w , si ferma avendo solo $f(w)$ sul nastro.

Esempio 5.2.1 – Funzione calcolabile

Le funzioni calcolabili possono essere trasformazioni di descrizioni di macchine. Per esempio, una funzione calcolabile f prende in input w e restituisce la descrizione di una macchina di Turing $\langle M' \rangle$ se $w = \langle M \rangle$ è la codifica di una macchina di Turing M . La macchina M' è una macchina che riconosce lo stesso linguaggio di M , ma non tenta mai di muovere la testina oltre il limite sinistro del suo nastro. La funzione f realizza questo compito con l'aggiunta di vari stati nella descrizione di M . La funzione restituisce ϵ se w non è una codifica legale di una macchina di Turing.

5.2.2 Definizione formale di riducibilità mediante funzione

Definizione 5.2.2 – Riducibilità mediante funzione

Un linguaggio A si dice riducibile mediante funzione al linguaggio B , e si denota con $A \leq_m B$, se esiste una funzione calcolabile $f : \Sigma \rightarrow \Sigma^*$, dove per ogni w ,

$$w \in A \iff f(w) \in B$$

La funzione f è chiamata riduzione da A a B .

Una riduzione mediante funzione da A a B fornisce un modo per convertire problemi di appartenenza ad A in problemi di appartenenza a B . Per verificare se $w \in A$, usiamo la riduzione per mappare w in $f(w)$ e verifichiamo se $f(w) \in B$ o meno.

Teorema 5.2.1 – Riducibilità mediante funzione decidibile

Se $A \leq_m B$ e B è decidibile, allora A è decidibile.

Dimostrazione. Siano M il decisore per B ed f la riduzione da A a B . Descriviamo un decisore N per A come segue.

$N =$ "Su input w

1. Computa $f(w)$
2. Esegue M su input $f(w)$ e restituisce lo stesso output di M ."

Ovviamente, se $w \in A$ allora $f(w) \in B$ perché f è una riduzione da A a B . Quindi, M accetta $f(w)$ ogni volta che $w \in A$. Quindi, N svolge il compito desiderato.

Corollario 5.2.1

Se $A \leq_m B$ e A è indecidibile, allora B è indecidibile.

Esempio 5.2.2 – Da A_{TM} a $HALT_{TM}$

Possiamo fornire una riduzione mediante funzione da A_{TM} a $HALT_{TM}$ come segue.
Per farlo, dobbiamo fornire una funzione calcolabile f che prende in input $\langle M, w \rangle$ e restituisce in output $\langle M', w' \rangle$ dove

$$\langle M, w \rangle \in A_{TM} \text{ se e solo se } \langle M', w' \rangle \in HALT_{TM}$$

La seguente macchina F calcola una riduzione f .
 $F =$ "Su input $\langle M, w \rangle$:

1. Costruisce la seguente macchina M' .

$M' =$ "Su input x :

- (a) Esegue M su x .
- (b) Se M accetta, accetta.
- (c) Se M rifiuta, cicla."

2. Output $\langle M', w \rangle$ "

Qui sorge un problema minore che riguarda le stringhe in input che non sono della forma corretta. Se la TM F determina che il suo input non è della forma corretta, in accordo a quanto specificato dalla linea dell'input "Su input $\langle M, w \rangle$:" e, quindi, che l'input non è in A_{TM} , la TM dà in output una stringa non in $HALT_{TM}$. In generale, quando descriviamo una macchina di Turing che calcola una riduzione da A a B , assumiamo che gli input che non sono della forma corretta siano mappati in stringhe al di fuori di B .

Esempio 5.2.3 – $E_{TM} \leq_m EQ_{TM}$

Mostriamo che $E_{TM} \leq_m EQ_{TM}$.

Per farlo, dobbiamo fornire una funzione calcolabile f che prende in input $\langle M \rangle$ e restituisce in output $\langle M, M' \rangle$ dove

$$\langle M \rangle \in E_{TM} \text{ se e solo se } \langle M, M' \rangle \in HALT_{TM}$$

La seguente macchina F calcola una riduzione f .
 $F =$ "Su input $\langle M \rangle$:

1. Costruisci una TM M' definita come:

$M' =$ "Data la stringa x in input:

- (a) Rifiuta"

2. Restituisci in output la stringa $\langle M, M' \rangle$ "

Notiamo che:

$$\langle M \rangle \in E_{TM} \iff L(M) \neq \emptyset \iff f(\langle M \rangle) = \langle M, M' \rangle \in EQ_{TM}$$

Di conseguenza, poiché:

$$\langle M \rangle \in E_{TM} \iff f(\langle M \rangle) \in EQ_{TM}$$

ne segue che $E_{TM} \leq_m EQ_{TM}$. Infine, poiché $E_{TM} \notin DEC$, concludiamo che $EQ_{TM} \notin DEC$.

Teorema 5.2.2 – Riducibilità mediante funzione Turing-riconoscibile

Se $A \leq_m B$ e B è Turing-riconoscibile, allora A è Turing-riconoscibile.

La dimostrazione è uguale a quella che abbiamo dato per il Teorema 5.2.2, tranne per il fatto che M ed N sono riconoscitori invece di decisor.

Corollario 5.2.2 – Riducibilità mediante funzione non Turing-riconoscibile

Se $A \leq_m B$ e A non è Turing-riconoscibile, allora B non è Turing-riconoscibile.

In una tipica applicazione di questo corollario, supponiamo che A sia $\overline{A_{TM}}$, il complemento di A_{TM} . Sappiamo che $\overline{A_{TM}}$ non è Turing-riconoscibile. La definizione di riduzione mediante funzione implica che $A \leq_m B$ ha lo stesso significato di $\overline{A} \leq_m \overline{B}$. Per dimostrare che B non è riconoscibile possiamo mostrare che $A_{TM} \leq_m \overline{B}$.

Teorema 5.2.3 – Riducibilità complementare

Dati due linguaggi A e B , si ha che $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.

Dimostrazione. Data la riduzione f tale che $A \leq_m B$, si ha che:

$$w \in \overline{A} \iff w \notin A \iff f(w) \notin B \iff f(w) \in \overline{B}$$

Teorema 5.2.4 – EQ_{TM} non è né Turing-riconoscibile né co-Turing-riconoscibile

EQ_{TM} non è né Turing-riconoscibile né co-Turing-riconoscibile.

Dimostrazione. Come primo passo, dimostriamo che EQ_{TM} non è Turing-riconoscibile. Lo facciamo dimostrando che A_{TM} è riducibile a EQ_{TM} . La funzione di riduzione f opera come segue.

$F =$ "Su input $\langle M, w \rangle$, dove M è una TM e w è una stringa:

1. Costruisce le seguenti due macchine, M_1 e M_2 .

M_1 "Su ogni input:

(a) Rifiuta."

M_2 "Su ogni input:

(a) Esegue M su w . Se accetta, accetta."

2. Restituisce $\langle M_1, M_2 \rangle$."

Qui, M_1 non accetta alcuna stringa. Se M accetta w , M_2 accetta qualsiasi stringa, quindi le due macchine non sono equivalenti. Viceversa, se M non accetta w , M_2 non accetta alcuna stringa, ed esse sono equivalenti. Perciò f riduce A_{TM} a EQ_{TM} come desiderato.

Per dimostrare che $\overline{EQ_{TM}}$ non è Turing-riconoscibile diamo una riduzione da A_{TM} al complemento di EQ_{TM} cioè, $\overline{EQ_{TM}}$. In questo modo dimostriamo che $A_{TM} \leq_m \overline{EQ_{TM}}$. La seguente TM G calcola la funzione di riduzione g .

$G =$ "Su input $\langle M, w \rangle$, dove M è una TM e w è una stringa:

1. Costruisce le seguenti due macchine, M_1 ed M_2 .

$M_1 =$ "Su ogni input:

(a) Accetta."

$M_2 =$ "Su ogni input:

(a) Esegue M su w .

(b) Se accetta, accetta."

2. Restituisce $\langle M_1, M_2 \rangle$."

La sola differenza tra f e g si trova nella macchina M_1 . In f , la macchina M_1 rifiuta sempre, mentre in g accetta sempre. Sia in f che in g , M accetta w sse M_2 accetta ogni input. In g , M accetta w sse M_1 ed M_2 sono equivalenti. Questo è il motivo per cui g è una riduzione da A_{TM} a $\overline{EQ_{TM}}$.

5.3 Esercizi svolti

Esercizio 5.3.1

Dato il seguente linguaggio $L = \{\langle M \rangle \mid M \text{ TM}, L(M) = \{w \in \Sigma^* \mid |w| \text{ è dispari}\}\}$ dimostrare che è indecidibile, ossia che $L \notin DEC$.

Dimostrazione. Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F = \text{"Data la stringa } \langle M, w \rangle \text{ in input:}$

1. Costruisci una TM M' definita come:

$M' = \text{"Data la stringa } x \text{ in input:}$

- (a) Se $|x|$ è pari, rifiuta. Altrimenti, esegui il programma di M su input w
- (b) Se l'esecuzione accetta, allora M' accetta, altrimenti rifiuta"

2. Restituisci in output la stringa $\langle M' \rangle$ "

Supponiamo che $w \notin L(M)$. In tal caso, M' rifiuterà qualsiasi stringa, implicando che $L(M') = \emptyset$ e dunque che $\langle M' \rangle \notin L$.

Supponiamo ora che $w \in L(M)$. In tal caso, abbiamo che: se $|x|$ è pari, allora $x \notin L(M')$; se $|x|$ è dispari, allora $x \in L(M')$ poiché $M(w)$ accetta di conseguenza, otteniamo che:

$$x \in L(M') \iff |x| \text{ è dispari}$$

implicando che $\langle M' \rangle \in L$. Di conseguenza, concludiamo che $w \in L(M) \iff L(M') \in L$.
 A questo punto, notiamo che:

$$\langle M, w \rangle \in A_{TM} \iff w \in L(M) \iff f(\langle M, w \rangle) = \langle M' \rangle \in L$$

implicando quindi che $A_{TM} \leq_m L$. Infine, poiché $A_{TM} \notin DEC$, concludiamo che $L \notin DEC$.

Esercizio 5.3.2

Dato il seguente linguaggio $L = \{\langle M \rangle \mid M \text{ TM}, L(M) = \{0y \mid y \in \Sigma^*\}\}$ dimostrare che è indecidibile, ossia che $L \notin DEC$.

Dimostrazione. Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F = \text{"Data la stringa } \langle M, w \rangle \text{ in input:}$

1. Costruisci una TM M' definita come:

$M' = \text{"Data la stringa } x \text{ in input:}$

- (a) Se $x \notin \{0y \mid y \in \Sigma^*\}$, rifiuta. Altrimenti, esegui il programma di M su input w .
- (b) Se l'esecuzione accetta, allora M' accetta, altrimenti rifiuta"

2. Restituisci in output la stringa $\langle M' \rangle$ "

Supponiamo che $w \notin L(M)$. In tal caso, M' rifiuterà qualsiasi stringa, implicando che $L(M') = \emptyset$ e dunque che $\langle M' \rangle \notin L$.

Supponiamo ora che $w \in L(M)$. In tal caso, abbiamo che: se $x \notin \{0w \mid w \in \Sigma^*\}$, allora $x \notin L(M')$; se $x \in \{0w \mid w \in \Sigma^*\}$, allora $x \in L(M')$ poiché $w \in L(M)$ di conseguenza, otteniamo che:

$$x \in L(M') \iff x \in \{0w \mid w \in \Sigma^*\}$$

implicando che $\langle M' \rangle \in L$. Di conseguenza, concludiamo che $w \in L(M) \iff L(M') \in L$.
 A questo punto, notiamo che:

$$\langle M, w \rangle \in A_{TM} \iff w \in L(M) \iff f(\langle M, w \rangle) = \langle M' \rangle \in L$$

implicando quindi che $A_{TM} \leq_m L$. Infine, poiché $A_{TM} \notin DEC$, concludiamo che $L \notin DEC$.

Esercizio 5.3.3

Dato il seguente linguaggio $L = \{\langle T, T' \rangle \mid T, T' \text{ TM}, L(T) \cup L(T') = \Sigma^*\}$ dimostrare che è indecidibile, ossia che $L \notin DEC$.

Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

$F = \text{"Data la stringa } \langle M, w \rangle \text{ in input:}$

1. Costruisci una TM T definita come:

$T = \text{"Data la stringa } x \text{ in input:}$

(a) Rifiuta"

2. Costruisci una TM T' definita come:

$T' = \text{"Data la stringa } x \text{ in input:}$

(a) Esegui il programma di M con input w .

(b) Se l'esecuzione accetta, T' accetta, altrimenti rifiuta"

3. Restituisci in output la stringa $\langle T, T' \rangle$ "

Notiamo che:

$$\langle M, w \rangle \in A_{TM} \iff w \in L(M) \implies L(T) = \emptyset, L(T') = \Sigma^* \implies L(T) \cup L(T') = \Sigma^* \iff f(\langle M, w \rangle) = \langle T, T' \rangle \in L$$

e inoltre che:

$$\langle M, w \rangle \notin A_{TM} \iff w \notin L(M) \implies L(T) = \emptyset, L(T') = \emptyset \implies L(T) \cup L(T') = \emptyset \implies f(\langle M, w \rangle) = \langle T, T' \rangle \notin L$$

implicando quindi che $A_{TM} \leq_m L$. Infine, poiché $A_{TM} \notin DEC$, concludiamo che $L \notin DEC$.

Parte III

Teoria della complessità

Capitolo 6

Complessità di tempo

Anche quando un problema è decidibile, e quindi in linea di principio computazionalmente risolvibile, può non essere risolvibile in pratica se la soluzione richiede una quantità eccessiva di tempo o di memoria. In questa parte introduciamo la teoria della complessità computazionale, uno studio del tempo, della memoria o di altre risorse necessarie per risolvere problemi computazionali.

6.1 Misure di complessità

Per semplicità si calcola il tempo di esecuzione di un algoritmo semplicemente in funzione della lunghezza della stringa che rappresenta l'input e non si considerano eventuali altri parametri. Nell'analisi del caso peggiore, che noi considereremo, si valuta il tempo di esecuzione massimo tra tutti gli input di una determinata lunghezza. Nell'analisi del caso medio, si considera la media dei tempi di esecuzione su tutti gli input di una determinata lunghezza.

Definizione 6.1.1 – Complessità temporale di una TM

Sia M una macchina di Turing deterministica che si ferma su tutti gli input. Il tempo di esecuzione o la complessità di tempo di M è la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ dove $f(n)$ è il numero massimo di passi che M utilizza su un qualsiasi input di lunghezza n .

Ad esempio sia M il decisore definito come:

$M = \text{"Data la stringa } w \text{ in input:}$

1. Muovi la testina a destra finché non viene letto il simbolo $\sqcup$
2. Accetta"

Data in input una stringa w tale che $|w| = n$, la TM M impiega n passi. Di conseguenza, il suo tempo computazionale è $f(n) = n$.

6.1.1 Notazione O-grande ed o-piccola

Poiché il tempo esatto di esecuzione di un algoritmo è spesso un'espressione complessa, solitamente ci limitiamo ad ottenerne una stima. Il metodo di stima, detto analisi asintotica, permette di valutare il tempo di esecuzione dell'algoritmo quando viene eseguito su grandi input. Lo facciamo prendendo in considerazione solo il termine di ordine maggiore dell'espressione del tempo di esecuzione dell'algoritmo, trascurando sia il coefficiente di tale termine, che tutti i termini di ordine inferiore, perché il termine di ordine più alto domina gli altri termini quando l'input è grande.

Definizione 6.1.2 – O-grande

Siano f e g funzioni $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Si dice che $f(n) = O(g(n))$ se esistono interi positivi c e n_0 tali che per ogni $n \geq n_0$,

$$f(n) \leq cg(n)$$

Quando $f(n) = O(g(n))$ diciamo che $g(n)$ è un limite superiore asintotico per $f(n)$.

Definizione 6.1.3 – o-piccolo

Siano f e g funzioni $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Diciamo che $f(n) = o(g(n))$ se

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

In altri termini, $f(n) = o(g(n))$ significa che per ogni numero reale $c > 0$ esiste un numero n_0 , tale che $f(n) < cg(n)$ per ogni $n \geq n_0$.

Esempio 6.1.1 – Rapporto di tempo tra TM multinastro e TM

Sia $t(n)$ una funzione tale che $t(n) \geq n$. Per ogni TM multinastro M con tempo $t(n)$ esiste una TM M' tale che $L(M) = L(M')$ avente tempo $O(t^2(n))$.

Consideriamo la TM S in grado di simulare una TM multinastro. Analizziamo quindi la complessità temporale di S :

1. Il numero di nastri k è indipendente dall'input.
2. Per preparare il nastro sono necessari $kO(n) = O(n)$ passi.
3. Per ogni passo simulato della TM multinastro, S richiede $kO(t(n)) = O(t(n))$ passi.
4. Il numero di passi della TM multinastro è $t(n)$.

Di conseguenza, la complessità temporale di S risulta essere:

$$t'(n) = O(n) + t(n)O(t(n)) = O(n) + O(t^2(n)) = O(t^2(n))$$

Definizione 6.1.4 – Complessità temporale di una NTM

Sia N un decisore non-deterministico. Definiamo come complessità temporale di N la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ tale che $f(n)$ sia il massimo numero di passi necessari ad ogni ramo dell'esecuzione di D per processare una stringa di lunghezza n .

Esempio 6.1.2 – Rapporto di tempo tra NTM e TM

Sia $t(n)$ una funzione tale che $t(n) \geq n$. Per ogni NTM N con tempo $t(n)$ esiste una TM M' tale che $L(M) = L(M')$ avente tempo $2^{O(t(n))}$.

Consideriamo la TM M in grado di simulare una NTM. Analizziamo quindi la complessità temporale di M :

1. Il numero massimo b di figli di ogni nodo è indipendente dall'input.
2. Di conseguenza, il numero di foglie dell'albero computazionale risulta essere $k \leq b^{t(n)}$, implicando che il numero massimo di nodi sia $m \leq 2b^{t(n)}$, dunque $m = O(b^{t(n)})$.
3. M simula ogni nodo dell'albero computazionale ripartendo sempre dalla radice, dunque esegue $O(t(n)b^{t(n)})$ passi.

A questo punto, notiamo che:

$$t(n)b^{t(n)} = 2\log_2(t(n)b^{t(n)}) = 2^{\log_2(t(n)) + t(n)\log_2 b} = 2^{O(t(n))}$$

Di conseguenza, la complessità temporale di M risulta essere:

$$t'(n) = O(t(n)b^{t(n)}) = O(2^{O(t(n))}) = 2^{O(t(n))}$$

6.2 Classe P

Definizione 6.2.1 – Classe DTIME

Data la funzione $t : \mathbb{N} \rightarrow \mathbb{R}^+$, definiamo come classe dei linguaggi decibili in tempo $O(t(n))$ il seguente insieme:

$$DTIME(t(n)) = \{L \in DEC \mid L \text{ decibibile da una TM in tempo } O(t(n))\}$$

Definizione 6.2.2 – Classe P

P è la classe di linguaggi che sono decibili in tempo polinomiale su una macchina di Turing deterministica a singolo nastro. In altre parole,

$$P = \bigcup_k TIME(n^k)$$

La classe P ha un ruolo centrale nella nostra teoria ed è importante perché è una classe matematicamente robusta rispetto alle varianti di TM che abbiamo studiato.

Definizione 6.2.3 – Classe EXP

Definiamo la classe dei linguaggi decibili in tempo esponenziale come:

$$EXP = \bigcup_{k \in \mathbb{N}} DTIME(2^{n^k})$$

Ovviamente $P \subseteq EXP$ e $P \neq EXP$.

Esempio 6.2.1 – Problema dei cammini

Un grafo orientato G contiene nodi s e t . Il problema $PATH$ consiste nel determinare, se esiste, un cammino diretto s a t .

Sia $PATH = \{(G, s, t) \mid G \text{ è un grafo orientato che contiene un cammino diretto da } s \text{ a } t\}$.

Dobbiamo dimostrare se $PATH$ è polinomialmente decibibile, ossia $PATH \in P$.

Osservazione. Dimostriamo questo teorema presentando un algoritmo di tempo polinomiale che decide $PATH$. Un potenziale cammino è una sequenza di nodi in G avente lunghezza al più m , dove m è il numero di nodi in G . (Se esiste un qualche cammino diretto da s a t , ne deve esistere necessariamente almeno uno avente lunghezza al massimo m , perché non è mai necessario ripetere un nodo.)

Per ottenere un algoritmo polinomiale per $PATH$, si può utilizzare un metodo visita dei grafi come la visita in ampiezza. In questo caso, contrassegniamo successivamente tutti nodi in G che sono raggiungibili da s mediante cammini diretti di lunghezza 1, poi 2, poi 3, fino ad m .

Dimostrazione. Un algoritmo in tempo polinomiale M per $PATH$ funziona come segue.

$M =$ "Su input (G, s, t) , dove G è un grafo orientato con nodi s e t :

1. Marca il nodo s .
2. Ripete il seguente passo fino a quando nessun nuovo nodo viene marcato:
 - (a) Scansiona tutti gli archi di G . Se trova un arco (a, b) che va da un nodo a marcato ad un nodo b non marcato, marca il nodo b .
3. Se t è marcato, accetta. Altrimenti, rifiuta."

Ora analizziamo questo algoritmo per dimostrare che lavora in tempo polinomiale. Ovviamente, le fasi 1 e 3 sono eseguite una sola volta. La fase 2 è eseguita al più m volte perché ogni volta, tranne l'ultima, marca un nodo aggiuntivo in G . Quindi il numero totale di fasi utilizzate è al massimo $1 + 1 + m$, che da un polinomio nella dimensione di G .

6.3 Classe NP

Definizione 6.3.1 – Classe NTIME

Data la funzione $t : \mathbb{N} \rightarrow \mathbb{R}^+$, definiamo come classe dei linguaggi decidibili non-deterministicamente in tempo $O(t(n))$ il seguente insieme:

$$NTIME(t(n)) = \{L \in DEC \mid L \text{ decidibile da una NTM in tempo } O(t(n))\}$$

Definizione 6.3.2 – Verificatore

Un verificatore per un linguaggio A è un algoritmo V , dove

$$A = \{w \mid V \text{ accetta } \langle w, c \rangle \text{ per qualche stringa } c\}$$

dove la stringa c viene detta certificato di appartenenza. Misuriamo il tempo di un verificatore solo in termini della lunghezza di w , quindi un verificatore in tempo polinomiale viene eseguito in tempo polinomiale nella lunghezza di w . Un linguaggio A è polinomialmente verificabile se ammette un verificatore in tempo polinomiale.

Definizione 6.3.3 – Classi NP e NEXP

Definiamo la classe dei linguaggi decidibili non-deterministicamente in tempo polinomiale come:

$$NP = \bigcup_{k=1}^{\infty} NTIME(n^k)$$

oppure si può definire NP come la classe dei linguaggi che ammettono un verificatore in tempo polinomiale.

Analogamente, definiamo la classe dei linguaggi decidibili non-deterministicamente in tempo esponenziale come:

$$NEXP = \bigcup_{k=1}^{\infty} NTIME(2^{n^k})$$

Corollario 6.3.1 – Gerarchia temporale dei linguaggi

Date le classi P , NP e EXP , si ha che:

$$P \subseteq NP \subseteq EXP \subseteq NEXP$$

Dimostrazione. Per definizione stessa, una TM risulta essere anche una NTM. Di conseguenza, ne segue automaticamente che $P \subseteq NP$ e che $EXP \subseteq NEXP$. Inoltre, per il corollario precedente, abbiamo che:

$$L \in NP \implies \exists k \in \mathbb{N} \mid L \in NTIME(n^k) \implies L \in DTIME(2^{O(n^k)}) \implies L \in EXP$$

Definizione 6.3.4 – CLIQUE

Definiamo come *CLIQUE* il seguente linguaggio:

$$CLIQUE = \{\langle G, k \rangle \mid G \text{ grafo non diretto con una } k\text{-clique}\}$$

dove una k -clique è un sottoinsieme di nodi dove ognuno di quest'ultimi è adiacente a tutti gli altri nodi del sottoinsieme.

Ovvero una clique, in un grafo non orientato, è un sottografo in cui ogni due nodi sono collegati da un arco. Una k -clique è una clique che contiene k nodi.

Esempio 6.3.1 – Problema della tricolorazione dei grafi

Definiamo il linguaggio del problema della tricolorazione dei grafi come:

$$3COL = \{\langle G \rangle \mid G = (V_G, E_G) \text{ grafo 3-colorabile}\}$$

dove un grafo è detto 3-colorabile se ad ogni suo nodo è possibile assegnare un colore diverso dai suoi nodi adiacenti utilizzando al massimo tre colori.

Vogliamo dimostrare se $3COL$ è polinomialmente verificabile ovvero $3COL \in NP$.

Dimostrazione. Sia V la TM definita come:

$V =$ "Data la stringa $\langle \langle G \rangle, c \rangle$ in input, dove $G = (V_G, E_G)$ è un grafo:

1. Interpreta c come $c = \langle c_1, \dots, c_m \rangle$, dove $m = |V_G|$ e $\forall k \in [1, m], c_k \in \{R, G, B\}$.
2. Ripeti lo step successivo per ogni arco $(v_i, v_j) \in E_G$:
 - (a) Se $c_i = c_j$, allora rifiuta.
3. Accetta"

Per costruzione stessa di V , si ha che:

$$\begin{aligned} \langle G \rangle \in 3COL &\iff \text{Esiste una colorazione valida } c_1, \dots, c_n \\ &\iff \exists c = \langle c_1, \dots, c_n \rangle \in \Sigma^*, \langle \langle G \rangle, c \rangle \in V \end{aligned}$$

dunque V risulta essere un verificatore di $3COL$. Analizziamo quindi il costo di V :

1. La prima istruzione è eseguibile in $O(|w|) = O(n)$.
2. La seconda istruzione è eseguibile in $O(n^k)$ per qualche $k \in \mathbb{N}$.
3. Il ciclo viene eseguito per $O(|E_G|) = O(m^2) = O(n^2)$ iterazioni, dove ognuna di quest'ultime richiede $O(1)$.

Di conseguenza il costo computazione di V risulta essere:

$$t(n) = O(n) + O(n^k) + O(n^2)O(1) = O(n^k)$$

implicando che V sia un verificatore polinomiale e quindi che $3COL \in NP$.

Teorema 6.3.1 – Classe NP (definizione alternativa)

Data la classe NP , si ha che $NP = \{L \in DEC \mid \exists V \text{ verificatore polinomiale per } L\}$.

In altre parole, possiamo definire NP anche come la classe dei linguaggi verificabili in tempo polinomiale.

Dimostrazione. Dato $L \in NP$, sia N la NTM tale che $L = L(N)$. Durante la sua computazione, N effettuerà una serie di scelte, creando così il suo albero di computazione. Sia V la TM definita come:

$V =$ "Data la stringa $\langle w, c \rangle$ in input:

1. Interpreta c come una serie di scelte da effettuare
2. Simula N su input w eseguendo le scelte dettate da c
3. Se la simulazione accetta, V accetta. Altrimenti, rifiuta"

Per costruzione stessa di V , si ha che:

$$w \in L = L(N) \iff \text{Esiste un ramo di } N \text{ che accetta } w \iff \exists c \in \Sigma^*, \langle w, c \rangle \in L(V)$$

dunque V risulta essere un verificatore di $L = L(N)$. Inoltre, poiché $L = L(N) \in NP$, la lunghezza massima dei rami dell'albero di computazione risulta essere $O(n^k)$. Di conseguen-

za, ogni certificato può avere al massimo $O(n^k)$ scelte, implicando che V sia un verificatore polinomiale.

Dato un linguaggio L , supponiamo che esista un verificatore V che verifica L in tempo polinomiale, implicando che:

$$\exists k \in \mathbb{N} | V \text{ verifica } L \text{ in } O(n^k)$$

Sia N la NTM definita come:

$N = \text{"Data la stringa } \langle w \rangle \text{ in input:}$

1. Scegli non deterministicamente una stringa c di lunghezza $O(n^k)$.
2. Simula V su input $\langle w, c \rangle$.
3. Se la simulazione accetta, B accetta. Altrimenti, rifiuta"

Essendo N una NTM, l'esecuzione del primo passo genererà un albero computazionale in cui ogni ramo analizza una sola tra tutte le possibili stringhe di lunghezza $O(n^k)$. In particolare, almeno uno di questi rami analizzerà un certificato valido. Di conseguenza, si ha che:

$$w \in L \iff \exists c \in \Sigma^*, \langle w, c \rangle \in V \iff \text{Esiste un ramo di } N \text{ che accetta } w \iff x \in L$$

implicando che $L \in L(N) \in NP$.

6.4 NP-completezza

Definizione 6.4.1 – Funzione calcolabile in tempo polinomiale

Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è una funzione calcolabile in tempo polinomiale se esiste una macchina di Turing di tempo polinomiale M che si arresta con $f(w)$ soltanto sul suo nastro, quando ha iniziato con un qualsiasi input u .

Definizione 6.4.2 – Riducibile in tempo polinomiale

Il linguaggio A è riducibile mediante funzione in tempo polinomiale, o semplicemente riducibile in tempo polinomiale, al linguaggio B , denotato con $A \leq_P B$ se esiste una funzione calcolabile in tempo polinomiale $f : \Sigma^* \rightarrow \Sigma^*$ dove, per ogni w ,

$$w \in A \iff f(w) \in B$$

La funzione f è chiamata riduzione di tempo polinomiale di A a B .

Esempio 6.4.1 – Problema della soddisfacibilità delle 3CNF

Definiamo il linguaggio del problema della soddisfacibilità delle 3CNF come:

$$3SAT = \{ \langle \phi \rangle | \phi \text{ è una 3CNF soddisfacibile} \}$$

dove una 3CNF è una formula logica formata da un *and* logico tra n clausole, ciascuna formata da un *or* logico tra 3 letterali, ossia una variabile negata o non. La seguente formula è una 3CNF:

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_5 \vee x_6) \wedge (x_3 \vee \bar{x}_6 \vee x_4) \wedge (x_4 \vee x_5 \vee x_6)$$

Inoltre, tale formula è soddisfacibile dal seguente assegnamento:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 1, x_5 = 1, x_6 = 0$$

Teorema 6.4.1 – 3SAT riducibile in tempo polinomiale a CLIQUE

3SAT è riducibile in tempo polinomiale a CLIQUE.

Osservazione. La riduzione di tempo polinomiale f che mostriamo di 3SAT a CLIQUE converte formule in grafi. Nei grafi costruiti, clique di una dimensione specificata corrispondono ad assegnamenti soddisfacenti per la formula. Le strutture all'interno del grafo sono progettate per simulare il comportamento delle variabili e delle clausole.

Dimostrazione. Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F = \text{"Data la stringa } \langle \phi \rangle \text{ in input:}$

1. Costruisci un grafo G definito come:
 - (a) Conta il numero k di clausole *or* in ϕ .
 - (b) Per ogni clausola, crea un nodo x_i per ogni letterale della clausola, creando dei doppioni se il letterale compare più volte.
 - (c) Organizza i nodi di G in k -triple (una per clausola *or*), dove tre nodi appartengono alla stessa tripla se e solo se i loro letterali compaiono all'interno di una stessa clausola *or*.
 - (d) Crea un arco tra ogni coppia di nodi fatta eccezione di nodi appartenenti alla stessa tripla e nodi rappresentanti letterali opposti tra loro (es: x_i e $\bar{x}_i$).
2. Restituisci in output la stringa $\langle G, k \rangle$

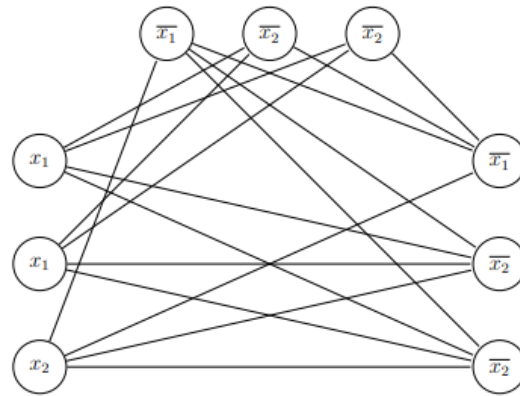


Figura 6.1: Grafo generato dalla funzione f della dimostrazione a partire dalla formula $\phi = (x_1 \vee x_1 \vee x_2) \wedge (x_1 \vee x_2 \vee x_2) \wedge (x_1 \vee x_2 \vee x_2)$

Supponiamo che $\langle \phi \rangle \in 3SAT$. Poiché ϕ è una 3CNF, affinché essa sia soddisfacibile ne segue che ogni clausola sia soddisfacibile, dunque che almeno uno dei letterali di ognuna di tali clausole sia vero. Sia quindi $C = \{x_1, \dots, x_k\}$ l'insieme dei nodi tali che $\forall i \in [1, k], x_i$ è il nodo corrispondente al letterale vero della i -esima tripla. Poiché tali nodi appartengono tutti a triple diverse e poiché $\nexists x_i, x_j \in C$ tali che $x_i = \bar{x}_j$ in quanto non possono essere entrambi veri, ne segue che essi siano tutti due a due adiacenti, implicando che C sia una k -clique. Di conseguenza, abbiamo che:

$$\langle \phi \rangle \in 3SAT \implies f(\langle \phi \rangle) = \langle G, k \rangle \in CLIQUE$$

Supponiamo ora che $f(\langle \phi \rangle) = \langle G, k \rangle \in CLIQUE$, implicando che esista una k -clique al suo interno. Dunque, per costruzione di G , non esistono nodi all'interno della clique i cui letterali rappresentati appartengono alla stessa tripla. A questo punto, poiché tali letterali non interferiscono tra loro trovandosi in triple diverse, esiste un assegnamento che soddisfi ogni clausola. In particolare, tale assegnamento risulta sempre possibile in quanto all'interno della clique non possano appartenere sia x_i che $\bar{x}_i$. Di conseguenza, abbiamo che:

$$f(\langle \phi \rangle) = \langle G, k \rangle \in CLIQUE \implies \langle \phi \rangle \in 3SAT$$

implicando quindi che:

$$\langle \phi \rangle \in 3SAT \iff f(\langle \phi \rangle) = \langle G, k \rangle \in CLIQUE$$

Inoltre, poiché F svolge solo operazioni eseguibili in tempo polinomiale, concludiamo che $3SAT \leq_m^P CLIQUE$.

6.4.1 Riducibilità in tempo polinomiale

Definizione 6.4.3 – Riducibilità in tempo polinomiale

Dati due linguaggi A e B , diciamo che A è riducibile in tempo polinomiale a B tramite mappatura, indicato come $A \leq_m^P B$, se esiste una funzione calcolabile $f : \Sigma^* \rightarrow \Sigma^*$, detta riduzione in tempo polinomiale da A a B , tale che:

$$w \in A \iff f(w) \in B$$

e f è calcolabile in tempo polinomiale.

Teorema 6.4.2 – Decidibilità polinomiale tramite riduzione

Dati due linguaggi A e B tali che $A \leq_m^P B$, si ha che:

$$B \in P \implies A \in P$$

Dimostrazione. Dato $B \in P$, sia DB il decisore tale che $L(DB) = B$. Sia DA la TM definita come:

$DA = \text{"Data la stringa } w \text{ in input:}$

1. Calcola $f(w)$
2. Esegui il programma di DB con input $f(w)$.
3. Se l'esecuzione accetta, D accetta. Altrimenti, D rifiuta"

Per costruzione stessa di DA , si ha che:

$$w \in L(DA) \iff f(w) \in L(DB) = B \iff w \in A$$

implicando che $L(DA) = A$. Inoltre, poiché DB è un decisore e poiché f è calcolabile, ne segue che anche DA sia un decisore e quindi che $A = L(DA) \in DEC$.

Teorema 6.4.3 – Riducibilità polinomiale transitiva

Dati tre linguaggi $A, B, C \subseteq \Sigma^*$, si ha che:

$$A \leq_m^P B, B \leq_m^P C \implies A \leq_m^P C$$

In altre parole, la relazione $\leq_m^P$ è transitiva.

Dimostrazione. Siano $f, g : \Sigma^* \rightarrow \Sigma^*$ le due funzioni calcolabili polinomialmente per cui rispettivamente si ha che $A \leq_m^P B$ e $B \leq_m^P C$. Sia quindi H la TM definita come:
 $H = \text{"Data la stringa } w \text{ in input:}$

1. Calcola $f(w)$. Sia k l'output di tale calcolo.
2. Calcola $g(k)$ e restituisci l'output"

Risulta evidente che H sia la TM che calcola la funzione $g \circ f$ e che:

$$w \in A \iff f(w) \in B \iff g(f(w)) = (g \circ f)(w) \in C$$

Inoltre, poiché f e g sono calcolabili polinomialmente, ne segue automaticamente che H svolga solo operazioni in tempo polinomiale, concludendo che $A \leq_m^P C$.

6.4.2 Classe NP-Complete

Definizione 6.4.4 – Classe NP-Hard

Definiamo la classe dei linguaggi NP-Difficili come:

$$NP - Hard = \{B \subset \Sigma^* \mid \forall A \in NPA \leq_m^P B\}$$

dove $B \neq \emptyset, \Sigma^*$.

Nota: non è detto che un linguaggio NP-Hard sia decidibile o riconoscibile.

Definizione 6.4.5 – Classe NP-Complete

Definiamo la classe dei linguaggi NP-Completi come:

$$NP - Complete = NP \cap NP - Hard$$

Teorema 6.4.4 – Completezza transitiva

Dati due linguaggi B e C tali che $B \leq_m^P C$ e $C \in NP$, si ha che:

$$B \in NP - Complete \implies C \in NP - Complete$$

Dimostrazione. Poiché la relazione $\leq_m^P$ è transitiva e poiché $B \in NP - Complete$, ne segue automaticamente che:

$$\forall A \in NPA \leq_m^P B \leq_m^P C$$

implicando dunque che $C \in NP \cap NP - Hard = NP - Complete$.

Teorema 6.4.5 – Linguaggio NP-completo in P

Dato un linguaggio $B \in NP - Complete$, si ha che:

$$B \in P \iff P = NP$$

Dimostrazione. Iniziamo con la prima implicazione. Per definizione stessa di $NP - Complete$, si ha che $\forall A \in NPA \leq_m^P B$. Di conseguenza, poiché $B \in P$, ne segue automaticamente che:

$$\forall A \in NPB \in P \implies A \in P$$

implicando dunque che $NP \subseteq P$. Inoltre, poiché sappiamo già che $P \subseteq NP$, concludiamo che $P = NP$.

Passiamo ora alla seconda implicazione. Per definizione stessa di $NP - Complete$, si ha che $B \in NP$. Di conseguenza, se $P = NP$, ne segue automaticamente che $B \in P$.

Esercizio 6.4.1 – Padding argument

Dimostrare che se $P = NP$ allora $EXP = NEXP$.

Dimostrazione. Sappiamo già che $EXP \subseteq NEXP$, quindi è sufficiente dimostrare che in tal caso si ha che $NEXP \subseteq EXP$. Dato $L \in NEXP$, sappiamo che esiste una NTM N che decide L in tempo esponenziale. Consideriamo quindi il seguente linguaggio:

$$L' = \{x1^{2^{|x|^c}} \mid x \in L\}$$

dove c è una costante arbitraria. Sia N' la NTM definita come:

N' = "Data la stringa y in input:

1. Genera una stringa casuale x .
2. Verifica che $y = x1^{2^{|x|}c}$. Se falso, rifiuta.
3. Simula N su input x . Se la simulazione accetta, accetta. Altrimenti, rifiuta."

Dove per definizione stessa di N' risulta evidente che $L(N') = L'$.

A questo punto, è opportuno notare che la dimensione dell'input di N' sia di per sé esponenziale. In particolare, per ogni stringa $x \in L$ tale che $|x| = n$, l'input di N' avrà lunghezza $m = n + 2^{n^c}$. Per tanto, poiché la simulazione di N su x richiede tempo esponenziale 2^{n^d} , dove d è un'altra costante, tale simulazione richiede $O(m^k)$ passi, dove $k = \max(c, d)$, dunque è polinomiale rispetto alla dimensione dell'input. Ciò conclude che N' sia un decisore non-deterministico per L' .

Poiché in ipotesi si ha che $P = NP$, deve esistere un decisore di tempo polinomiale M' tale che $L(M') = L'$. Definiamo quindi la seguente TM:

M = "Data la stringa x in input:

1. Simula M' su input $x1^{2^{|x|}c}$. Se la simulazione accetta, accetta. Altrimenti, rifiuta.

Dove per definizione stessa di N' risulta evidente che $L(M) = L$. Inoltre, dato $|x| = n$, la simulazione di M' richiede tempo esponenziale rispetto alla dimensione dell'input, dunque concludiamo che $L \in EXP$.

6.4.3 Teorema di Cook-Levin

Teorema 6.4.6 – Teorema di Cook-Levin

SAT è NP-completo.

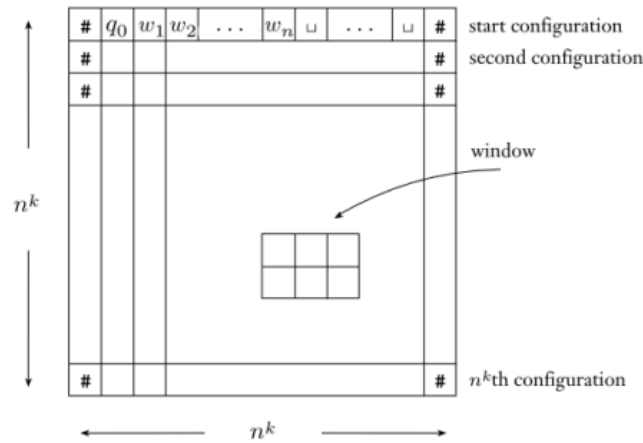
Osservazione. Mostrare che *SAT* appartiene a NP è facile, e lo faremo velocemente. La parte difficile della dimostrazione è far vedere che qualsiasi linguaggio appartenente a NP è riducibile in tempo polinomiale a *SAT*.

Per far ciò, costruiremo una riduzione di tempo polinomiale per ciascun linguaggio A appartenente a NP a *SAT*. La riduzione per A prende una stringa w e produce una formula booleana che simula la macchina NP per A su input w . Se la macchina accetta, ϕ ha un assegnamento che la soddisfa che corrisponde ad una computazione accettante. Se la macchina non accetta, nessun assegnamento soddisfa ϕ . Pertanto, w appartiene ad A se e solo se è soddisfacibile.

Dimostrazione. Prima di tutto, mostriamo che *SAT* appartiene a NP. Una macchina non deterministica di tempo polinomiale può ipotizzare un assegnamento per una data formula ed accettare se l'assegnamento soddisfa ϕ .

Successivamente, prendiamo un qualsiasi linguaggio A appartenente a NP e facciamo vedere che A è riducibile in tempo polinomiale a *SAT*. Sia N una macchina di Turing non deterministica che decide A in tempo n^k per qualche costante k . (Per comodità in effetti assumiamo che N computi in tempo $n^k - 3$).

Un tableau per N su w è una tabella $n^k \times n^k$ le cui righe sono le configurazioni di una diramazione della computazione di N su input w_1 come mostrato nella figura seguente.



Per convenienza nel seguito assumiamo che ciascuna configurazione inizi e finisca con un simbolo $\#$. Pertanto, la prima e l'ultima colonna di un tableau sono tutti $\#$. La prima riga di un tableau è la configurazione iniziale di N su w , e ciascuna riga segue dalla precedente in accordo alla funzione di transizione di N . Un tableau è accettante se qualche riga del tableau è una configurazione accettante.

Ogni tableau accettante per N su w corrisponde ad una diramazione della computazione accettante di N su w . Quindi, il problema di stabilire se N accetta w è equivalente al problema di stabilire se esiste un tableau accettante per N su w .

Passiamo ora alla descrizione della riduzione di tempo polinomiale f di A a SAT. Su input w , la riduzione produce una formula ϕ . Iniziamo descrivendo le variabili di ϕ . Siano Q e Γ l'insieme degli stati e l'alfabeto del nastro di N , rispettivamente. Sia $C = Q \cup \Gamma \cup \{\#\}$. Per ciascun i e j tra 1 ed n^k e per ciascun s in C , introduciamo una variabile, $x_{i,j,s}$.

Ciascuna delle $(n^k)^2$ entrate di un tableau è chiamata cella. La cella in riga i e colonna j viene denotata con $cell[i, j]$ e contiene un simbolo di C . Rappresentiamo i contenuti delle celle con le variabili di ϕ . Se $x_{i,j,s}$ assume il valore 1, significa che $cell[i, j]$ contiene s .

Progettiamo a questo punto ϕ in modo tale che un assegnamento che soddisfa le variabili di ϕ corrisponda ad un tableau accettante per N su w . La formula ϕ è l'AND di quattro parti: $\phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$. Descriviamo ciascuna parte, una per volta.

Come anticipato precedentemente, porre a 1 la variabile $x_{i,j,s}$ corrisponde a posizionare il simbolo s in $cell[i, j]$. La prima cosa che dobbiamo garantire al fine di ottenere una corrispondenza tra un assegnamento ed un tableau è che l'assegnamento ponga ad 1 esattamente una variabile per ciascuna cella. La formula ϕ_{cell} garantisce questo requisito esprimendolo in termini di operazioni booleane:

$$\phi_{cell} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s, t \in C, s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

I simboli $\bigwedge$ e $\bigvee$ stanno per AND ed OR iterati. Per esempio, l'espressione nella formula precedente è un'abbreviazione per

$$\bigvee_{s \in C} x_{i,j,s}$$

è un'abbreviazione per

$$x_{i,j,s_1} \vee x_{i,j,s_2} \vee \dots \vee x_{i,j,s_l}$$

dove $C = \{s_1, s_2, \dots, s_l\}$. Quindi, ϕ_{cell} è in realtà un'espressione grande che contiene un frammento per ciascuna cella nel tableau, poiché i e j variano da 1 a n^k . La prima parte del frammento stabilisce che almeno una variabile assume valore 1 nella cella corrispondente. La seconda parte di ciascun frammento stabilisce che non più di una variabile assume valore 1 (letteralmente, stabilisce che, in ogni coppia di variabili, almeno una assume valore 0) nella cella corrispondente. Questi frammenti sono collegati attraverso operazioni $\wedge$.

La prima parte di ϕ_{cell} all'interno delle parentesi garantisce che almeno una variabile che è associata con ciascuna cella vale 1, laddove la seconda parte garantisce che non più di una

variabile vale 1 per ciascuna cella. Qualsiasi assegnamento alle variabili che soddisfa ϕ (e di conseguenza ϕ_{cell}) deve avere esattamente una variabile ad 1 per ogni cella. Pertanto, qualsiasi assegnamento che soddisfa ϕ specifica un simbolo in ciascuna cella della tabella. Le parti ϕ_{start} , ϕ_{move} , e ϕ_{accept} garantiscono che questi simboli corrispondano realmente ad un tableau accettante come segue.

La formula ϕ_{start} assicura che la prima riga della tabella sia la configurazione iniziale di N su w , richiedendo esplicitamente che le variabili corrispondenti siano ad 1:

$$\phi_{start} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \dots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\sqcup} \wedge \dots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}$$

La formula ϕ_{accept} garantisce che una configurazione accettante sia presente nel tableau. Essa assicura che q_{accept} , il simbolo per lo stato di accettazione, compaia in una delle celle del tableau, richiedendo che una delle variabili corrispondenti sia a 1:

$$\phi_{accept} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{accept}}$$

Infine, la formula ϕ_{move} garantisce che ciascuna riga del tableau corrisponda ad una configurazione che segue legittimamente dalla configurazione della riga precedente, in accordo alle regole di N . Lo fa assicurando che ogni finestra di celle 2×3 sia lecita. Diciamo che una finestra 2×3 è lecita se non viola le azioni specificate dalla funzione di transizione di N . In altre parole, una finestra è lecita se può esser presente quando una configurazione segue correttamente da un'altra.

a	q_1	b
q_2	a	c

a	q_1	b
a	a	q_2

a	a	q_1
a	a	b

#	b	a
#	b	a

a	b	a
a	b	q_2

b	b	b
c	b	b

Le finestre (a) e (b) sono lecite perché la funzione di transizione permette ad N di muovere nella direzione indicata. La finestra (c) è lecita perché, con q_1 presente sul lato destro della riga di sopra, non sappiamo su quale simbolo la testina sia. Detto simbolo potrebbe essere una a , e q_1 potrebbe cambiarlo in una b e muovere a destra. Una tale possibilità genererebbe questa finestra, quindi essa non viola le regole di N . La finestra (d) è ovviamente lecita perché la riga di sopra e quella di sotto sono identiche, situazione che si verifica se la testina non è adiacente alla posizione della finestra. Si noti che # in una finestra lecita può comparire a sinistra o a destra sia nella riga superiore che in quella inferiore. La finestra (e) è lecita perché lo stato q_1 potrebbe essere immediatamente a destra della riga superiore e la macchina, leggendo una b , si sarebbe mossa a sinistra nello stato q_2 che ora compare all'estremità destra della riga inferiore. Infine, la finestra (f) è lecita perché lo stato q_1 potrebbe essere stato immediatamente a sinistra della riga superiore, e la macchina potrebbe aver cambiato la b in una c ed effettuato una mossa a sinistra.

a	b	a
a	a	a

a	q_1	b
q_2	a	a

b	q_1	b
q_2	b	q_2

Nella finestra (a), il simbolo centrale nella riga superiore non può cambiare perché non c'è uno stato ad esso adiacente. La finestra (b) non è lecita perché la funzione di transizione specifica che la b viene cambiata in una c ma non in una a . La finestra (c) non è lecita perché due stati compaiono nella riga inferiore.

Se la riga superiore del tableau è la configurazione iniziale ed ogni finestra nel tableau è lecita, ciascuna riga del tableau è una configurazione che segue legittimamente dalla precedente. Proviamo l'asserto considerando ogni coppia di configurazioni adiacenti nel tableau, indicate come la configurazione superiore e la configurazione inferiore. Nella configurazione superiore, ogni cella che contiene un simbolo di nastro e non è adiacente ad un simbolo di stato rappresenta la cella centrale superiore in una finestra in cui la riga superiore non contiene stati. Pertanto, questo stesso simbolo deve comparire al centro in basso nella finestra. Quindi, esso è presente nella stessa posizione nella configurazione inferiore.

La finestra contenente il simbolo di stato nella cella centrale superiore garantisce che le tre posizioni corrispondenti vengano aggiornate consistentemente tramite la funzione di transizione. Pertanto, se la configurazione superiore è una configurazione lecita, altrettanto lo è la configurazione inferiore, e quella inferiore discende da quella superiore in accordo alle regole di N .

Torniamo ora alla costruzione di ϕ_{move} . Essa garantisce che tutte le finestre nel tableau sono lecite. Ciascuna finestra contiene sei celle, che possono essere impostate in un numero fissato di modi per produrre una finestra lecita. La formula ϕ_{move} stabilisce che le impostazioni delle sei celle devono essere uno di questi modi, ossia

$$\phi_{move} = \bigwedge_{1 \leq i < n^k, 1 \leq j < n^k} (\text{la finestra } (i, j) \text{ è lecita})$$

La finestra (i, j) ha $cell[i, j]$ in posizione centrale in alto. Sostituiamo il testo "la finestra (i, j) è lecita" in questa formula con la formula seguente. Denotiamo il contenuto delle sei celle di una finestra con $a_1, \dots, a_6$.

$$V(x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6})$$

Nel seguito analizziamo la complessità della riduzione per mostrare che essa opera in tempo polinomiale. Per far ciò, esaminiamo la dimensione di ϕ . Prima di tutto, stimiamo il numero di variabili che ha. Si ricordi che il tableau è una tabella $n^k \times n^k$ quindi contiene n^{2k} celle. Ciascuna cella ha l variabili associate ad essa, dove l è il numero di simboli in C . Poiché l dipende soltanto dalla TM N e non dalla lunghezza dell'input n , il numero totale di variabili è $O(n^{2k})$.

Stimiamo la dimensione di ciascuna delle parti di ϕ . La formula ϕ_{cell} contiene un frammento di dimensione fissata della formula per ciascuna cella del tableau, quindi la sua dimensione è $O(n^{2k})$. La formula ϕ_{start} ha un frammento per ciascuna cella nella riga superiore, quindi la sua dimensione è $O(n^k)$. Le formule ϕ_{move} e ϕ_{accept} contengono ciascuna un frammento di dimensione fissata della formula per ciascuna cella del tableau, quindi la loro dimensione è $O(n^{2k})$. Pertanto, la dimensione complessiva di ϕ è $O(n^{2k})$.

6.4.4 Classi coP, coNP e coEXP

Definizione 6.4.6 – Classi coP, coNP e coEXP

Definiamo le classi dei linguaggi coP , $coNP$ e coP come:

$$\begin{aligned} coP &= \{A \in DEC \mid \bar{A} \in P\} \\ coNP &= \{A \in DEC \mid \bar{A} \in NP\} \\ coEXP &= \{A \in DEC \mid \bar{A} \in EXP\} \end{aligned}$$

Teorema 6.4.7 – Chiusura del complemento di P

Date le classi P e coP , si ha che:

$$P = coP$$

In altre parole, la classe P è chiusa nel complemento.

Dimostrazione. Analizziamo la prima implicazione. Dato $A \in P$, sia D il decisore polinomiale tale che $L(D) = A$. Sia D la TM definita come:

$D =$ "Data la stringa w in input:

1. Esegui il programma di D su input w
2. Se l'esecuzione accetta, rifiuta. Altrimenti, accetta"

Per costruzione stessa di $\bar{D}$, abbiamo che:

$$x \in L(\bar{D}) \iff x \notin L(D) = A$$

implicando che $L(\bar{D}) = \bar{A}$. Inoltre, poiché D è un decisore polinomiale, ne segue che anche $\bar{D}$ lo sia, concludendo che $\bar{A} = L(\bar{D}) \in P$ e dunque che $A \in coP$. La seconda implicazione è analoga alla prima.

Teorema 6.4.8 – Chiusura del complemento di EXP

Date le classi EXP e $coEXP$, si ha che:

$$EXP = coEXP$$

In altre parole, la classe EXP è chiusa nel complemento. La dimostrazione è analoga al Teorema 6.4.4.

Corollario 6.4.1 – $P \subseteq coNP \subseteq EXP$

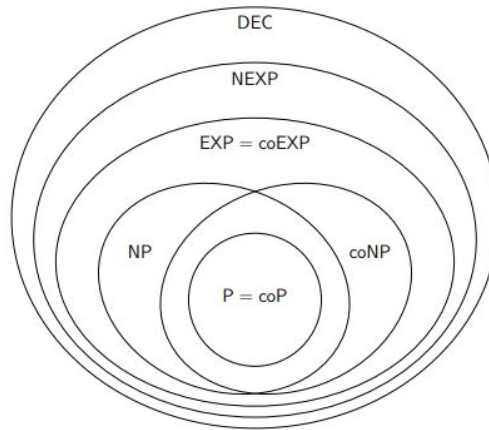
Date le classi P , $coNP$ e EXP , si ha che:

$$P \subseteq coNP \subseteq EXP$$

Dimostrazione. Dato $A \in P$, per i Teoremi 6.3 e 6.4.4, abbiamo che:

- $A \in P \implies A \in P \subseteq NP \implies A \in coNP$
- $A \in coNP \implies A \in NP \subseteq EXP \implies A \in EXP$

implicando che $P \subseteq coNP \subseteq EXP$.



Teorema 6.4.9 – $P = NP \implies P = coNP$

Date le classi P , NP e $coNP$, si ha che:

$$P = NP \implies P = coNP$$

Dimostrazione. Per il corollario precedente, sappiamo che $P \subseteq coNP$. Dato $A \in coNP$, per l'ipotesi e per il Teorema 6.4.4, si ha che:

$$A \in coNP \implies A \in NP = P \implies A \in P$$

concludendo quindi che $P = coNP$.

Corollario 6.4.2 – $NP \neq coNP \implies P \neq NP$

Date le classi P , NP e $coNP$, si ha che:

$$NP \neq coNP \implies P \neq NP$$

Dimostrazione. Tramite il Teorema 6.4.4, si ha che:

$$P = NP \implies P = coNP \implies coNP = NP$$

Dunque, per contronominale, otteniamo che:

$$NP \neq coNP \implies P \neq NP$$

Definizione 6.4.7 – Classe $coNP$ – Complete

Definiamo la classe dei linguaggi $coNP$ – Complete come:

$$coNP - Complete = \{L \in DEC | L \text{ è } coNP - Completo\}$$

dove un linguaggio B è detto $coNP$ – Completo se:

- $B \in coNP$
- $\forall A \in coNPA \leq_m^P B$

Teorema 6.4.10 – Completezza del complemento in NP-Complete

Dato un linguaggio B si ha che:

$$B \in NP - Complete \iff B \in coNP - Complete$$

Dimostrazione. Si ha che:

$$\begin{aligned} B \in NP - Complete &\iff B \in NP, \forall A \in NPA \leq_m^P B \iff B \in NP, \forall A \in NPA \leq_m^P B \iff \\ &B \in coNP, \forall A \in coNPA \leq_m^P B \iff B \in coNP - Complete \end{aligned}$$

Teorema 6.4.11 – Uguaglianza tra NP e $coNP$

Dato un linguaggio $B \in NP - Complete$, si ha che:

$$B \in coNP \iff NP = coNP$$

Dimostrazione. Analizziamo la prima implicazione. Poiché $B \in coNP \iff B \in NP$ e poiché $B \in NP - Complete$, si ha che:

$$A \in NP \implies A \leq_m^P B \iff A \leq_m^P B \implies A \in NP \iff A \in coNP$$

implicando che $coNP \subseteq NP$. Viceversa, si ha che:

$$A \in coNP \implies A \in NP \implies A \leq_m^P B \implies A \in NP$$

implicando che $NP \subseteq coNP$.

Per la seconda implicazione. Se $NP = coNP$, allora $B \in NP - Complete \subseteq NP = coNP$.

Definizione 6.4.8 – Problema dell'insoddisfacibilità

Definiamo il linguaggio del problema dell'insoddisfacibilità come:

$$UNSAT = \{\langle \phi \rangle | \phi \text{ è una formula insoddisfacibile}\}$$

Teorema 6.4.12 – $coNP$ -Completezza di $UNSAT$

Dato il linguaggio $UNSAT$, si ha che $UNSAT \in coNP - Completo$.

Dimostrazione. Per il Teorema di Cook-Levin sappiamo che SAT sia $NP - Completo$. Di conseguenza, per il Teorema 6.4.4 sappiamo che $\overline{SAT}$ sia $coNP$ Completo.

Sia quindi $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F :

$F =$ "Data la stringa x in input:

1. Verifica se $x = \langle \emptyset \rangle$ per qualche formula $\emptyset$.
2. Se vero, restituisci x . Se falso, restituisci la stringa $\langle a \wedge \neg a \rangle$ "

Notiamo che se $x \notin \overline{SAT}$ allora $x \in SAT$ e dunque sicuramente $x = \langle \emptyset \rangle$ per qualche formula soddisfacibile $\emptyset$, implicando che $f(x) = \langle \emptyset \rangle \notin UNSAT$. Invece, se $x \in \overline{SAT}$ allora abbiamo due opzioni:

- Se $x \in \overline{SAT}$ in quanto x non è una formula valida allora $f(x) = \langle a \wedge \neg a \rangle \in UNSAT$ in quanto la formula $a \wedge \neg a$ è insoddisfacibile.
- Se $x \in \overline{SAT}$ in quanto x è una formula valida allora $f(x) = \langle \emptyset \rangle \in UNSAT$ in quanto la formula $\emptyset$ è insoddisfacibile.

Ciò conclude che $x \in \overline{SAT} \iff f(x) \in UNSAT$. Inoltre, la macchina F richiede tempo polinomiale, concludendo che $\overline{SAT} \leq^P mUNSAT$. Infine, poiché $\overline{SAT}$ è $coNP - Completo$ ed è riducibile a $UNSAT$, concludiamo che anche $UNSAT$ sia $coNP - Completo$.

Capitolo 7

Complessità spaziale

Definizione 7.0.1 – Complessità spaziale di una TM e di una NTM

Sia D un decisore. Definiamo come complessità spaziale (o spazio di esecuzione) di D la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ tale che $f(n)$ sia il massimo numero di celle utilizzate da D per processare una stringa di lunghezza n . Analogamente, nel caso delle NTM, $f(n)$ corrisponde al massimo numero di celle utilizzate da ogni ramo dell'esecuzione per processare una stringa di lunghezza n .

Nota: le celle della stringa in input non vengono considerate all'interno del costo spaziale.

Teorema 7.0.1 – Rapporto di spazio tra TM multinastro e TM

Sia $s(n)$ una funzione tale che $s(n) \geq n$. Per ogni TM multinastro M con tempo $s(n)$ esiste una TM M' tale che $L(M) = L(M')$ avente tempo $O(s(n))$.

Osservazione 7.0.1 – TM con input tape e work tape

Poiché le celle della stringa in input non vengono considerate nel costo spaziale, nell'ambito della complessità spaziale consideriamo l'uso esclusivo di TM dotate di due nastri di base:

- *Input tape:* un nastro read-only contenente la stringa in input
- *Work tape:* un nastro read-write su cui la TM lavora

Di conseguenza, all'interno del costo spaziale consideriamo solo le celle utilizzate sul work tape.

Esempio 7.0.1

Consideriamo il seguente linguaggio $L = \{0^n 1^n \mid n \in \mathbb{N}\}$.

Sia M il decisore definito come:

$M = \text{"Data la stringa } w \text{ in input:}$

1. Controlla se w è nella forma 0^*1^* .
2. Sul nastro di lavoro, inizializza un contatore a 0 e torna all'inizio dell'input.
3. Leggendo l'input, incrementa in contatore finché viene letto il simbolo 0, per poi decrementarlo finché viene letto il simbolo 1.
4. Se a fine lettura il contatore è uguale a 0, accetta. Altrimenti, rifiuta"

Innanzitutto, risulta evidente che $L = L(M)$ e che M sia un decisore. Per quanto riguarda il costo spaziale di M , le uniche celle utilizzate dalla TM sono relative al contatore presente sul nastro di lavoro, il quale può facilmente essere realizzato in spazio $O(\log n)$ (si pensi ad

un contatore binario, dunque costo $\log_2 n$, o un contatore decimale, dunque costo $\log_{10} n$). Di conseguenza, il suo spazio computazionale è $O(\log n)$.

Osservazione 7.0.2 – Differenza tra spazio e tempo

Poiché le celle del nastro possono essere riutilizzate, il costo spaziale tende ad essere "più flessibile" rispetto al costo temporale.

Esempio 7.0.2

Consideriamo il seguente linguaggio $L = \{ww^R \mid w \in \Sigma^*\}$

Sia M il decisore definito come:

$M =$ "Data la stringa w in input:

1. Calcola $|w| = n$
2. Ripeti la seguente istruzione per $i = 1, \dots, n$:
3. Verifica se $w_i = w_{n+1-i}$. Se falso, rifiuta
4. Accetta."

Innanzitutto, risulta evidente che $L = L(M)$ e che M sia un decisore. Analizziamo quindi il costo spaziale di M .

Il calcolo di $|w| = n$ può essere realizzato tramite un contatore, dunque ha costo $O(\log n)$. Essendo i un contatore, esso ha costo $O(\log n)$. Per calcolare $n + 1 - i$ ad ogni iterazione, utilizziamo la seguente procedura:

1. Copio n su un nastro secondario e i su un nastro terziario, richiedente $2O(\log n)$ spazio
2. Incremento di 1 il nastro contenente n , riutilizzando le celle precedenti
3. Decremento il nastro contenente $n + 1$ e il nastro contenente i finché non si ha che $i = 0$, riutilizzando le celle precedenti

Di conseguenza, il costo spaziale di M risulta essere:

$$s(n) = O(\log n) + 2O(\log n) = O(\log n)$$

Definizione 7.0.2 – Classi DSPACE e NSPACE

Data la funzione $s : \mathbb{N} \rightarrow \mathbb{R}^+$, definiamo come classe dei linguaggi decidibili in spazio $O(s(n))$ il seguente insieme:

$$DSPACE(s(n)) = \{L \in DEC \mid L \text{ decidibile da una TM in spazio } O(s(n))\}$$

Analogamente, definiamo come classe dei linguaggi decidibili non deterministicamente in spazio $O(s(n))$ il seguente insieme:

$$NSPACE(s(n)) = \{L \in DEC \mid L \text{ decidibile da una NTM in spazio } O(s(n))\}$$

Teorema 7.0.2 – PATH decidibile in $O(\log^2 n)$

Il linguaggio PATH è decidibile in $O(\log^2 n)$, ossia $PATH \in DSPACE(\log^2 n)$.

Dimostrazione. Sia $G = (V, E)$ un grafo. Consideriamo la seguente procedura ricorsiva definita su G :

$Path?(x, y, k)$:

1. Se $k = 0$, verifica se $(x, y) \in E$ o se $x = y$. Se vero, accetta. Altrimenti, rifiuta

2. Se $k > 0$, ripeti la seguente istruzione per ogni nodo v in V :

- (a) Se $\text{Path?}(x, v, k-1)$ accetta e $\text{Path?}(v, y, k-1)$ accetta, allora la procedura accetta. Altrimenti, essa rifiuta

Per costruzione stessa di Path? , si ha che:

$$\text{Path?}(x, y, k) \text{ accetta} \iff \text{Esiste cammino } x \rightarrow y \text{ di massimo } 2^k \text{ nodi}$$

Sia quindi M la TM definita come:

$M = \text{"Data la stringa } \langle G, s, t \rangle \text{ in input;}$

1. Esegui la procedura $\text{Path?}(s, t, \lceil \log n \rceil)$.
2. Se la procedura accetta, accetta. Altrimenti, rifiuta"

Notiamo quindi che:

$$\begin{aligned} \langle G, s, t \rangle \in L(M) &\iff \text{Path?}(s, t, \lceil \log n \rceil) \text{ accetta} \iff \\ &\iff \text{Esiste cammino } s \rightarrow t \text{ con massimo } 2^{\lceil \log n \rceil} \text{ nodi} \iff \\ &\iff \text{Esiste cammino } s \rightarrow t \text{ con massimo } n \text{ nodi} \iff \langle G, s, t \rangle \in \text{PATH} \end{aligned}$$

implicando che $L(M) = \text{PATH}$.

Le due chiamate della procedura necessitano di memorizzare i due nodi in input, richiedendo $2O(\log n) = O(\log n)$ spazio. Tale spazio è riutilizzabile ad ogni iterazione del ciclo e ad ogni ricorsione. Inoltre, l'altezza dell'albero di ricorsione corrisponde a $\lceil \log n \rceil = O(\log n)$.

Di conseguenza, concludiamo che il costo spaziale di M sia:

$$s(n) = O(\log n)O(\log n) = O(\log^2 n)$$

concludendo che $\text{PATH} = L(M) \in \text{DSPACE}(\log^2 n)$.

7.1 Rapporto tra spazio e tempo

Teorema 7.1.1 – Tempo limitante lo spazio

Data una funzione $f(n)$ tale che $f(n) \geq n$, si ha che:

$$\begin{aligned} \text{DTIME}(f(n)) &\subseteq \text{DSPACE}(f(n)) \\ \text{NTIME}(f(n)) &\subseteq \text{NSPACE}(f(n)) \end{aligned}$$

Dimostrazione. Sia D una TM tale che $L(D) \in \text{DTIME}(f(n))$. Poiché il tempo è $O(f(n))$, il numero massimo di passi che la TM può effettuare sono $O(f(n))$. Di conseguenza, anche se la TM utilizzasse una nuova cella ad ogni passo, il massimo numero di celle utilizzabili sarebbe $O(f(n))$, implicando che $L(D) \in \text{DSPACE}(f(n))$. Per argomento analogo, concludiamo anche che $\text{NTIME}(f(n)) \subseteq \text{NSPACE}(f(n))$.

Teorema 7.1.2 – Spazio limitante il tempo

Data una funzione $f(n)$ tale che $f(n) \geq \log n$, si ha che:

$$\text{DSPACE}(f(n)) \subseteq \text{DTIME}(2^{O(f(n))})$$

Dimostrazione. Sia M una TM tale che $L(M) \in \text{DSPACE}(f(n))$. Per comodità, assumiamo che M sia dotata di un solo work tape.

Indichiamo le configurazioni di M con la seguente notazione:

$$\text{WORK}_i \text{TAPE}_j$$

dove: *WORKTAPE* è il contenuto attuale del work tape; q_i è lo stato attuale e il simbolo alla sua destra è il simbolo su cui si trova la testina del work tape; j indica che la testina dell'input tape si trova attualmente sulla j -esima cella.

Sia $t(n)$ il tempo massimo di computazione di M . Poiché M è un decisore, le configurazioni di ogni computazione non possono ripetersi, poiché altrimenti essa andrebbe in loop infinito. Di conseguenza, $t(n)$ coinciderà con il numero massimo di configurazioni possibili. Per via della notazione utilizzata per descrivere le configurazioni di M , tale numero massimo di configurazioni corrisponde al numero di disposizioni possibili per una configurazione, ossia:

$$t(n) = |\Gamma|^{f(n)} |Q|n$$

Inoltre, poiché per ipotesi si ha che:

$$f(n) \geq \log n \implies 2^{f(n)} \geq n$$

ne segue che:

$$t(n) = |\Gamma|^{f(n)} |Q|n \leq |\Gamma|^{f(n)} |Q|2^{f(n)} = 2^{O(f(n))}$$

concludendo che $L(M) \in DTIME(2^{O(f(n))})$.

7.2 Teorema di Savitch

Teorema 7.2.1 – Teorema di Savitch

Data una funzione $f(n) \geq \log n$, si ha che:

$$NSPACE(f(n)) \subseteq DSPACE(f^2(n))$$

Dimostrazione. Sia N una NTM tale che $L(N) \in NSPACE(f(n))$.

Per comodità, assumiamo che in N vi sia un solo stato accettante q_{accept} (abbiamo visto in capitoli precedenti come ogni NTM possa essere modificata senza alterare il suo linguaggio). Siano inoltre q_{start} e δ rispettivamente lo stato iniziale di N e la sua funzione di transizione. Consideriamo la notazione $WORK_{q_i}TAPE_j$ per indicare le configurazioni di N (come nella dimostrazione 7.1). Di conseguenza, il numero massimo di configurazioni durante una qualsiasi computazione di N corrisponde a $2^{O(f(n))}$.

Consideriamo quindi il grafo $G_{N,w} = (V, E)$ definito come:

- Ad ogni nodo di V corrisponde una configurazione possibile di N durante la computazione di w ;
- Per ogni nodo $v_i, v_j \in V$, esiste un arco se e solo se la computazione può passare dalla configurazione c_i alla configurazione c_j tramite δ .

Per costruzione di $G_{N,w}$, si ha che:

$$w \in L(N) \iff \text{Esiste cammino } c_{start} \rightarrow c_{accept} \text{ in } G_{N,w}$$

Consideriamo quindi la procedura $Path_{G_{N,w}}?$, una versione modificata della procedura $Path?$, dove il grafo $G_{N,w}$ viene costruito durante la ricorsione stessa.

Sia quindi M la TM definita come:

$M = \text{"Data la stringa } w \text{ in input:}$

1. Esegui la procedura $Path_{G_{N,w}}?(c_{start}, c_{accept}, [\log m])$
2. Se la procedura accetta, accetta. Altrimenti, rifiuta"

Per costruzione stessa di M , si ha che:

$$\begin{aligned} w \in L(M) &\iff Path_{G_{N,w}}?(c_{start}, c_{accept}, [\log m]) \text{ accetta} \iff \\ &\iff \text{Esiste cammino } c_{start} \rightarrow c_{accept} \text{ in } G_{N,w} \iff w \in L(N) \end{aligned}$$

implicando che $L(M) = L(N)$.

Notiamo quindi che la costruzione parziale del grafo richieda solo qualche "variabile" di appoggio, mantenendo il costo della chiamata $\text{Path}_{G_{N,w}}(c_{\text{start}}, c_{\text{accept}}, [\log m])$ inalterato, ossia pari a $O(\log^2 m)$, dove m è la dimensione del grafo in input.

Di conseguenza, poiché la dimensione di $G_{N,w}$ risulta essere $m = 2^{O(f(n))}$, il costo spaziale di M risulta essere:

$$s(n) = O(\log^2 m) = O(\log^2(2^{O(f(n))})) = O(f^2(n))$$

concludendo che $L(N) = L(M) \in DSPACE(f^2(n))$

7.3 Classe PSPACE

Definizione 7.3.1 – Classi PSPACE e NPSPACE

Definiamo la classe dei linguaggi decidibili in spazio polinomiale come:

$$PSPACE = \bigcup_{k=1}^{\infty} DSPACE(n^k)$$

Analogamente, definiamo la classe dei linguaggi decidibili non-deterministicamente in spazio polinomiale come:

$$NPSPACE = \bigcup_{k=1}^{\infty} NSPACE(n^k)$$

Definizione 7.3.2 – Classi EXPSPACE e NEXPSPACE

Definiamo la classe dei linguaggi decidibili in spazio esponenziale come:

$$EXPSPACE = \bigcup_{k=1}^{\infty} DSPACE(2^{n^k})$$

Analogamente, definiamo la classe dei linguaggi decidibili non-deterministicamente in spazio esponenziale come:

$$NEXPSPACE = \bigcup_{k=1}^{\infty} NSPACE(2^{n^k})$$

Teorema 7.3.1 – Rapporto tra spazio e tempo

Dato un alfabeto Σ , si ha che:

$$P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP \subseteq EXPSPACE$$

Dimostrazione. Tramite il Teorema 7.1, si ha che:

$$P \subseteq NP \subseteq PSPACE \quad EXP \subseteq NEXP \subseteq EXPSPACE$$

Tramite il Teorema 7.1, invece, si ha che:

$$PSPACE \subseteq EXP$$

Teorema 7.3.2 – $PSPACE = NPSPACE$

Date le classi $PSPACE$ e $NPSPACE$:

$$PSPACE = NPSPACE$$

Dimostrazione. Per definizione stessa, si ha che $PSPACE \subseteq NPSPACE$. Inoltre, per il Teorema di Savitch, si ha che:

$$L \in NPSPACE \implies \exists k \in \mathbb{N} | L \in NSPACE(n^k) \subseteq DSPACE(n^{2k}) \implies L \in PSPACE$$

implicando che $NPSPACE \subseteq PSPACE$.

Teorema 7.3.3 – $EXPSPACE = NEXPSPACE$

Date le classi $EXPSPACE$ e $NEXPSPACE$:

$$EXPSPACE = NEXPSPACE$$

(dimostrazione analoga al Teorema 7.3).

Definizione 7.3.3 – Classi $coPSPACE$ e $coEXPSPACE$

Definiamo le classi dei linguaggi $coPSPACE$ e $coEXPSPACE$ come:

$$\begin{aligned} coPSPACE &= \{A \in DEC | \bar{A} \in PSPACE\} \\ coEXPSPACE &= \{A \in DEC | \bar{A} \in EXPSPACE\} \end{aligned}$$

Teorema 7.3.4 – Chiusura del complemento di $PSPACE$

Date le classi $PSPACE$ e $coPSPACE$, si ha che:

$$PSPACE = coPSPACE$$

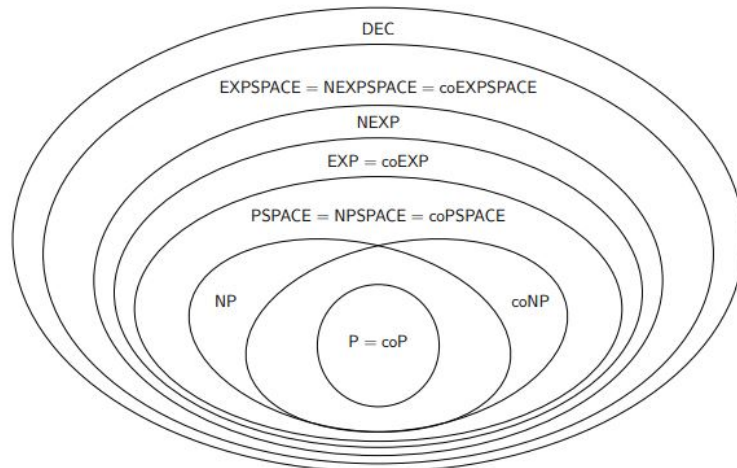
In altre parole, la classe $PSPACE$ è chiusa nel complemento (dimostrazione analoga al Teorema 6.4.4).

Teorema 7.3.5 – Chiusura del complemento di $EXPSPACE$

Date le classi $EXPSPACE$ e $coEXPSPACE$, si ha che:

$$EXPSPACE = coEXPSPACE$$

In altre parole, la classe $EXPSPACE$ è chiusa nel complemento (dimostrazione analoga al Teorema 6.4.4).



7.4 Classe L

Definizione 7.4.1 – Classi L e NL

Definiamo la classe dei linguaggi decidibili in spazio logaritmico come:

$$L = DSPACE(\log n)$$

Analogamente, definiamo la classe dei linguaggi decidibili non-deterministicamente in spazio logaritmico come:

$$NL = NSPACE(\log n)$$

Corollario 7.4.1 – Rapporto tra L e P

Date le classi L , NL e P , si ha che:

$$L \subseteq NL \subseteq P$$

(segue dal Teorema 7.1).

Osservazione 7.4.1 – $NL = L$?

Tramite il Teorema di Savitch, abbiamo che:

$$NL = NSPACE(\log n) \subseteq DSPACE(\log 2n)$$

ma non sappiamo se $NL = L$. Spesso si dice che $NL \subseteq L^2$.

A differenza delle classi NP e $NEXP$, nel caso della classe NL la definizione alternativa tramite verificatore ha bisogno di un piccolo cambiamento al fine di poter rispettare i vincoli di spazio:

- Vincolando il certificato ad una lunghezza logaritmica otteniamo un sottoinsieme troppo ristretto di problemi in quanto il certificato risulta troppo piccolo, dunque l'insieme dei problemi verificabili in spazio logaritmi con un certificato logaritmico non coinciderebbe con NL .
- Mantenendo il certificato di lunghezza polinomiale, esso può essere utilizzato come memoria aggiuntiva: potremmo leggere più volte la stessa informazione evitando di doverla memorizzare nel work tape, risparmiando una notevole quantità di spazio.

In altre parole, nel primo caso otteniamo dei verificatori troppo deboli, mentre nel secondo sono troppo potenti. Per ottenere una via di mezzo tra le due cose, possiamo tuttavia imporre che il certificato sia read-once: il verificatore possiede un nastro aggiuntivo contenente il certificato le cui celle possono essere lette una sola volta (dunque la testina non può spostarsi a sinistra). In tal modo, preserviamo la possibilità che il certificato sia di lunghezza polinomiale ed impediamo che esso possa essere utilizzato come memoria aggiuntiva.

Teorema 7.4.1 – Classe NL (definizione alternativa)

Data la classe NL , si ha che:

$$NL = \{A \in DEC \mid \exists V \text{ verif. con cert. read-once in spazio logaritmico per } A\}$$

In altre parole, possiamo definire NL anche come la classe dei linguaggi verificabili in spazio logaritmico con un certificato read-once.

Dimostrazione. Analizziamo la prima implicazione. Dato $A \in NL$, sia N la NTM tale che $A = L(N)$. Durante la sua computazione, N effettuerà una serie di scelte, creando così il suo albero di computazione. Sia V la TM definita come:

$V = \text{"Data la stringa } \langle w, c \rangle \text{ in input:}$

1. Interpreta c come una serie di scelte da effettuare
2. Simula N su input w eseguendo le scelte dettate da c leggendole in sequenza

3. Se la simulazione accetta, V accetta. Altrimenti, rifiuta"

Per costruzione stessa di V , si ha che:

$$w \in L = L(N) \iff \text{Esiste un ramo di } N \text{ che accetta } w \iff \exists c \in \Sigma^*, \langle w, c \rangle \in L(V)$$

dunque V risulta essere un verificatore con certificato read-once di $L = L(N)$.

Inoltre, poiché $A = L(N) \in NL$, lo spazio massimo utilizzato da ogni ramo dell'albero di computazione risulta essere $O(\log n)$ e poiché la lettura del certificato non influenza lo spazio, concludiamo che V sia un verificatore in spazio logaritmico.

Per la seconda implicazione. Dato un linguaggio A , supponiamo che esista un verificatore con certificato read-once V che verifica A in spazio logaritmico, implicando che:

$$\exists k \in \mathbb{N} | V \text{ verifica } L \text{ in spazio } O(\log n)$$

Sia N la NTM definita come:

$N = \text{"Data la stringa } \langle w \rangle \text{ in input:}$

1. Scegli non deterministicamente una stringa c
2. Simula V su input $\langle w, c \rangle$
3. Se la simulazione accetta, B accetta. Altrimenti, rifiuta"

Per costruzione stessa di N , si ha che

$$w \in A \iff \exists c \in \Sigma^*, \langle w, c \rangle \in V \iff \text{Esiste un ramo di } N \text{ che accetta } w \iff x \in A$$

implicando che $A = L(N)$.

Notiamo che le prime due operazioni di N possano essere svolte in contemporanea, generando il prossimo simbolo della stringa ad ogni passo di computazione. Inoltre, poiché V è un verificatore read-once in spazio logaritmico, non è sufficiente mantenere memorizzate le celle precedentemente generate. Dunque, è sufficiente memorizzare un solo simbolo per volta, richiedendo spazio $O(\log n)$.

Dunque, concludiamo che $A = L(N) \in NL$.

Definizione 7.4.2 – Classe coL

Definiamo la classe dei linguaggi coL come:

$$coL = \{A \in DEC | \bar{A} \in L\}$$

Teorema 7.4.2 – Chiusura del complemento di L

Date le classi L e coL , si ha che:

$$L = coL$$

In altre parole, la classe L è chiusa nel complemento (dimostrazione analoga al Teorema 6.4.4).

7.5 Riduzione in spazio logaritmico

Definizione 7.5.1 – Riducibilità in spazio logaritmico

Dati due linguaggi A e B , diciamo che A è riducibile in spazio logaritmico a B tramite mappatura, indicato come $A \leq_m^L B$, se esiste una funzione calcolabile $f : \Sigma^* \rightarrow \Sigma^*$, detta riduzione

in spazio logaritmico da A a B , tale che:

$$w \in A \iff f(w) \in B$$

e f è calcolabile in spazio logaritmico.

Teorema 7.5.1 – Decidibilità logaritmica tramite riduzione

Dati due linguaggi A e B tali che $A \leq_m^L B$, si ha che:

$$B \in L \implies A \in L$$

Dimostrazione. Dato $B \in DEC$, sia D_B il decisore tale che $L(D_B) = B$. Sia D_A la TM definita come:

$D_A =$ "Data la stringa w in input:

1. Calcola $f(w)$
2. Esegui il programma di D_B con input $f(w)$.
3. Se l'esecuzione accetta, D accetta. Altrimenti, D rifiuta"

Per costruzione stessa di D_A , si ha che:

$$w \in L(D_A) \iff f(w) \in L(D_B) = B \iff w \in A$$

implicando che $L(D_A) = A$.

Tuttavia, notiamo che, poiché $f(w)$ potrebbe richiedere spazio $O(\log n)$, è necessario eseguire le prime due operazioni in contemporanea, calcolando il prossimo simbolo di $f(w)$ ad ogni passo di computazione di D_B . Inoltre, poiché D_B computa in spazio logaritmico, concludiamo che $A = L(D_A) \in L$.

Teorema 7.5.2 – Verificabilità logaritmica tramite riduzione

Dati due linguaggi A e B tali che $A \leq_m^L B$, si ha che:

$$B \in NL \implies A \in NL$$

(dimostrazione analoga al Teorema 7.5).

Teorema 7.5.3 – Riducibilità logaritmica transitiva

Dati tre linguaggi $A, B, C \subseteq \Sigma^*$, si ha che:

$$A \leq_m^L B, B \leq_m^L C \implies A \leq_m^L C$$

In altre parole, la relazione $\leq_m^L$ è transitiva.

Dimostrazione. Siano $f, g : \Sigma^* \rightarrow \Sigma^*$ le due funzioni calcolabili polinomialmente per cui rispettivamente si ha che $A \leq_m^L B$ e $B \leq_m^L C$.

Sia quindi H la TM definita come:

$H =$ "Data la stringa w in input:

1. Calcola $f(w)$. Sia k l'output di tale calcolo.
2. Calcola $g(k)$ e restituisci l'output"

Risulta evidente che H sia la TM che calcola la funzione $g \circ f$ e che:

$$w \in A \iff f(w) \in B \iff g(f(w)) = (g \circ f)(w) \in C$$

Eseguendo contemporaneamente le due operazioni, ossia il prossimo simbolo di $f(w)$ assieme al prossimo simbolo di $g(k)$, è sufficiente utilizzare $2O(\log n)$ spazio, concludendo che $A \leq_m^L C$.

Teorema 7.5.4 – Riducibilità logaritmica complementare

Dati due linguaggi A e B , si ha che:

$$A \leq_m^L B \iff \bar{A} \leq_m^L \bar{B}$$

(dimostrazione analoga al Teorema della riducibilità complementare).

7.6 Classe NL-Complete

Definizione 7.6.1 – Classe NL-Complete

Definiamo la classe dei linguaggi *NL – Completi* come:

$$NL - Complete = \{L \in DEC \mid L \text{ è } NL - Completo\}$$

dove un linguaggio B è detto *NL – Completo* se:

- $B \in NL$
- $\forall A \in NL \ A \leq_m^L B$

Teorema 7.6.1 – NL-Completezza di PATH

Dato il linguaggio *PATH*, si ha che $PATH \in NL - Complete$.

Dimostrazione. Sia N la NTM definita come:

$N = \text{"Data la stringa } \langle G, s, t \rangle \text{ in input, dove } G = (V, E) \text{ è un grafo:}$

1. Se $s = t$, accetta
2. Calcola $|V| = m$
3. Inizializza $currNode = s$ su un nastro secondario
4. Ripeti le seguenti istruzioni per $i = 1, \dots, m$
 - (a) Seleziona non deterministicamente un nodo $u \in V$
 - (b) Se $(currNode, u) \notin E$, rifiuta
 - (c) Se $u = t$ accetta, altrimenti poni $currNode = u$.
5. Rifiuta"

Per costruzione stessa di N , si ha che:

$$\langle G, s, t \rangle \in L(N) \iff \text{Esiste cammino } s \rightarrow t \text{ in } G \iff \langle G, s, t \rangle \in PATH$$

implicando che $L(N) = PATH$.

Inoltre, poiché i cammini vengono letti nodo per nodo senza tenere traccia dei precedenti, lo spazio richiesto risulta essere $O(\log n)$, concludendo che $PATH = L(N) \in NL$.

Sia quindi $A \in NL$ e sia N' una NTM tale che $A = L(N')$ in spazio $O(\log n)$. Sia inoltre $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

$F = \text{"Data la stringa } w \text{ in input:}$

1. Costruisci il grafo $G_{N,w}$ delle configurazioni della computazione di N su input w come nella dimostrazione del Teorema 7

2. Restituisci in output la stringa $\langle G_{N,w}, c_{start}, c_{accept} \rangle$

Notiamo che:

$$w \in L(N') \iff \text{Esiste cammino } c_{start} \rightarrow c_{accept} \text{ in } G_{N,w} \iff f(w) = \langle G_{N,w}, c_{start}, c_{accept} \rangle \in PATH$$

Inoltre, poiché la computazione contemporanea di $f(w)$ assieme a $\langle G_{N,w}, c_{start}, c_{accept} \rangle \in PATH$ richiede spazio $O(\log n)$, concludiamo che $A \leq_m^L PATH$.

7.6.1 Teorema di Immerman-Szelepcsényi

Lemma 7.6.1 – Uguaglianza tra NL e coNL

Dato un linguaggio $B \in NL - Complete$, si ha che:

$$B \in coNL \iff NL = coNL$$

(dimostrazione analoga al Teorema 6.4.4).

Teorema 7.6.2 – Teorema di Immerman-Szelepcsényi

Date le classi NL e coNL, si ha che:

$$NL = coNL$$

In altre parole, la classe NL è chiusa nel complemento.

Dimostrazione. Poiché $PATH \in NL - Complete$, tramite il lemma precedente abbiamo che:

$$\overline{PATH} \in NL \iff PATH \in coNL \iff NL = coNL$$

Vogliamo quindi dimostrare che $\overline{PATH}$ sia in NL. L'idea dietro la seguente dimostrazione risiede nella possibilità di abusare della lunghezza polinomiale del certificato: non possiamo leggere più volte le stesse celle, ma possiamo inserire nel certificato un numero adeguato di copie delle celle precedenti.

Sia $m = |V(G)|$ e siano $R_0, \dots, R_m$ gli insiemi dei vertici raggiungibili da s con massimo $i \in [0, m-1]$ passi:

$$R_i = \{v \in V(G) \mid s \rightarrow v \text{ con massimo } i \text{ archi}\}$$

Assumiamo inoltre che i vertici di G siano identificati da un numero.

Per ogni vertice $v \in V(G)$, verificare che $v \in R_i$ risulta molto facile: è sufficiente un certificato $\langle v_0, \dots, v_k \rangle$ che descriva il cammino, affinché il verificatore possa svolgere la seguente procedura:

`certify_v_in_Ri` ($\langle v_0, \dots, v_k \rangle, i, v$):

1. Inizializza un contatore $h = 0$
2. Verifica se $v_0 = s$. Se vero, incrementa h , altrimenti la procedura rifiuta.
3. Verificare che per ogni $0 < j < k$ è vero che $(v_{j-1}, v_j) \in E(G)$. Se vero, incrementa h , altrimenti la procedura rifiuta.
4. Verifica se $v_k = v$. Se vero incrementa h (a questo punto avremo che $h = k$), altrimenti la procedura rifiuta.
5. Verifica se $h \leq i$. Se vero, la procedura accetta, altrimenti rifiuta.

Tramite tale ordine delle operazioni, la lettura del certificato risulta essere read-once. Inoltre, poiché i vertici di G sono identificati da un numero, la lunghezza massima di tale certificato risulta essere $O(m \log n)$.

La procedura riportata sopra, tuttavia, ci permette solo di certificare che $v \in R_i$, ma non che $v \notin R_i$. Tale operazione risulta difatti più complessa, la questione risulta più complessa, ma può essere strutturata con certificati ricorsivi.

Supponiamo di sapere che $|R_i| = \ell_i$ e di voler verificare che $v \notin R_i$. Consideriamo il sotto-certificato $\langle c_{u_1}, \dots, c_{u_{\ell_i}} \rangle$, dove c_{u_k} è a sua volta un sotto-certificato che verifica se $u_k \in R_i$. Inoltre, visto che i vertici di G sono identificati da un numero, affinché ogni sotto-certificato sia diverso è sufficiente verificare che $u_1 < \dots < u_{\ell_i}$. Definiamo quindi la seguente procedura:

`certify_v_not_in_Ri_with_|Ri|` ($\langle c_{u_1}, \dots, c_{u_{\ell_i}} \rangle, i, v, \ell_i$):

1. Inizializza $w = 0$ e $k = 1$.
2. Ripeti fino a fine certificato:
 - (a) Verifica se $u_k \in R_i$ tramite la procedura `certify_v_in_Ri` (c_{u_k}, i, u_k). Se la procedura rifiuta, anche questa procedura rifiuta.
 - (b) Se $w \neq 0$, verifica se $w < u_k$. Se vero, poni $w = u_k$ e incrementa k , altrimenti rifiuta.
 - (c) Verifica $u_k \neq v$. Se falso, rifiuta.
3. Verifica se $k = \ell_i$. Se vero accetta, altrimenti rifiuta.

Tramite tale ordine delle operazioni, anche questo certificato risulta essere read-once. Inoltre, poiché per ogni i abbiamo che $|R_i| \leq m$, il certificato contiene massimo m sotto-certificati, i quali sappiamo avere lunghezza $O(m \log n)$. Per tanto, questo certificato ha lunghezza $O(m^2 \log n)$.

Supponiamo invece di sapere che $|R_{i-1}| = \ell_{i-1}$ e di voler verificare che $v \notin R_i$. Similmente al caso precedente, consideriamo il sotto-certificato $\langle c_{u_1}, \dots, c_{u_{\ell_{i-1}}} \rangle$, dove c_{u_k} questa volta è un sotto-certificato che verifica se $u_k \in R_{i-1}$. La procedura risulta essere analoga alla precedente, ma con una differenza fondamentale: verifichiamo che nessuno dei vertici in R_{i-1} sia un vertice adiacente a v .

`certify_v_not_in_Ri_with_|Ri-1|` ($\langle c_{u_1}, \dots, c_{u_{\ell_{i-1}}} \rangle, i, v, \ell_{i-1}$):

1. Inizializza $w = 0$ e $k = 1$.
2. Ripeti fino a fine certificato:
 - (a) Verifica se $u_k \in R_i$ tramite la procedura `certify_v_in_Ri` ($c_{u_k}, i - 1, u_k$). Se la procedura rifiuta, anche questa procedura rifiuta.
 - (b) Se $w \neq 0$, verifica se $w < u_k$. Se vero, poni $w = u_k$ e incrementa k , altrimenti rifiuta.
 - (c) Verifica se $w \neq v$ e che $w \neq u$ per ogni vertice adiacente a v . Se falso, rifiuta.
3. Verifica se $k = \ell_{i-1}$. Se vero accetta, altrimenti rifiuta.

Ovviamente, anche questo certificato risulta essere read-once e la sua lunghezza è $O(m^2 \log n)$. A questo punto, è necessario definire l'ultimo step: sapendo che $|R_{i-1}| = \ell_{i-1}$, vogliamo verificare che $|R_i| = \ell_i$. Consideriamo il sotto-certificato $\langle c_{v_0}, \dots, c_{v_{m-1}} \rangle$. Per ogni vertice v di G abbiamo solo due opzioni: il vertice appartiene a R_i , dunque possiamo verificare ciò tramite un certificato da dare alla prima procedura, oppure esso non appartiene a R_i , dunque sapendo $|R_{i-1}|$ possiamo verificare ciò tramite un certificato da dare alla terza procedura.

`certify_|Ri-1|_with_|Ri-1|` ($\langle c_{v_0}, \dots, c_{v_{m-1}} \rangle, i, \ell_i, \ell_{i-1}$):

1. Inizializza un contatore $h = 0$.
2. Ripeti per $k = 0, \dots, m - 1$:
 - (a) Verifica se $v_k \in R_i$ tramite la procedura `certify_v_in_Ri` (c_{v_k}, i, v_k). Se la procedura accetta, incrementa h .
 - (b) Altrimenti, verifica se $v_k \notin R_i$ tramite la procedura `certify_v_not_in_Ri_with_|Ri-1|` ($c_{v_k}, i, v_k, \ell_{i-1}$). Se la procedura rifiuta, anche questa procedura rifiuta.

(c) Se $h = \ell_i$ allora accetta, altrimenti rifiuta.

Tale procedura dunque non fa altro che calcolare la cardinalità di R_i , per poi verificare se essa sia uguale al valore ℓ_i dato. Intuitivamente, notiamo che il certificato sia composto da m altri sotto-certificati ognuno di lunghezza $O(m^2 \log n)$. Per tanto, questo certificato risulta avere lunghezza $O(m^3 \log n)$.

Una volta definite tutte le procedure necessarie, definiamo la seguente TM V :

$V = \text{"Data la stringa } \langle \langle G, s, t \rangle, c \rangle \text{ in input:}$

1. Interpreta $c = \langle \ell_0, \dots, \ell_{m-1}, c_0, \dots, c_{m-1}, c_t \rangle$
2. Ripeti per $i = 0, \dots, m - 1$:
 - (a) Esegui la procedura `certify_ $|R_{i-1}|$ _with_ $|R_{i-1}|$` ($c_i, i, \ell_i, \ell_{i-1}$). Se la procedura rifiuta, anche questa procedura rifiuta.
3. Esegui la procedura `certify_v_not_in_ $R_i$ _with_ $|R_m|$` (c_t, m, t, ℓ_m). Se la procedura accetta, anche questa procedura accetta, altrimenti rifiuta.

In altre parole, la TM V utilizza i valori $\ell_0, \dots, \ell_{m-1}$ e i sotto-certificati $c_0, \dots, c_{m-1}$ per arrivare a certificare che $|R_m| = \ell_m$, per poi utilizzare il sotto-certificato c_t per determinare se $t \in R_m$, concludendo quindi che V sia un verificatore per $PATH$.

Poiché ognuna delle procedure risulta essere read-once e il certificato finale è composto da m interi ognuno necessitante $\log n$ bit, $m + 1$ certificati ognuno necessitante $O(m^3 \log n)$ bit e poiché m è in $O(n)$, concludiamo che il costo spaziale del verificatore V sia:

$$O(m \log n + (m + 1)(m^3 \log n)) = O(n^4 \log n)$$

e dunque che $PATH \in NL$.

Bibliografia

1. Michael Sipser, *Introduzione alla teoria della computazione*, 3^a edizione, Maggioli Editore, 2016.
Traduzione dell'opera originale *Introduction to the Theory of Computation*.